

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

August 28, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	August 28, 2026	110 pages, 6 appendix pages

Tiivistelmä — Referat — Abstract

Context. Radio Access Network (RAN) control planes are written mostly in C++ for performance, at a well-known cost in memory safety and concurrency. Rust promises comparable performance with compile-time safety, but its fit for this latency-critical setting is unproven.

Objective. The thesis compares modern C++ and Rust on a UE Context Management service modelled on a real Nokia gNB-CU-CP component. Average-throughput comparisons leave four dimensions open: tail latency and jitter under overload, the C++ defect classes Rust prevents at compile time, the overhead of its safety model, and the cost of incremental adoption.

Methods. One language-agnostic specification is frozen first, then implemented twice with identical algorithms, concurrency model, allocator, and build settings. A synthetic workload is swept to overload, both versions pass the same parity tests, and the evaluation adds fault injection and an FFI prototype.

Results. Safe Rust rejects all five injected memory-safety and data-race defect classes at compile time. In C++ the same five build cleanly and show up only at runtime, if a sanitiser catches them. Five further classes fail in both languages. On two of them C++ tooling reports what Rust does not. The seam to C++ leaves the core `unsafe`-free and costs at most 10.91% of one operation. Performance changes at saturation. Below it median latency matches, yet an end-to-end call spends only 0.250% on language work. Cross-thread handoff takes the rest, so the sweep reflects the design more than the language. Timed alone, Rust's operation is 1.6× faster and the process is a few MiB larger. Above saturation the order flips: C++ holds 1.7× the throughput of Rust in 20/20 interleaved repeats with both allocators. The far tail below saturation cannot be settled on a virtualised host.

Conclusions. The main cost of adopting Rust is engineering effort and the learning curve, not run-time performance at nominal load. The gain is freedom from data races, not from concurrency defects in general. Overload narrows that claim: a service that must degrade gracefully depends on the standard library's synchronisation primitives, not on the safety model, and a nominal-load test would not expose the gap.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

List of Figures **iv**

List of Tables **v**

1 Introduction **1**

1.1 Motivation and industrial context 2
1.2 Problem statement 4
1.3 Research questions and hypotheses 5
1.4 Contributions, results, and limitations 6
1.5 Significance and beneficiaries 7

2 Background: Systems Languages for Latency-Critical Software **8**

2.1 RAN control-plane systems and the gNB-CU-CP 8
2.2 Latency, throughput, and predictability in real-time software 9
2.3 C++ for latency-critical systems 10
2.4 The Rust ownership and safety model 12
2.5 Concurrency models for control-plane software 14
2.6 Interoperability between C++ and Rust 15
2.7 Research gap 16

3 Problem Context: UE Context Management and Predictability **18**

3.1 The UE Context Management subsystem 18
3.2 Memory-safety and concurrency defect classes 19
3.3 Tail latency and predictability in RAN 22

4 Research Methodology **24**

4.1 Research approach 24
4.2 Methods for the four research questions 25
4.2.1 Performance measurement under a load sweep (RQ1, RQ3) 26
4.2.2 Controlled fault injection (RQ2) 27
4.2.3 Foreign-function-interface migration case study (RQ4) 28

4.3 Fairness as a controlled-variable strategy . . . 28
4.4 Statistical treatment of measurements . . . 30
4.5 Reproducibility and research artifacts . . . 32

5 Design and Implementation 34
5.1 Shared specification: the fairness baseline . . . 34
5.2 Conceptual model and concurrency design . . . 34
5.3 Architectural choices and technology stack . . . 35
5.4 Implementation details . . . 36
5.4.1 The UE context and the state machine . . . 36
5.4.2 Sharding, the store, and the bounded queue . . . 37
5.4.3 Requests, responses, and the service lifecycle . . . 38
5.4.4 Forced idioms, and what the two cores cost . . . 39
5.5 Public API and operation semantics . . . 42
5.6 Parity verification . . . 43

6 Experimental Setup and Evaluation 44
6.1 Testbed configuration . . . 44
6.2 Workload and evaluation measurements . . . 45
6.3 Evaluation data . . . 47
6.4 Evaluation results . . . 49
6.4.1 Below saturation (RQ1, RQ3) . . . 49
6.4.2 Past saturation (RQ1) . . . 57
6.4.3 Resource use (RQ3) . . . 60
6.4.4 What the machine decided, and what the code decided . . . 62
6.4.5 Safety via fault injection (RQ2) . . . 67
6.4.6 Migration (RQ4) . . . 72

7 Discussion 78
7.1 Summary of findings . . . 78
7.2 Discussion on the research questions . . . 80
7.3 Revisiting the hypotheses . . . 85
7.4 Engineering guidelines for incremental adoption . . . 86
7.5 Related work . . . 88
7.6 Threats to validity and mitigations . . . 93
7.7 Ethical and professional reflection . . . 98

8 Conclusion and Future Work 100

Statement on the Use of AI Tools **103**

Bibliography **105**

A Abbreviations and key terms

B Literature search protocol

C Statistical design: family membership and resolving power

D Host and campaign configuration

E Fault-injection example

List of Figures

2.1 Where the studied subsystem sits in the 5G NG-RAN 8
2.2 Why the tail, not the mean, governs RAN deadlines 9
2.3 Coordinated omission under open- and closed-loop load 10
2.4 The same use-after-free in C++ and in safe Rust 13
2.5 Two concurrency models for control-plane state 15
2.6 What the neighbouring studies control, and what they leave varying 17

3.1 UE-context lifecycle state machine 18
3.2 The overload cliff 23

5.1 The end-to-end path of a single operation 35
5.2 Shared-nothing, thread-per-shard architecture 35
5.3 The shared UE-context data model 37

6.1 The two workloads, and the lifecycle the mixed one walks 46
6.2 The tested family drawn as paired effects 51
6.3 The end-to-end operation decomposed, against what the sweep can resolve 55
6.4 The load sweep: tail latency and achieved throughput 57
6.5 Latency across the percentile range, and how far it resolves 58
6.6 Divergence past saturation 59
6.7 Resource use, and the two effects inside peak RSS 62
6.8 Where the guarantee stops: the outcome matrix 72
6.9 The C++-to-Rust FFI migration seam 73
6.10 The seam's cost on one operation, against its own uncertainty 74

7.1 Where Rust's compile-time guarantee stops 83

List of Tables

2.1 How each language enforces the properties measured here 14

3.1 The injected catalogue and its incarnations in UECM 22

4.1 Research design: how each research question is answered 25
4.2 The controlled-variable strategy 29

5.1 The forced-idiom log 40
5.2 Size and cyclomatic-complexity proxies for the two cores 41
5.3 Where the line-count difference between the cores comes from 41

6.1 Per-operation latency at the representative load 50
6.2 Inter-operation jitter at the representative load 51
6.3 Exact paired permutation test of C++ against Rust 52
6.4 How large each effect could be 53
6.5 Where an end-to-end operation's cost actually goes 54
6.6 What each variant sustains past saturation 59
6.7 Exact paired test past saturation 59
6.8 Throughput through the contended synchronisation primitives alone 60
6.9 Resource use at the representative load 62
6.10 The resident set decomposed by the allocator 65
6.11 Defect-class taxonomy: measured outcomes 71
6.12 FFI migration prototype: seam size, safety, and cost 73
6.13 Adoption-cost axes: build time, static footprint, third-party surface 75

7.1 Summary of findings per research question 79
7.2 This study beside the five vendor theses 92
7.3 Threats to validity, their mitigations, and what remains 95

A.1 Abbreviations and key terms i

D.1 The three measurement hosts ii
D.2 Configuration of the service, the workload, and the campaign ii

1 Introduction

Mobile networks are moving from specialised hardware to software on ordinary servers. Functions that once ran on fixed-purpose, vertically integrated equipment are now re-implemented as programs on general-purpose machines. The Radio Access Network (RAN), long the most hardware-bound part of the stack, is part of that shift.

In 5G, the base station (gNB) is split into a central unit and one or more distributed units. The central unit is split again into control-plane and user-plane functions [1, 2]. Open-RAN pushes the same idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces [3]. The control plane that results, the gNB-CU-CP, is a large, stateful, highly concurrent program. It tracks every connected device, runs RRC signalling [4], and coordinates mobility and handovers, all within strict latency limits.

Load at a busy site is high. A single site can run thousands of connection-management and mobility procedures per second, each with a budget of a few milliseconds [5]. A deployed control plane may track millions of active devices at once. As the RAN moves to cloud-native, disaggregated deployments, the correctness, reliability, and timing of this software increasingly set the quality of the whole network. An outage or a tail-latency spike here is not a local defect. Every device the site serves is affected [6, 7].

This control-plane software is, today, written mostly in C++. The choice is a rational one: C++ offers the low-level control, mature tooling, and performance that latency-critical systems demand. But that performance comes with a well-known cost. C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [8, 9]. Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time, with performance intended to be competitive with C++ [10, 11]. Whether that promise holds for a telecom-grade, latency-critical control plane, where predictability under overload matters as much as raw speed, is not well understood. This thesis, carried out at Nokia, addresses the question directly: it develops two functionally equivalent implementations of a representative RAN subsystem, a UE Context Management service modelled on a real Nokia gNB-CU-CP component, in modern C++ and in Rust, and compares them under a realistic synthetic signalling workload.

Three terms recur throughout. The *gNB-CU-CP* is the control-plane software this work is about. *UE Context Management (UECM)* is the specific subsystem reimplemented and measured. *Tail latency*, the p99 and p99.9 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines [6, 7]. Appendix A fixes these and the rest of the domain vocabulary for readers outside telecom.

1.1 Motivation and industrial context

Three trends make this comparison timely. The first is the cloud-native evolution of the RAN itself. Moving control-plane functions onto general-purpose servers raises, rather than lowers, the premium on software quality: the software is longer-lived, evolves continuously, runs at higher concurrency, and must meet its deadlines without the determinism that purpose-built hardware once provided. This shift is accelerated by Open-RAN disaggregation, which turns the base station into virtualised, multi-vendor software connected by open interfaces [3], so that the control plane a vendor ships increasingly competes on software-engineering quality (reliability, predictable timing, security, and the ease with which a large codebase can be evolved safely) and no longer on purpose-built hardware. In this setting a latency regression or a memory-corruption defect is not a localised bug but a network-wide reliability and security risk, and a language-selection question stops being academic and becomes a concrete engineering decision.

The second trend is that AI is moving from a tool used to tune the network into a workload the network is built to carry, and the next generation is being specified that way from the start. IMT-2030, the ITU framework for 6G, lists artificial intelligence and communication among its six usage scenarios and ubiquitous intelligence among its four design principles [12]. The architecture forming around that is AI-RAN, in which telecommunications and AI workloads are orchestrated together on shared, accelerated infrastructure, so the sites that run the radio also host distributed edge AI platforms [13]. The AI-RAN Alliance states the operational requirement directly: one common infrastructure has to run both kinds of workload concurrently, and their service-level agreements may differ [14]. That changes what a language choice has to buy for a control plane. Its software no longer has the host to itself. It contends for CPU, memory, and cache with inference work whose arrival pattern it does not control, and a co-tenant that takes capacity away lowers the load at which the control plane saturates. Saturation is where the two languages separate most sharply in the measurements of Chapter 6. Memory footprint and behaviour past that point are becoming selection criteria in their own right, and neither is settled by the architecture above the code.

The third trend is an industry-wide shift toward memory-safe languages. Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them [8, 9, 15, 16]. The case no longer rests on vendor reporting alone: a group spanning academia and industry has argued that memory safety now needs standardised terminology and requirements if it is to be specified and procured at all [17]. This concern has risen to the level of national policy: cybersecurity agencies now urge software manufacturers to publish "memory-safe roadmaps" and to migrate away from memory-unsafe languages such as C and C++ [18, 19]. In the market this subsystem is sold into it has also become a regulatory matter. The EU Cyber Resilience Act places secure-by-design and lifecycle vulnerability-handling obligations on manufacturers of products with digital elements, with the vulnerability-reporting obligations applying from September 2026 and most of the remainder from December 2027 [20]. It names no programming language, and it would be wrong to read it as requiring one. What it changes is that the recurring defect classes of a memory-unsafe language become a compliance exposure as well as an engi-

neering one. Rust in particular has moved from a niche language into operating-system kernels, cloud infrastructure, and increasingly into telecommunications. Production hypervisors such as AWS Firecracker, which underpins its serverless platform, are already written in Rust [21]. Yet the evidence base for Rust's suitability rests largely on general-purpose software and on average-case performance benchmarks. The specific question a telecom vendor must answer, whether Rust can meet the tail-latency and predictability requirements of a RAN control plane, whether it genuinely prevents the defect classes that plague the existing C++ code, and what it costs to adopt it incrementally inside a large C++ codebase, has not been answered with direct, controlled evidence. Providing such evidence for a representative subsystem is the motivation for this work.

Industrial setting. This thesis is carried out at Nokia, one of the small number of vendors that build, maintain, and evolve production RAN software end to end. That market is concentrated in very few hands: the five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high [22, 23]. The subsystem it studies, UE Context Management (UECM), is modelled on a real gNB-CU-CP control-plane component rather than a textbook example.

The gNB-CU-CP is not a minor component. In the 3GPP disaggregated architecture it is the control-plane half of the central unit, hosting RRC and managing the UE context lifecycle [1, 4], under the millisecond-scale procedure budgets the 5G system places on control signalling [5] and at the event rates and connected-device counts a deployed network sustains. Section 2.1 places it in the wider architecture. Because it is written in C++, it inherits both that language's performance and its memory-safety and concurrency hazards. That is precisely where the three trends above converge, and software of this class is shipped, evolved, and operated across live operator networks, so the language-selection decision is a production concern rather than a hypothetical one.

UECM is a deliberate choice of study object within that setting, for the reasons Section 3.1 sets out: it is small enough to reimplement faithfully and reason about in full, yet it carries the properties that make control-plane code hard. The implementation is driven entirely by a public, frozen specification rather than by proprietary code, so the comparison stays reproducible by readers outside Nokia.

A decision with a generational horizon. A language choice in software of this class is not revisited often, and it outlives the generation it was made for. The next one is being specified now: 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, against an ITU calendar that asks for technology proposals in early 2029 and the submitted specifications by mid-2030 [24]. That work explicitly covers migration from and interworking with 5G, so a control plane written for the current generation is maintained alongside the next one rather than retired when it arrives. Nor do the next generation's objectives relax what motivates this comparison. IMT-2030 names security and resilience as one of its four design principles, expects the system to be secure by design, defines resilience as the ability to keep operating correctly during and after a disruptive event, and carries the ultra-reliable low-latency scenario forward in

strengthened form [12]. Freedom from whole classes of defect and predictable behaviour under disturbance are being written into the objectives of the next generation rather than left to implementers.

The policy dates named above fall inside the same window. CISA and the FBI classify the development of new product lines in a memory-unsafe language as a bad practice where the product serves critical infrastructure, a scope mobile-network infrastructure sits squarely inside, and they exempt an existing product from the memory-safe roadmap expectation only when its announced support ends before January 2030 [25]. A vendor that defers the language question until the next generation is specified will not thereby have avoided making it. It will have made it by default, in the incumbent language, for another decade of deployments.

What this thesis contributes to that decision is bounded, and the boundary is worth stating plainly. The measurements here are taken on a subsystem modelled on a current-generation component and evaluated against requirements that are already published. No claim is made about requirements that have not yet been written. What crosses the generational boundary is not a latency figure but the language-level comparison itself: which defect classes a compiler can rule out, what the safety model costs, and what incremental adoption inside a large C++ codebase involves. Those questions have the same answers whichever generation the specification above them names, and they are cheapest to answer before the next codebase is started rather than after.

A concrete failure mode. To make the stakes concrete, consider a mobility storm in which many UEs hand over at once. A single mishandled pointer, a UE context freed on release while an in-flight signalling message still references it, produces a use-after-free. In C++ this compiles cleanly and may pass ordinary testing, only to corrupt an unrelated UE's state or crash the worker thread under precisely the high-concurrency conditions the system exists to handle. The same latent race can equally show up as an unpredictable tail-latency spike instead of an outright crash. In safe Rust the same design does not compile: ownership and borrowing reject the dangling reference at build time. This one scenario captures why the thesis measures both *safety* (which defects are prevented, and when) and *predictability* (tail latency under overload) rather than average throughput alone.

1.2 Problem statement

Two properties of this domain make the language choice consequential. First, manual memory and concurrency management in C++ exposes the software to defects such as use-after-free, double-free, data races, and iterator invalidation, which are silent at compile time and costly to find and fix at telecom scale. Second, what governs whether RAN deadlines are met is not average throughput but worst-case and tail behaviour under load and overload [6], a property that is easy to regress and hard to measure correctly. Rust promises to remove the first class of defect at compile time while claiming performance competitive on the second, but that claim rests mostly on general-purpose benchmarks rather than on stateful, concurrent, latency-critical telecom workloads.

Bringing these threads together, the concrete problem this thesis addresses is the following. Telecom vendors are under growing pressure, both technical and increasingly regulatory, to adopt memory-safe languages, yet for the RAN control plane they lack *direct, controlled* evidence on which to base that decision. It is not established whether a Rust implementation of a stateful, concurrent control-plane subsystem can match modern C++ in *tail latency and predictability under overload* (as opposed to average throughput), whether Rust's compile-time guarantees actually eliminate the specific defect classes that affect the existing C++ code, and what incremental adoption *costs* inside a large C++ codebase. The available evidence is largely general-purpose and average-case [26, 27], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem. Answering them under controlled, reproducible conditions is the problem this thesis sets out to solve.

1.3 Research questions and hypotheses

The thesis is organised around four research questions, ordered to put predictability and migration ahead of raw average performance:

RQ1 How do Rust and modern C++ compare in latency predictability (median, p99, p99.9 and p99.99 tail latency, and the maximum) for a RAN control-plane workload under increasing load and overload?

RQ2 Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage and cost are the remaining ones detected?

RQ3 What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

RQ4 What are the practical implications (effort, FFI interoperability, and risk) of adopting Rust incrementally within an existing C++ RAN software ecosystem?

The questions are approached with three prior expectations, stated here so that the evaluation can confirm or refute them instead of merely describing results after the fact. **H1:** for this workload the two languages are performance-equivalent in the operationally relevant region below saturation, with any difference confined to the far tail. **H2:** safe Rust prevents at compile time the memory-safety and concurrency defect classes that C++ leaves to best-effort, after-the-fact detection. **H3:** the run-time cost of C++-to-Rust interoperability is negligible relative to the cost of the operations invoked, so that the true cost of incremental adoption is engineering effort rather than performance. H2 is stated so that it can fail. The defect catalogue behind RQ2 spans classes safe Rust is expected to prevent and classes it is expected not to, so the study can locate the boundary of the guarantee instead of only confirming it. These hypotheses are revisited against the evidence in Section 7.3.

1.4 Contributions, results, and limitations

The thesis makes six contributions. Two are artifacts: a reproducible benchmarking framework for telecom control-plane workloads, and a frozen shared specification with two parity-tested implementations of a RAN control-plane subsystem, one in C++ and one in Rust. Three are results. The first is a comparison of latency and resource behaviour that divides at the saturation knee, where the ordering between the two languages reverses. The second is a defect-class taxonomy from controlled fault injection that marks where the compile-time guarantee stops, in both directions. The third is a bound on what a comparison of this shape can attribute to a language at all, obtained by decomposing an end-to-end operation into the work the language performs and the cross-thread handoff around it. The last is a set of engineering and migration guidelines for language selection in cloud-native RAN evolution (Section 7.4). Neither the reversal nor the attribution bound was anticipated. Both were found while reading results the study had planned to report differently, and Section 7.3 records what the first of them cost H1.

The findings divide three ways. On *safety* (RQ2), every injected memory-safety and data-race defect class is prevented at compile time in safe Rust. The same defects all compile in C++, and each is caught, if at all, only at run time and only under the right sanitiser, or on one class under none of them. Seven boundary classes were chosen deliberately, and neither language prevents them. On two of those the C++ tooling is ahead, and on one the defect that safe Rust rules out returns as soon as the program writes `unsafe`. On *migration* (RQ4), the FFI seam is small and keeps the core free of `unsafe`. Measured against the same operation called natively it costs at most 10.91% of one operation, so the real adoption cost is API design and not run-time overhead. Three costs beyond the seam are measured for the same reason, and two of them come back against the received wisdom: most of the Rust build's extra cost is compiling third-party code and not the compiler's own work, and the binary-size ordering depends on which linkage policy the two toolchains are held to rather than on the languages. Both of these results are final.

On *performance* (RQ1, RQ3) the answer divides twice, and drawing both divisions correctly is where most of the analysis went. The first division separates what the sweep measures from what it can attribute. An end-to-end operation in this design is 0.250% language work and a cross-thread handoff takes the rest, so a matched-pair comparison on that path bounds the design and not the language. The limit holds whichever way such a comparison comes out, and it reaches any study that reports an end-to-end median without measuring the share. Timed apart from the handoff, the operation is 1.6× faster in Rust, which the sweep cannot see.

The second division is by regime. Below saturation the two implementations are equivalent end to end in central behaviour, by a margin no deployment would notice. The paired test does resolve a median difference of 1.3 μs in Rust's favour under both allocators, on an end-to-end round-trip of 70 μs, so the equivalence is practical and not statistical. The far tail in the same regime resolves on the reported host and favours Rust. Pooled over 3 360 051 to 3 360 084 operations per variant, the two languages' p99.99 values do not meet, and the widest gap between variants is 87 μs. A second bare-metal campaign at another shard count agrees on the direction (Section 6.4.4). Past saturation the ordering reverses,

and where each implementation stops meeting its offered rate is itself a difference: the Rust arms track to 448 kops/s and the C++ arms to 392. Deeper still, where all four have collapsed, the C++ build sustains 1.7× the throughput of the Rust build in 20/20 interleaved repeats under both allocators, and the Rust build spends 1.8× the CPU on each operation it does complete, which is what rules out the reading that it is simply doing less. On resources the Rust process holds a few MiB more resident memory, again under both allocators, in the same direction and the same order of magnitude on all three machines measured. How it compares against the cost of the allocator choice is not: the two are comparable on bare metal and the allocator's was an order of magnitude larger on the virtualised host this study used first, so the comparison between them is a property of the machine rather than of the allocators.

Four limitations bound all of this: a synthetic workload, one simplified subsystem, single-author implementation bias, and an allocator footprint whose size moved with the host. Chapter 7 treats each in detail.

The scope is drawn deliberately to protect the internal validity of the language comparison. In scope are the UE Context Management functionality of Chapter 3, the shared-nothing concurrency design, the load sweep to overload, the fault-injection catalogue, and the FFI prototype. Explicitly out of scope are the full gNB system and its remaining control-plane domains, PHY/L2 processing and the data-heavy user plane, real captured traffic traces (as opposed to a spec-justified synthetic workload), managed-language alternatives (excluded on the tail-latency grounds argued in Chapter 2), and production deployment and operational concerns such as upgrade, observability, and multi-tenancy. These exclusions trade external validity for the controlled conditions a fair language comparison needs, and each is revisited as a limitation in Chapter 7.

1.5 Significance and beneficiaries

The results are intended to be directly actionable. For RAN vendors and cloud-native network operators, the thesis supplies controlled evidence on the question that gates any language decision. That question is not "is Rust fast on average?" but "does Rust hold its tail latency and predictability under overload, and what does incremental adoption cost?", answered for a subsystem representative of production control-plane code. For the standards and policy discussion around memory-safe roadmaps [18, 19], it grounds a general mandate in a concrete telecom data point. For the research community, the frozen specification, the parity-tested implementations, the reproducible benchmarking framework, and the defect-class taxonomy form a reusable baseline for further language and systems studies, and the latency decomposition gives such a study a way to check what its own measurements can attribute. And for practising engineers, the fault catalogue and the migration prototype double as worked examples of what Rust's guarantees do and do not buy in this domain.

2 Background: Systems Languages for Latency-Critical Software

2.1 RAN control-plane systems and the gNB-CU-CP

The NG-RAN architecture splits the gNB into a central unit (gNB-CU) and distributed units, with the control plane (gNB-CU-CP) handling RRC and connection management for each UE [1, 2, 4]. Such control-plane systems must satisfy demanding non-functional requirements: very high event rates (UE mobility, signalling storms), strict and predictable latency bounds that the 5G service and RAN specifications quantify as KPIs [5, 28], long-lived software with continuously evolving complexity, and heavily concurrent workloads. In these systems the worst-case and tail behaviour, rather than the average, determines whether deadlines are met, which motivates the predictability focus of this thesis.

Functionally, the gNB is disaggregated along two independent axes. The first is a *central/distributed* split: a central unit (gNB-CU) hosts the higher layers of the protocol stack, while one or more distributed units (gNB-DU) host the lower, more time-critical layers close to the radio. The second is a *control/user-plane* split inside the central unit, separating the control-plane function (gNB-CU-CP) from the user-plane function (gNB-CU-UP). These parts communicate over standardised interfaces: NG towards the 5G core, F1 between the CU and the DU, and E1 between the CU-CP and the CU-UP [1, 2, 29]. Open-RAN initiatives standardise these interfaces further so that each part can be supplied by a different vendor and run as software on commodity hardware [3]. Figure 2.1 places the subsystem studied in this thesis, UE Context Management (UECM), inside the gNB-CU-CP: it owns the per-UE control-plane state and mediates the RRC signalling and mobility procedures, which is why its correctness and timing behaviour are representative of the control plane as a whole.

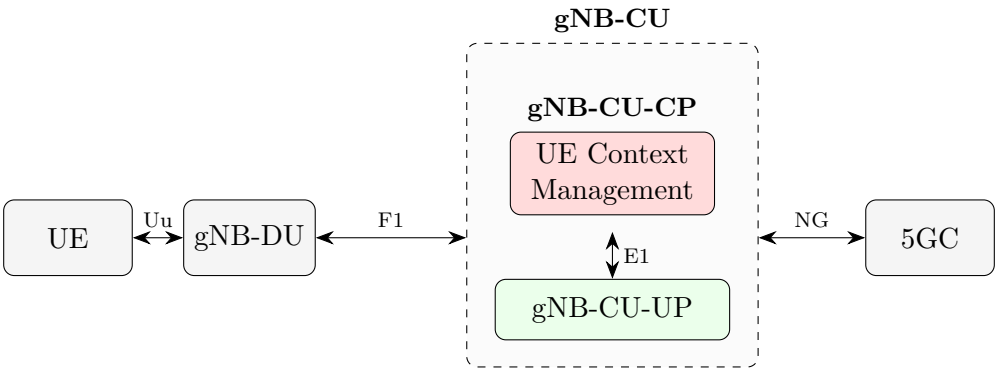


Figure 2.1: Where the studied subsystem sits in the 5G NG-RAN. The gNB is split into a distributed unit (gNB-DU) and a central unit (gNB-CU), and the central unit is split again into control-plane (gNB-CU-CP) and user-plane (gNB-CU-UP) functions, connected by the standardised F1, E1, and NG interfaces. UE Context Management (highlighted) is the per-UE control-plane state manager inside the gNB-CU-CP that this thesis reimplements and measures.

2.2 Latency, throughput, and predictability in real-time software

Latency-critical systems are judged less by how much work they do on average than by how *predictably* they do it. Three metrics must be distinguished. *Throughput* is the sustained rate of completed operations, *latency* is the time an individual operation takes, and *jitter* is the variability of that latency. A system can have excellent mean latency and still miss deadlines often. The meaningful figures are therefore the high-percentile *tail* latencies, the 99th (p99) and 99.9th (p99.9) percentiles, rather than the mean. At the scale of a network, tail latency dominates user-visible behaviour: when a single user action fans out into many internal operations, the slowest of them sets the response time, so rare per-operation delays become common per-request delays [6]. Modern data-centre and networking software increasingly operates in a regime where the relevant latencies are measured in microseconds, a range in which conventional operating-system and software overheads are no longer negligible [7]. Fine-grained measurement studies show that, even at moderate utilisation, hardware, operating-system, and application-level effects such as background interference, scheduling, interrupt routing, and power-saving states inflate the tail far beyond what queueing models predict [30]. Figure 2.2 sets the two effects side by side: the distance between the median and the high percentiles, and the fan-out that turns a delay which is rare per operation into one that is common per procedure.

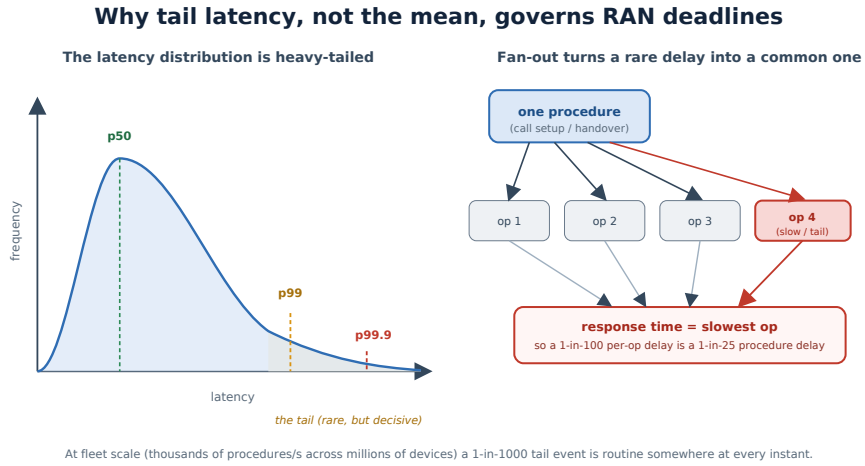


Figure 2.2: Why the tail, not the mean, governs RAN deadlines. On the left, most operations finish quickly and a few take far longer, so the p99 and p99.9 percentiles sit well to the right of the median. On the right, a procedure such as a call setup runs several operations and finishes only when the slowest one does, so a delay hitting one operation in a hundred reaches a much larger share of procedures. At fleet scale a one-in-a-thousand delay is happening somewhere at every instant.

Real-time systems are conventionally graded by the consequence of a missed deadline: in *hard* real-time a miss is a failure, in *firm* real-time a late result is useless but not catastrophic, and in *soft* real-time a miss merely degrades quality. A RAN control plane is best seen as soft-to-firm real-time: an occasional late signalling response degrades service rather than endangering it, but at fleet scale and under signalling storms such degradations accumulate into a network-wide reliability problem. This is compounded by queueing behaviour: as offered load approaches service capacity, queueing delay grows

sharply and the latency tail explodes near saturation. Characterising a system therefore requires driving it not just at nominal load but *into overload*, which is why the evaluation in this thesis sweeps load up to and beyond the saturation point.

Measuring tail latency correctly is itself a pitfall. A naive closed-loop load generator that waits for each response before issuing the next request suffers *coordinated omission*: when the system stalls, the generator stalls with it and simply fails to record the many requests that would have arrived during the stall, systematically hiding the very tail it is meant to expose. Figure 2.3 contrasts the two generator models. The distinction beneath this is a modelling one, not an implementation detail. In a closed system a new request is triggered only by the completion of a previous one, whereas in an open system arrivals are independent of completions, and the two produce systematically different response-time behaviour on the same service process, with the closed model reporting the lower figures [31]. Sound measurement therefore requires an open-loop model with an intended schedule, correction for coordinated omission, high-dynamic-range latency histograms that retain resolution across many orders of magnitude, and statistically rigorous reporting with repeated runs and confidence intervals rather than single point estimates [32]. These techniques are applied in Chapter 6.

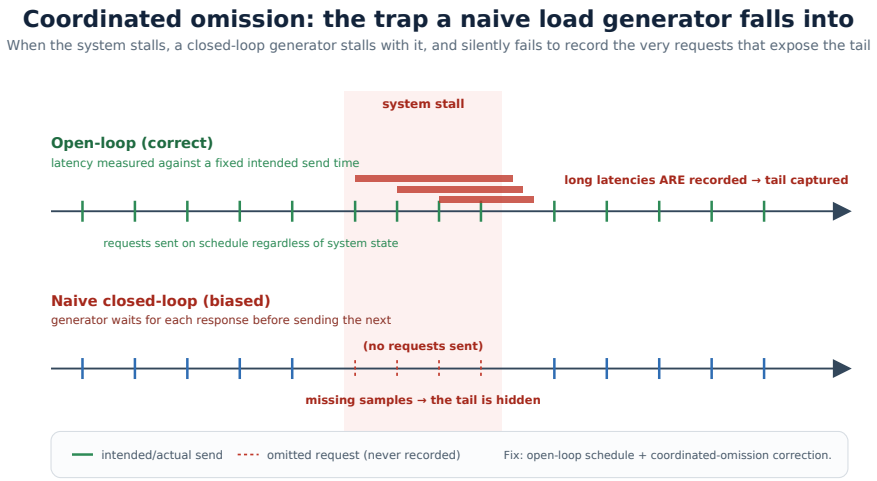


Figure 2.3: Coordinated omission. The open-loop generator at the top sends on a fixed timetable even while the system is stalled, and times each request from the moment it was due, so stalled requests are recorded with their full delay and the tail shows up. The closed-loop generator at the bottom sends the next request only after the previous reply arrives. When the system stalls it stops sending too, so the slowest requests are never issued and never measured, and the tail vanishes from the data.

2.3 C++ for latency-critical systems

C++ is the default language for RAN control-plane software for good reasons. It offers zero-overhead abstractions, direct control over memory layout and allocation, deterministic destruction through the Resource Acquisition Is Initialization (RAII) idiom, and mature compilers, libraries, and profiling tools, all without a garbage collector that could inject unpredictable pauses. Modern C++ (C++11 and later) adds smart pointers (`std::unique_ptr`, `std::shared_ptr`) that automate ownership, move semantics that avoid needless copies, and a formal memory model with `std::atomic` for lock-based and

lock-free concurrency. These features make it possible to write fast and, with discipline, reasonably safe systems code.

The difficulty is that the compiler does not enforce this safety. The programmer does. RAII and smart pointers reduce lifetime errors without eliminating them. A raw pointer or reference can outlive the object it points to, a container mutation can invalidate an iterator, an object can be used after it has been moved from, and nothing in the type system stops a mutable object being shared and written by several threads at once. The result is a familiar catalogue of defects, use-after-free, double-free, buffer overflow, data races, deadlocks, and iterator invalidation, that are easy to introduce and expensive to find at telecom scale. Memory-corruption errors of this kind have been a persistent security problem for decades, and deployed mitigations remain only partial defences [33]. Their prevalence is well documented in industry. Memory-safety issues account for roughly 70 % of the security vulnerabilities reported at Microsoft [8], and Google reports a sharp drop in such vulnerabilities as memory-safe languages are adopted in Android, to below a fifth of that platform's total vulnerabilities by 2025 [9, 15]. This specific catalogue is developed in detail in Chapter 3 and forms the basis for the fault-injection study in Chapter 6.

C++ practice mitigates these defects with tooling rather than with language guarantees, and the tooling is inherently partial. Dynamic sanitisers such as AddressSanitizer (ASan), ThreadSanitizer (TSan), and UndefinedBehaviorSanitizer detect many errors, but only on the code paths that a given test actually executes with the right inputs, and they impose runtime and memory overheads that make them unsuitable for production. TSan in particular reports only those data races that happen to occur during a run, so a race that manifests only under a rare interleaving can pass unnoticed. Its deadlock detector inherits that limitation from the technique it implements, which infers deadlock potentials from the lock-acquisition graph of a single trace [34], and it ships as an experimental component covering only the POSIX mutex families [35]. Static analysers help but suffer false negatives and cannot reason about the whole program. Detection is therefore best-effort and after the fact, which is precisely the situation a language with compile-time guarantees is meant to change.

That describes C++ as it can be deployed today, and the language is not standing still. Since 2023 the standards committee has pursued *profiles*: named sets of rules covering type, bounds, lifetime, and arithmetic safety that a conforming implementation must enforce wherever they are enabled [36]. That direction was chosen over *Safe C++*, a proposal for a safe subset with borrow checking and its own standard library, which is Rust's model expressed in C++ [37]. The comparison drawn here nonetheless stays live. Profiles are not yet standardised, and in January 2026 their own authors were still calling for implementations and treating the design of the initial profiles as work still to be done [38]. The concurrency profile, the one that would bear on the defect classes catalogued in Chapter 3, is placed after that initial set and not within it: "more profiles should be defined and implemented, notably a concurrency profile" [38]. A rule enforced only when it is switched on is also a different kind of thing from a property the type system carries unconditionally, especially where the enforcement is a run-time check and not a compile-time rejection. The implementations compared here are therefore built against C++23 as specified and shipped, and Section 7.6 records what that pins the comparison

to.

2.4 The Rust ownership and safety model

Rust enforces memory safety and data-race freedom at compile time through its ownership and borrowing model [10], and ships with modern tooling in Cargo and integrated testing. The type system statically forbids mutation of shared state and confines the unavoidable exceptions to `unsafe` code kept behind safe APIs [39].

These guarantees rest on more than assertion. The soundness of the safe fragment of the language, including the role of `unsafe` as an explicitly marked escape hatch, has been established formally by the RustBelt project [11], and the aliasing discipline that even `unsafe` code must respect has been given a precise operational semantics [40]. Empirical evidence supports the model in practice: in real codebases `unsafe` is used sparingly and in localised ways [27, 41], and a study of all Rust memory-safety CVEs found that every one of them but a single compiler bug involved `unsafe` code [42]. The caveat is that unsafety propagates through dependencies. Fewer than a third of libraries use `unsafe` directly, but a much larger fraction cannot be fully statically checked because it hides somewhere in their call chain [43], which is a further argument for keeping the safety-critical core of a subsystem `unsafe`-free, as this thesis does.

One property matters most in a latency-critical setting. This safety comes from the type system and the borrow checker, not from runtime instrumentation, and is intended to be *zero-cost*. Systems research argues Rust attains safety without the overhead of garbage collection or runtime checks, and even enables efficiency techniques that are impractical in C [26]. Whether that zero-cost claim holds for a telecom control-plane workload is precisely the property this thesis evaluates.

Concretely, the borrow checker rejects at compile time the very use-after-free sketched in the introduction: a program that frees a context while a reference to it is still live does not build. Figure 2.4 puts that defect in its UE Context Management form beside the C++ version, which compiles and is caught, if at all, only at runtime.

The same discipline extends to concurrency. Rust encodes thread-safety in the type system through the `Send` and `Sync` marker traits: a value may cross a thread boundary only if its type is `Send`, and may be shared by reference across threads only if it is `Sync`. Sharing a value that permits unsynchronised mutation is therefore a compile error, not a latent race. This is the property Rust markets as “fearless concurrency.” Error handling is likewise explicit: fallible operations return a `Result` value that the caller must handle, rather than raising an exception that unwinds through hidden control flow. Both the cost and the control flow of the failure path therefore stay visible, which is an advantage in latency-sensitive code where surprises in the tail are what matter.

Rust is not the only memory-safe language, and the reason the comparison here is drawn against C++ rather than against a managed runtime belongs with this model. Java, C# and Go achieve memory safety through a runtime and garbage collection, which is an excellent trade for much software and a poor one for a tail-latency-critical control plane: collection pauses are a recognised source of tail latency in large systems [6], and a managed runtime gives up the control over memory layout and scheduling this domain relies on. C,

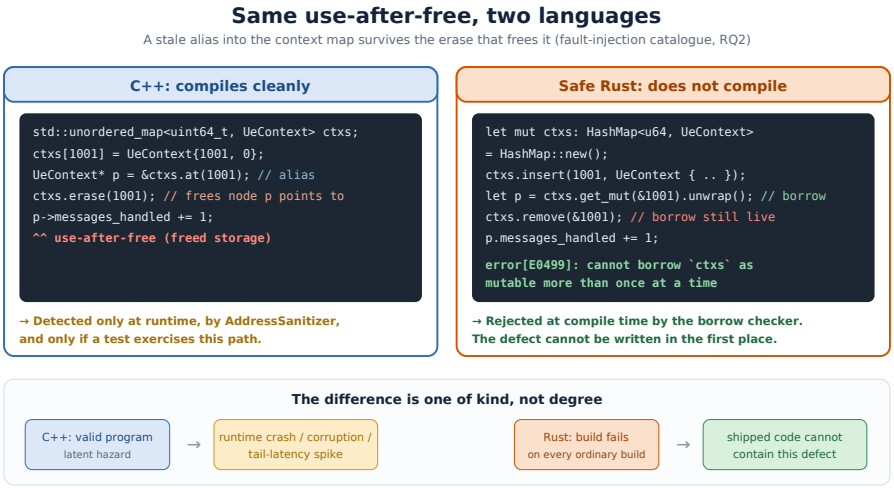


Figure 2.4: The same use-after-free defect in C++ and in safe Rust. The stale alias into the context map survives the `erase`/`remove` that frees it: in C++ this compiles and is caught only at runtime, and only under a sanitiser on a path a test exercises, whereas in safe Rust the borrow checker rejects it at compile time (`E0499`), so the defect cannot be written at all. The difference is one of kind, not degree.

at the other extreme, is as memory-unsafe as C++ while offering fewer abstractions for building large systems. Rust is the first widely supported language to combine compile-time memory and thread safety with the absence of a garbage collector and throughput and memory use alongside C and C++ [26, 39, 44]. That is why managed languages are out of scope and why the meaningful comparison for RAN control-plane software is C++ against Rust.

Table 2.1 sets the two languages' mechanisms side by side. The distinction it is built around is not which hazards each language names, since both name much the same ones, but *when* and *by what* each is ruled out, and the last two rows mark where the answer stops being "by the compiler" for either of them.

Table 2.1: How each language enforces the properties this thesis measures. The rows above the rule are the ones on which the two differ in kind rather than in degree: a compiler that rejects a program rules the defect out of every execution of it, whereas a sanitiser rules it out only of the executions an instrumented test happens to take. The rows below the rule are the boundary developed in Chapter 3 and measured in Chapter 6, where neither language offers a static guarantee.

Concern	Modern C++	Rust
Object lifetime	RAII and smart pointers, applied by convention. A raw pointer or reference may outlive its referent	Ownership: each value has one owner and is dropped exactly once
Aliasing	Unrestricted: any number of mutable aliases may coexist	Borrowing: either one mutable reference or many shared ones, never both
Bounds	Unchecked on `operator[]`, checked only through `at()`	Checked by default, elided where the index can be proved in range
Thread safety	Not expressed in the type system: sharing a mutable object across threads compiles	`Send` and `Sync` marker traits. Unsynchronised sharing is a type error
Error signalling	Exceptions or error codes: the failure path can be invisible at the call site	`Result` values the caller must handle. The failure path is explicit
Escape hatch	The whole language	`unsafe` blocks, delimited and mechanically countable
Stage of detection	Run time, under the right sanitiser, on a path a test exercises	Compile time, on every ordinary build, whatever the test coverage
Lock-acquisition order	Not checked	Not checked
Atomicity of a composed operation	Not checked	Not checked

2.5 Concurrency models for control-plane software

How concurrency is structured shapes both correctness and tail latency. The dominant model is *shared-memory* concurrency, in which threads mutate common state guarded by mutexes and atomics. It is flexible, but it is the source of races, deadlocks, lock contention, and priority inversion, and its worst case is hard to bound. The alternative is *message-passing* or *shared-nothing* concurrency, in which state is partitioned so that each piece is owned by a single thread and all interaction happens through messages, as in the actor model. It trades flexibility for scalability and much simpler reasoning, and it maps onto per-UE partitioning naturally, since each UE's state is largely independent. The choice also moves the quantity measured here: thread-per-core structuring changes application tail latency on its own [45], so the model is held fixed across both implementations instead of left to each language's idiom. Figure 2.5 contrasts the two.

A third structure, an asynchronous runtime over a work-stealing scheduler, is idiomatic in both languages and used in neither. Its timing behaviour belongs to the runtime, not

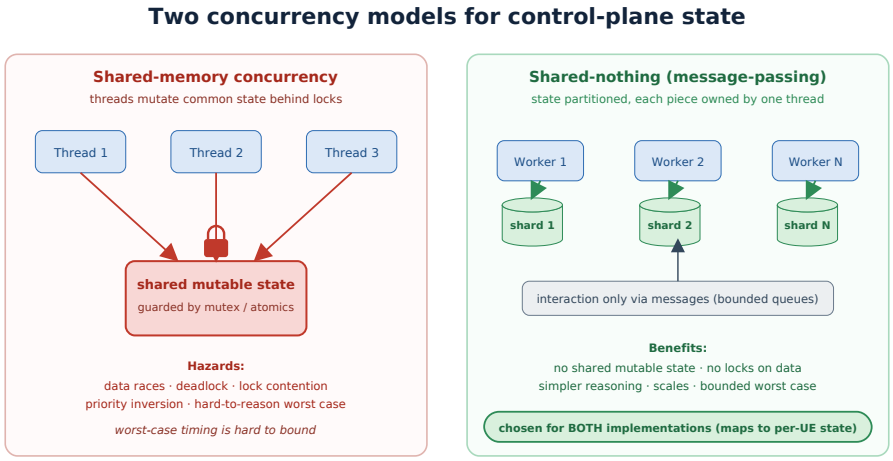


Figure 2.5: Two concurrency models for control-plane state. Shared-memory concurrency (left) lets threads mutate common state behind locks and is the source of data races, deadlock, and lock contention, with a worst case that is hard to bound. Shared-nothing message-passing (right) partitions state so each shard is owned by a single worker and all interaction goes through bounded queues, which removes locks on context data and maps onto per-UE partitioning. It is the model adopted for both implementations.

to the language hosting it: a real-time study of asynchronous Rust finds one application meeting or missing its deadlines depending on the runtime and thread-priority mapping it is given, and the configuration that meets them is not the idiomatic work-stealing one [46]. Adopting such a runtime would import the variable this study exists to exclude, and Section 7.5 returns to a comparison in which it was not excluded.

Queueing between producers and consumers is a second, independent choice. Unbounded queues mask overload and let latency and memory grow without limit. *Bounded* queues, whether they block for backpressure or shed, bound worst-case memory and make overload visible, which predictable tail latency requires. Both choices fix the architecture used by both implementations in Chapter 5: a shared-nothing, thread-per-shard design in which UE state is divided into independent slices called *shards*, each owned by exactly one worker thread, so no two threads ever touch the same context. Requests reach a shard through a bounded multi-producer, single-consumer queue that blocks when full.

2.6 Interoperability between C++ and Rust

Adopting a new language in a large existing system is necessarily incremental: a Rust component must coexist with and be callable from a substantial C++ codebase. The common denominator for that is the C application binary interface (ABI). Rust can expose functions with `extern "C"` linkage and hand out *opaque handles* to Rust-owned objects, which C++ calls through a C header. Tooling automates parts of this: `bindgen` (Rust bindings from C headers), `cbindgen` (C headers from Rust), and the `cxx` crate (a safer bidirectional bridge). Interoperability is not free. Crossing the boundary steps outside the checked language, so the seam must contain `unsafe` code, values beyond the simplest need marshalling, and the foreign interface has to be designed deliberately. Seam size, the `unsafe` it concentrates, and per-call crossing cost are the quantities RQ4 measures in Chapter 6.

One distinction bounds what those measurements can establish. Concentrating `unsafe` in the seam says what the compiler checks, not what can happen at run time. A Rust component linked into a C++ host shares one address space with it, so a defect on the C++ side can corrupt Rust-owned data however much of the Rust side the borrow checker verified. This has been shown experimentally, and the proposed remedies isolate the two heaps in hardware or in the runtime instead of trusting the discipline of the interface [47, 48]. The defect classes specific to Rust/C boundaries have been catalogued separately from those of either language [49], and the binding generators themselves carry bugs [50]. The consequence is stated once here and honoured throughout: an `unsafe`-free core means the core's own defects are ruled out at compile time, and it does not mean the core is protected from its host.

What has moved recently is the reach of the tooling, not the guarantee. A stateless model checker for Rust now explores the state space of a program together with its foreign dependencies, which puts data races arising from the interaction of the two languages within reach of a checker even though they sit outside what the borrow checker sees [51]. That capability arrived after the prototype measured here was built, and Chapter 8 records it as the next step for the seam.

2.7 Research gap

The preceding sections set up the gap. The RAN control plane is a soft-to-firm real-time, highly concurrent domain where tail behaviour under overload, not average throughput, governs the quality the network delivers. C++ supplies the control and performance the domain demands, but leaves whole classes of memory and concurrency defects to best-effort detection after the fact. Rust promises to prevent those classes at compile time at no run-time cost, and offers a credible incremental adoption path through the C ABI.

The domain is not unstudied, and the volume of prior work is easy to underestimate because almost none of it is indexed. At least five vendor-hosted master's theses, all carried out at Ericsson, have compared candidate languages for RAN and telecom software. One weighs runtime cost against development efficiency across C++, Rust, and Go [52]. One asks whether Rust could replace C in RAN node management [53]. One builds a thread-per-core packet processor on asynchronous Rust and measures it against the DPDK application it mirrors [54]. One measures median and 99th-percentile latency for control-plane procedures across five asynchronous programming frameworks [55]. The last builds matched C++ and Rust implementations of an ASN.1 decompiler and compares them statistically [56]. All five are directly relevant, and two report differences unfavourable to Rust.

Three of them vary the language together with the runtime and the concurrency model, so a measured difference cannot be attributed to the language rather than to the framework built on it [52, 54, 55]. The effect sizes reach a factor of five, which is an indication of how much of the measurement the runtime owns. The remaining two hold the design closer to fixed but report execution time and peak memory rather than tail behaviour under rising load [53, 56], and the closest of them, Emberg and Wadsten, measures a stateless transformation rather than a stateful, concurrent subsystem. Figure 2.6 sets the

neighbouring studies against the properties a controlled answer would require, and the pattern it shows is that each holds some of them and none holds all.

The concentration of that evidence is itself part of the gap. All five studies come from one vendor, and their conclusions have never been tested against another organisation's software, targets or conventions, so what is known about this question is what one company's frameworks report about it. The published record on the other side is emptier: there is no study of Rust among the indexed publications of the vendor hosting this work, and the nearest peer-reviewed position from that side argues for making memory-error sanitisation practical rather than for replacing it with a compile-time guarantee [57], which is the strategy this thesis tests against. Section 7.5 compares the five with this study directly.

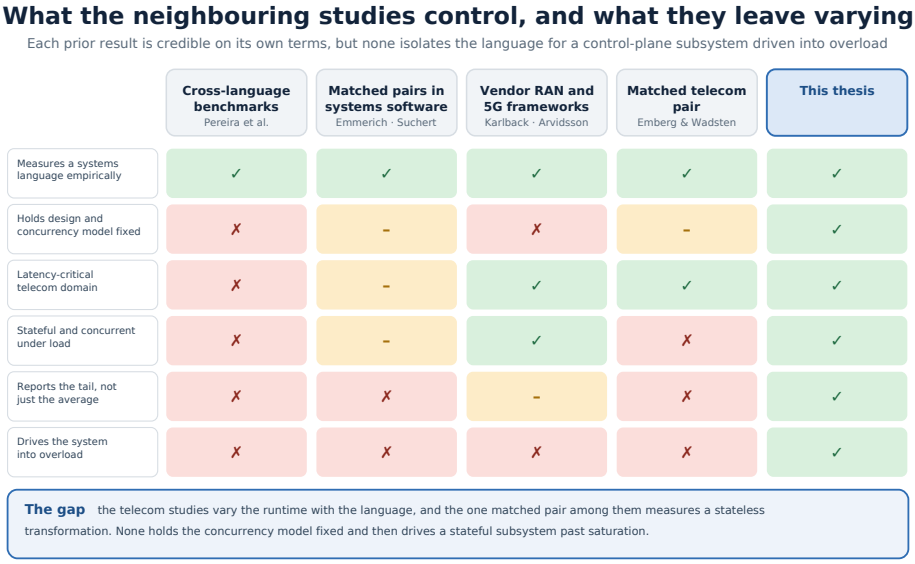


Figure 2.6: What the neighbouring studies control, and what they leave varying. Each row is a property that a controlled answer to the research questions would need. Each column is a study. The general cross-language comparisons measure the right kind of object but the wrong regime, reporting averages on general-purpose workloads. Matched pairs in systems software hold the design fixed, but outside this domain and below saturation. The vendor RAN and 5G framework studies reach the right domain, and one of them reports the tail, but in all of them the language varies together with the runtime and the concurrency model. The one matched telecom pair, Emberg and Wadsten, controls the design but measures a stateless transformation. Holding the design and the concurrency model fixed for a stateful, concurrent subsystem and then driving it past saturation is what this thesis adds. The contribution is the control, not a new measurement technique.

What is therefore missing from the literature is *direct, controlled* evidence for a representative control-plane subsystem: whether Rust actually holds its tail latency and predictability under overload against modern C++ *when the concurrency model, algorithms, and data structures are held fixed*, exactly which defect classes it eliminates and at what stage the remainder are caught, what its safety model costs in throughput and resources, and what incremental adoption costs in practice. Closing this gap is the purpose of the four research questions. The next chapter grounds them in the specific subsystem, defect classes, and timing properties at stake, and Chapter 4 sets out the methodology by which they are pursued.

3 Problem Context: UE Context Management and Predictability

3.1 The UE Context Management subsystem

The subsystem under study is a UE Context Management service. It is modelled on a real Nokia gNB-CU-CP service domain that owns the UE context lifecycle, RRC procedure execution [4], and mobility/handover, which are the functions the 3GPP architecture assigns to the control-plane part of a disaggregated gNB (Section 2.1).

Figure 3.1 shows the UE-context lifecycle state machine that both implementations encode identically. This state machine is exactly what the mixed workload of Section 6.2 walks each UE through.

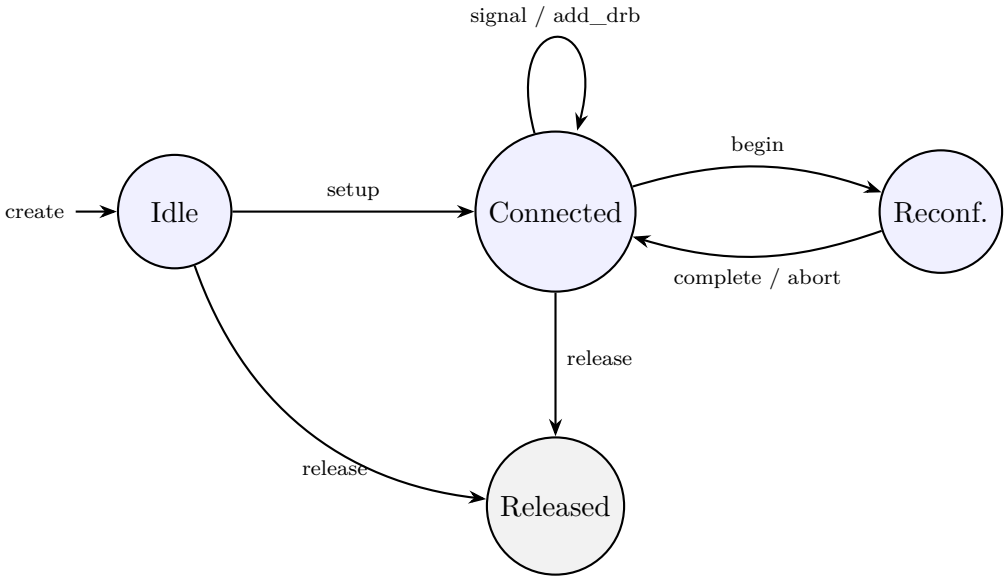


Figure 3.1: UE-context lifecycle state machine. `create` allocates a context in `Idle` and `setup` connects it. `signal` and `add_drb` are handled in place while `Connected`. A reconfiguration or handover moves through `Reconfiguring` and back on `complete` or `abort`, and `release` tears the context down. Operations invalid for the current state return an `OpError` instead of changing state.

Why this subsystem. UECM works as a study object because it is both representative and small enough to control. It is core RAN layer-3 functionality, and it carries the properties that make production control-plane code difficult: per-UE mutable state, a strict operation lifecycle, high concurrency across UEs, and the latency and reliability targets the 5G system sets for control-plane procedures [5, 28]. Conclusions about how a language handles those properties therefore transfer to other stateful, concurrent control-plane services. The subsystem is also small enough to specify precisely, implement faithfully in two languages, and reason about in full, which is what makes a parity-tested comparison possible at all. A larger subsystem would buy external validity at the cost of the internal validity a fair language comparison depends on. UECM sits where both stay

within reach.

Operation set. The interface exercised by the study is deliberately small and lifecycle-oriented. It can create and release a context, set up a connection (`Idle`→`Connected`), handle an in-session signalling message, add a data radio bearer, begin, complete, or abort a reconfiguration or handover, and look up a context's state under concurrent load. Every operation is defined for every state of the machine in Figure 3.1: an operation that is illegal in the current state returns a typed error and leaves the context unchanged, rather than throwing or corrupting state. This closed, well-typed operation set is what lets the same test vectors and the same workload drive both implementations along identical paths (Section 5.6).

3.2 Memory-safety and concurrency defect classes

The defects that affect C++ control-plane code of this kind fall into two families. The *memory-safety* family concerns the lifetime and bounds of objects: a *use-after-free* accesses storage that has already been released, a *double-free* releases the same storage twice, and a *buffer overflow* reads or writes past the end of an allocation. The *concurrency* family concerns unsynchronised or ill-ordered access from multiple threads: a *data race* is a pair of unsynchronised accesses to the same location of which at least one is a write, a *deadlock* is a cyclic wait for locks, and *iterator invalidation* is the use of an iterator or reference into a container after a mutation has moved or freed the underlying storage. What these share is that they are all *silent* in C++: each is accepted by the compiler and, on the paths ordinary tests happen to exercise, may appear to work, surfacing only later as corruption, a crash, or an unpredictable latency spike. These are the same memory-corruption classes that decades of security research have struggled to contain in C and C++ [33].

Each class has a natural incarnation in UE Context Management, which is why the subsystem is a good vehicle for studying them:

- **Use-after-free.** A signalling message holds a raw pointer or reference to a UE context that a concurrent `release` has already erased from the context map. The in-flight handler then dereferences freed storage.
- **Double-free.** A release path runs twice for the same UE, for example because a retransmitted release is not made idempotent, freeing the context's owned resources a second time.
- **Iterator invalidation.** Code iterates a context's data-radio-bearer list while adding or removing a bearer, so the iterator refers into storage the container has since reallocated.
- **Data race.** Two workers update the same context field, an activity counter or a state flag, without synchronisation, so the accesses race.
- **Unguarded shared mutation.** A context is shared by reference across threads and mutated without a lock or other discipline, a race whose root cause is the absence of any ownership rule for the shared state.

- **Use-after-move.** Admission moves a freshly built context into the store and then answers the request from the local variable it has just moved from, so the acknowledgement carries an empty identity. This class is carried for what the C++ tooling does *not* say about it. Nothing is corrupted and nothing is undefined, because a moved-from object is in a valid but unspecified state, so the defect has no runtime symptom for a sanitiser to find.

- **Thread-unsafe handle shared across workers.** A refcounted context handle whose counter is not atomic, the idiom a hot path reaches for when the atomic control block of a shared pointer costs more than it is worth, is passed to two shard workers. The constraint that makes the handle correct is written in a comment, because C++ has no way to state thread affinity in a type.

These seven classes lie squarely within the guarantees Rust claims to provide, which makes them the natural test of whether that claim holds. Testing only them, however, would build the expected answer into the choice of test cases, so seven further classes are carried into the study for the opposite reason, because there is good reason to expect that safe Rust does *not* prevent them:

- **Lock-order deadlock.** A handover locks the source UE context and then the target while a concurrent handover in the opposite direction locks target then source, and the two wait for each other indefinitely. Nothing about object lifetimes or aliasing is violated. The fault is in the order of acquisition.

- **Lost update.** Two workers increment a context's activity counter through a read-modify-write split across two separate critical sections. Every individual access is correctly synchronised, so this is a *race condition* rather than a *data race*: the atomicity violation lives in the composition of the operations, not in the memory accesses themselves, and it corrupts a value silently instead of crashing.

- **Leak through a reference cycle.** A context and the peer it is being handed over to hold owning references to each other, so neither reference count reaches zero and neither is ever destroyed. Reference counting cannot collect a cycle in either language, and leaking is safe in Rust by explicit design. This is the cleanest statement of a boundary in the catalogue, because both languages fail identically.

- **Integer overflow in a release build.** A bearer's TEID stride computation exceeds the width of its type. In C++ signed overflow is *undefined*, so the program has no meaning. In Rust it is *defined* to wrap, because the `opt-level = 2` profile this study compares leaves `overflow-checks` off, so the program has a meaning and the meaning is wrong. The pattern is not hypothetical: CVE-2026-64467 in `rust_binder` is a stride error of this shape.

- **Worker failure blast radius.** One shard worker fails part-way through a procedure. The concurrency model of Section 5.2 makes a shard a failure domain by construction, and neither language's type system enforces that it behaves as one: in C++ an exception escaping the thread callable terminates the whole process, and in

Rust the panic is confined to its thread but poisons the state it held, so the damage propagates along the data instead. Directly relevant to a carrier-grade availability argument, and addressed by nothing in either type system.

- **Double-free across the seam.** A C++ host holds an opaque handle to a Rust context and releases it twice, which is the retransmitted release of the double-free class above, now expressed the way a foreign-function boundary has to express it. Safe Rust rejects the double release because ownership tracks the value. A seam cannot keep that, since handing a raw pointer to a C caller is exactly the operation that ends the compiler's knowledge of it. The class therefore appears on both sides of the boundary, which is the sharpest available statement of what the boundary is.

- **Forgotten lock guard.** Bearer setup takes the context lock and a validation failure returns without releasing it, so the next worker to touch that context waits indefinitely. Rust removes the ordinary form of this bug, because a guard releases when it is dropped and there is no hand-written unlock to skip. It does not remove the bug, because a guard is a value and leaking a value is safe. Where the reference cycle leaks memory, this leaks the context itself.

Together these fourteen form the catalogue injected in Chapter 6, and the split between the two groups is what lets the study report a boundary rather than a verdict. Buffer overflow is described above for completeness but is not injected, because neither implementation performs raw pointer arithmetic on the context store, so an injected instance would be an artificial construct and not a plausible bug in this subsystem. Table 3.1 collects the catalogue with the outcome expected of each case *before* the study was run, which is the form in which the runner records it: an expectation written down in advance can be violated in either direction and reported as a discrepancy, whereas one formed after the fact cannot.

Table 3.1: The injected defect catalogue: each class in the form it would plausibly take in UE Context Management, and the outcome recorded for it in advance of the study. The last column is a prediction rather than a result. Table 6.11 reports what was measured, and the fault runner flags a mismatch in either direction instead of absorbing it into the conclusion. Buffer overflow is listed for completeness and not injected: neither implementation does pointer arithmetic on the context store.

Defect class	Incarnation in UE Context Management	How it fails	Expected in safe Rust
Memory-safety family: object lifetime and bounds			
Use-after-free	An in-flight signalling handler dereferences a context that a concurrent `release` has already erased from the map	Corruption or crash	Prevented
Double-free	A retransmitted release is not made idempotent, so the release path frees the context's resources a second time	Corruption or crash	Prevented
Use-after-move	Admission answers the request from the context it has just moved into the store	Silently empty identity	Prevented
Buffer overflow	No plausible incarnation here: the context store is reached only through container APIs	n/a	Not injected
Concurrency family: unsynchronised or ill-ordered access			
Iterator invalidation	A bearer is added while the context's data-radio-bearer list is being iterated	Corruption or crash	Prevented
Data race	Two workers update the same context field, a counter or a state flag, without synchronisation	Torn or lost value	Prevented
Unguarded shared mutation	A context is shared across threads and mutated with no ownership rule governing it	Torn or lost value	Prevented
Thread-unsafe handle shared	A context handle whose refcount is not atomic is passed to two shard workers	Corrupted refcount	Prevented
Boundary group: included because safe Rust is expected not *to prevent them*			
Lock-order deadlock	Opposing handovers lock source-then-target and target-then-source	Hang	Not prevented
Lost update	A counter's read-modify-write is split across two critical sections, so each access is synchronised but their composition is not atomic	Silently wrong value	Not prevented
Reference-cycle leak	A context and its handover peer hold owning references to each other, so neither reference count reaches zero	Unbounded growth	Not prevented
Integer overflow (release build)	A bearer's TEID stride computation exceeds the width of its type	Silently wrong value	Not prevented
Worker-failure blast radius	One shard worker fails part-way through a procedure while holding shard state	Outage beyond one shard	Not prevented
Double-free across the seam	A C++ host releases an opaque handle to a Rust context twice	Corruption or crash	Not prevented
Forgotten lock guard	Bearer setup returns on a validation failure without releasing the context lock	Hang	Not prevented

3.3 Tail latency and predictability in RAN

Section 2.2 set out why the tail rather than the mean governs a latency-critical system, and what measuring it correctly demands. What that section left general, this one makes specific, because two properties of the RAN setting are what turn a methodological pref-

erence into the governing metric.

Those properties are fan-out and scale, drawn in Figure 2.2. One user-visible action, a call setup or a handover, becomes many dependent control-plane operations whose slowest member gates completion, and a busy site drives thousands of such procedures per second across millions of connected devices. Put together, the p99.9 of a single UE Context Management operation is not a curiosity at the margin of a distribution. It is a rate of degraded procedures the network sustains continuously.

The scale at which those percentiles matter is not hypothetical. A measurement study of a disaggregated 5G RAN times the CU-CP-to-DU UE Context Setup procedure over F1-C, and the CU-CP-to-CU-UP bearer context setup over E1, under different interface-protection schemes, and finds the choice worth hundreds of microseconds: on F1-C, IPsec adds on the order of 120 μs on average to UE context setup, against roughly 430 μs for DTLS [58]. Overheads of that size are treated as consequential for precisely the procedure family this thesis reimplements, which places these operations in a regime where tens of microseconds are an engineering quantity rather than noise.

The study therefore characterises the full latency distribution, focusing on the high percentiles (p99, p99.9 and p99.99) and the maximum, and on how these degrade as offered load approaches and then exceeds saturation (Chapter 6). Stopping at nominal load would miss the regime that decides service quality, since queueing delay grows super-linearly near capacity. It is also the regime in which a difference between two implementations of one design, if there is one, has room to become visible.

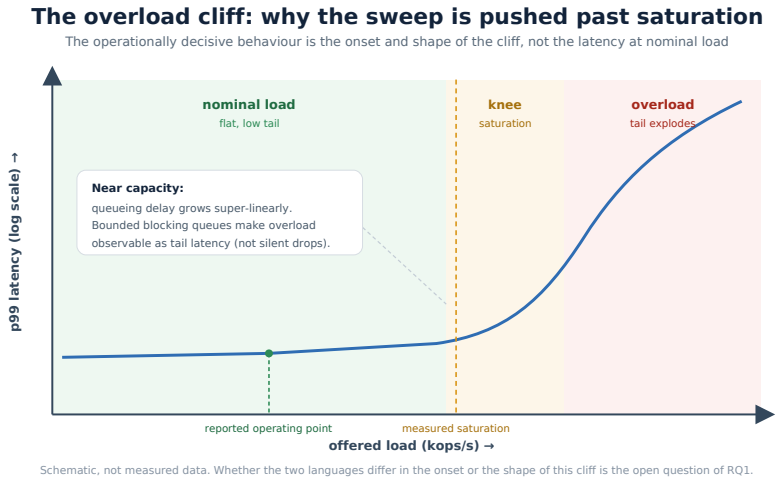


Figure 3.2: The overload cliff. Tail latency stays low and flat while offered load is below service capacity, then climbs steeply once the system is driven past saturation and requests queue behind the bounded, blocking queues. The operationally decisive behaviour is the onset and shape of this cliff, which is why the load sweep is pushed beyond the saturation point rather than stopping at nominal load. The figure is a schematic of the phenomenon and not a plot of this study's data: whether the two languages differ in where the cliff begins or in what they sustain beyond it is exactly what RQ1 asks, and Sections 6.4.1 and 6.4.2 report that they differ in both. How steeply the cliff rises is not compared between them, for the reason Section 4.4 fixes in advance.

4 Research Methodology

The previous chapter set out what the study must achieve. This one sets out how. It states the overall research strategy, the method chosen for each research question and why it is appropriate, the controls that make a language comparison fair, the statistical treatment of the measurements, and the reproducibility and ethical commitments under which the work is carried out.

4.1 Research approach

The study is experimental with a constructive (design-science) element. The artifact is a pair of functionally equivalent implementations of the subsystem, one in C++ and one in Rust, plus the framework that benchmarks them. Both are built from a single frozen specification and then evaluated under controlled experiments. Evidence is triangulated across three methods. Performance measurement under a load sweep answers RQ1 and RQ3, a controlled fault-injection study answers RQ2, and an FFI migration case study, part measured and part qualitative, answers RQ4. The three are independent. A weakness in one of them, such as the environment sensitivity of tail-latency measurement, therefore does not carry into the conclusions as a whole. Every measurement reports variance over repeated runs, and Section 4.3 sets out the controls that keep the language the only variable.

That design follows from the shape of the questions. Each asks how one outcome (tail latency, defects prevented, resource use, migration cost) depends on one variable, the implementation language. The language therefore has to be the only systematic difference between the two objects under test. This rules out the commonest form of language comparison in the literature, where two independently written programs are measured and the gap is attributed to the language. Such a gap also carries the design, the libraries and the skill of their authors.

Building both sides from one specification removes that confound. The comparison is then between two encodings of the same design, and the artifact becomes both the contribution and the instrument that measures it. Where a question can be answered with numbers, the treatment is quantitative. Where it cannot, as with the kind of guarantee in RQ2 and the API-design effort in RQ4, it is qualitative but still systematic.

The gap this design addresses (Section 2.7) is only as good as the search that looked for it, so that search is on record. Appendix B gives the databases, the term combinations, the citation chaining and the filtering rules. Two of its findings bear on the design and not just on the bibliography.

The vendor-hosted theses closest to this study (Section 7.5) appear in no citation database. Chaining therefore never reaches them, and they had to be found by searching the institutional repositories directly. The same searches establish an absence on the other side: no Finnish thesis on Rust in a radio-access or telecom context, and no indexed publication mentioning Rust from the vendor hosting this work. The domain-specific literature on

the language question is therefore Swedish and Ericsson-hosted, and whether it carries over here is untested. Those theses are grey literature, so they are cited as evidence of what a vendor has attempted and found, not as peer-reviewed results.

The search returned no study, indexed or grey, that measures the tail-latency behaviour of a Rust and a C++ implementation of the same stateful, concurrent control-plane subsystem under overload.

Table 4.1 maps each research question to the method, artifact, data and metric that answer it. All four chains end in the same place: Section 7.2 judges each question and Section 7.3 disposes of its hypothesis. RQ1 and RQ3 share H1, worded to cover performance equivalence in both the latency and the resource sense. That wording turns out to be the hypothesis's main weakness. RQ2 is insensitive to the environment, and RQ4 is so except along one axis: build time is a timing measurement like any other, and is therefore read as a ratio between two toolchains measured alternately on one machine. RQ1 and RQ3 depend on the measurement host and are reported against what it can carry (Section 6.1).

Table 4.1: Research design: how each research question is answered. Every question is tied to a concrete method, a published artifact, the data it produces, and the metric on which the conclusion rests. RQ1 and RQ3 carry two methods rather than one: the sweep measures the round-trip a deployment would experience, and the component decomposition measures the part of it the language is responsible for. Section 4.2.1 states why one cannot stand in for the other. RQ1 and RQ3 are tested against H1, RQ2 against H2, and RQ4 against H3.

RQ	Method	Artifact / data	Primary metric
RQ1	Load sweep to overload, interleaved repeats	Benchmark harness, latency histograms	median, p99/p99.9/p99.99 tail latency, and maximum vs. offered load
RQ3	Same sweep, resource capture	`getrusage` samples per run	peak RSS, CPU utilisation, throughput
RQ1, RQ3	Component decomposition: the core called directly, then through the service	Matched microbenchmark pair, interleaved repeats	per-operation cost split into the operation and the cross-thread handoff
RQ2	Controlled fault injection	Defect catalogue, ASan/TSan/UBSan, `rustc`, Loom/Shuttle and Miri logs	defect-class taxonomy (compile-time / runtime / undetected)
RQ4	FFI migration prototype	C++ host + Rust C-ABI seam	seam size, `unsafe` scope, per-call interop cost

4.2 Methods for the four research questions

Table 4.1 pairs each question with a method.

4.2.1 Performance measurement under a load sweep (RQ1, RQ3)

RQ1 and RQ3 are answered by driving both implementations with one deterministic synthetic workload across a sweep of offered load, from light load through saturation into overload. Two design decisions make the measurement sound. The load generator follows an *open-loop* schedule: each request has a fixed intended send time, and latency is measured against that time and not against the moment of admission. That corrects for coordinated omission as Chapter 2 defines it. The sweep is pushed past saturation deliberately, because the overload cliff appears only once offered load exceeds service capacity (Section 2.2). Latency goes into high-dynamic-range histograms configured identically on both sides, and resource use (peak resident set size, CPU utilisation, throughput) is sampled through the operating system.

Measuring the component as well as the round-trip. A round-trip bundles two costs of different character. One is the operation the language runs inside a single thread: a hash lookup, a state-machine guard, a field update, all compiler-generated. The other is the cross-thread handoff around it: an enqueue, a futex wake, a context switch, a completion, none of it language-specific. Only the first can plausibly differ between C++ and Rust, and no end-to-end measurement separates them.

The method therefore works at two levels. The operation processor is called directly, with no queue in the path, then through the full threaded service. The difference between the two is the handoff. Both levels are batch means over a long loop and not per-call medians, because a clock read costs tens of nanoseconds, the same order as the operation itself, and per-call timestamps would measure the instrument. The languages are interleaved across repeats and compared with the same exact paired test as the sweep, so the levels differ in what they measure and not in how they are analysed.

Reporting both separates two questions the research questions did not distinguish as originally worded: what a deployment of this shape would experience, and what the language is doing. Section 6.4.1 finds the two more than two orders of magnitude apart. The distinction therefore decides whether the result can be interpreted at all. The second level was added after a first pass of the campaign, and Section 7.6 records that revision.

Isolating the contended synchronisation primitives. The overload comparison in Section 6.4.2 attributes its gap to two forced-idiom entries where the standard libraries diverge. They are the bounded blocking queue and the one-shot completion handle. That attribution is argued from the shape of the full-service measurement, so a third matched microbenchmark drives those two primitives alone, with no `ShardCore` work in the path, under the same interleaved repeats and paired test. Section 6.4.2 gives its configuration and reports whether the isolated primitives reproduce the ordering the full service showed.

4.2.2 Controlled fault injection (RQ2)

RQ2 asks which defect classes a language prevents. Measuring defect-free code cannot answer that: the absence of a bug in a passing test says nothing about whether the language would have caught it. The method is therefore adversarial, following the established practice of software fault injection, where faults are introduced deliberately to see how a system behaves in their presence [59].

A catalogue of representative C++ defects is written *deliberately*, drawn from the classes that affect this subsystem (Chapter 3). Each is a small self-contained program flavoured with the domain: a stale pointer into the context map, a double release, a mutation during iteration, an unsynchronised shared update. Each is built and run under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer, recording which tool detects it, if any, and at what stage.

The same construct is then written in safe Rust and compiled. The compiler's outcome is recorded, and where it compiles, so is the program's behaviour. The boundary classes give the compiler nothing to say, so where another tool can reach one, the Rust side goes to that tool as well: the two concurrency classes to the model checkers Loom and Shuttle, and the class that lives inside an `unsafe` block to the Miri interpreter. Those three are the cases these tools were run on, chosen because each is the kind of defect the tool is built for, and answering "at what stage and at what cost" for them is what the extra runs buy. Each is paired with a control in which the defect is fixed, so that finding the defect and reporting everything are not the same observation.

Whether ThreadSanitizer reports the lock-order inversion depends on the schedule, so a single run would record whichever outcome the machine happened to give. The runner executes that case twelve times and reports the rate, so intermittent detection is not mistaken for reliable detection.

Why the C++ side is tooled with sanitisers only. No static analyser is run against the C++ catalogue, and that is a decision. RQ2 does not ask which tool finds the most defects. It asks at what *stage*, and with what force, each language rules a defect out, and the taxonomy's three outcomes (compile-time rejection, runtime detection, silent miss) are the categories that question needs. An opt-in pass belongs to none of them: a warning a build can be configured to ignore is a different kind of object from a compilation that cannot succeed, and that difference is what the method measures.

The cost is a C++ side narrower than the best a C++ project could manage, since a well-configured analyser could plausibly reach the small intra-procedural memory-safety cases, though not the concurrency ones. The study therefore reports the tooling it ran, the sanitiser-based practice of Section 2.3, and not a ceiling on what C++ tooling can achieve.

The injected fault is held fixed while the language varies, so any difference in outcome comes from the language's guarantees and not from the specific bug. What the method establishes is the *kind* of protection each language offers, not a performance number.

One design decision determines whether the taxonomy can be trusted. A catalogue drawn

only from classes the language is known to prevent could confirm the hypothesis but never disturb it, and would describe the author's choice of cases as much as the language. The catalogue is therefore built in two groups: classes inside Rust's stated guarantee, and classes just outside it, where ownership and the `Send`/`Sync` discipline give no protection. Each case carries a recorded expectation and the runner reports a violation in either direction, so an unexpected outcome surfaces as a discrepancy instead of being absorbed into the narrative. What this buys is a boundary and not a score: the study can state which classes the language removes and which comparable ones it leaves in place, which is what a migration decision needs.

4.2.3 Foreign-function-interface migration case study (RQ4)

RQ4 concerns the practical cost of adopting Rust inside an existing C++ system, an engineering question as much as a performance one. It is answered with a prototype built in the realistic direction of migration: a C++ host calling a Rust component across a C application binary interface. Three quantities are measured on it. The *size* of the seam counts its entry points, lines of code and dependencies. The *scope* of `unsafe` records where the checked language is left and whether the safety-critical core is affected. The *per-call cost* is the price of crossing the boundary, measured against the same operation invoked natively so the comparison isolates what the seam adds. The first two are read off the code, the third is a matched pair of microbenchmarks. Pairing them separates the run-time cost of interoperability from its true cost, which lies in API design and human learning.

Interoperability is one of the three terms RQ4 names, and a prototype answers it alone. Three further quantities cover the other two, chosen because a build engineer reaches them before anything the seam can show. *Build time* is measured clean and incrementally, for the core library alone and for every default target, with the same job count and interleaved repeats on both sides so that one host's load falls on both toolchains equally. *Static footprint* is measured on the conformance runner, the one executable both languages build from the same specification, and at both linkage policies instead of at each toolchain's default, because the two defaults differ in whether the standard library is linked in and a single pair of numbers would report that choice as a language difference. *Third-party surface* counts the transitive crates each configuration resolves and the `unsafe` sites inside them, which makes the dependency-free claim of Section 5.4.4 checkable. The first is host-sensitive and the other two are not.

4.3 Fairness as a controlled-variable strategy

Every research question attributes a difference to the language, so validity depends on holding everything else equal. Fairness is therefore the study's central control, enforced at several levels.

The *specification* is frozen before any code is written and fixes every design choice down to the sharding and concurrency model (Section 5.1). The *run-time substrate* is equalised by linking the same allocator on both sides and matching build-optimisation levels as closely as the toolchains allow. The *workload* comes from a driver identical in structure across

languages that, for a given seed and active-UE population, drives both implementations through the same sequence of operations. Any latency difference therefore reflects how each language executes the same work, not a difference in what work is done.

Where the two languages genuinely cannot express the same construct, the deviation is recorded in a forced-idiom log. What remains uncontrolled, principally single-author skill asymmetry, is acknowledged as a threat to validity (Chapter 7) and not claimed to be eliminated.

Table 4.2 states the strategy variable by variable, and its third column matters most. A controlled experiment is credible in proportion to how honestly it accounts for what it failed to control. Three rows record a residual and not a clean fix: the languages cannot always express the same construct, they do not ship the same hash-map layout, and one person wrote both implementations. The forced-idiom log handles the first, and the last row names the one variable left with no control at all.

Table 4.2: The controlled-variable strategy: what is held fixed so that the language remains the only systematic difference, and what each control does not reach. The third column is not a list of oversights but the study's own account of its limits. Each entry is picked up again as a threat to validity in Section 7.6. Only the last row names a variable for which there is no control at all.

Variable	How it is held fixed	What the control does not reach
Design	One specification, frozen before any code is written, fixing the public API, the state machine, the data structures and their algorithmic strategy, and how state is divided into shards (§5.1)	Where the two languages cannot express the same construct, the divergence is logged, not removed (Table 5.1)
Concurrency model	Thread-per-shard over bounded blocking MPSC queues on both sides, with no asynchronous runtime on either	Nothing outstanding
Hashing	SplitMix64 for shard selection and FNV-1a inside the shard, pinned identically, so neither standard library's default hasher becomes an accidental advantage	The container itself is still each language's own hash map, and the two differ in layout (§6.4.1)
Allocator	The same `jemalloc` linked on both sides, with a system-allocator build kept on each as a control	Nothing outstanding
Optimisation	`-O2` against `opt-level = 2`	Nominal parity is not identical code generation
Workload	One deterministic driver. For a given seed and active-UE population both implementations execute the same operation sequence, checked by per-kind counts	Synthetic and spec-justified rather than replayed from production traffic
Instrumentation	The same HDR histogram algorithm and configuration, the same open-loop schedule, the same repeat and interleaving discipline	Nothing outstanding
Behaviour	A language-neutral conformance suite whose canonical transcripts are diffed op for op (§5.6)	Parity bounds behaviour, not performance
Author	No control	One person wrote both implementations and is not equally expert in both

4.4 Statistical treatment of measurements

Performance measurements are non-deterministic, so single-run numbers are avoided throughout. Each configuration (language × allocator × load level) is measured over many independent repeats, and the four variants are *interleaved* within each repeat so that slow drift, thermal effects, and background activity perturb them equally instead of biasing one. Results are summarised by the median with a nonparametric bootstrap 95% confidence interval [60] rather than by the mean with a standard deviation, because latency distributions are heavy-tailed and skewed and a distribution-free interval on the median makes no normality assumption. This follows established guidance on statistically rigorous performance evaluation, which warns that reporting a single number invites conclusions that run-to-run variation does not support [32]. Tail percentiles (p99, p99.9, p99.99) are read from the HDR histograms, whose logarithmic bucketing retains three significant figures of resolution across the whole range so that high percentiles are not quantised away. Intervals are reported in full, not as a symmetric half-width, because a bootstrap interval on the median of a heavy-tailed sample is markedly asymmetric and a half-width wide enough to cover its upper bound would place the lower bound below zero.

Two of these quantities need their own note, because how far into the tail a campaign can see is a property of how many operations it recorded. At the representative load a single run retires on the order of 170 thousand measured operations, so its p99.99 rests on roughly seventeen observations: a per-run p99.99 is dominated by sampling noise even on a perfectly quiet host. Reading the far tail at all therefore requires persisting each run's complete HDR bucket contents alongside the summary percentiles, so that the repeats can be *pooled* into one distribution of several million operations first. The harness does that, and the design keeps pooling and averaging apart, because they are different statistics. The pooled figure takes the operation as the unit of analysis and resolves the far tail, at the cost of carrying no across-repeat interval, while the median-of-repeats figures reported with intervals keep the repeat as the unit and remain the only basis for a paired comparison. Jitter is likewise measured directly rather than inferred from the latency distribution, as the absolute difference between consecutive latencies within a producer. That is the packet-delay-variation quantity of RFC 3393 [61], and it matches the sense in which this thesis uses the term: variation from one operation to the next, not spread of the distribution as a whole. The reported campaign carries both. Its p99.99 is pooled across repeats by exactly the procedure described above and reaches Table 6.1 as a column of its own, marked there because it is the one column of that table computed at a different unit of analysis and so the one column carrying no interval. The analysis scripts read the pooled file and never the per-run field, so a campaign whose histogram sidecars have not been pooled loses the column rather than printing the per-run figure this paragraph has just called uninterpretable. Jitter is recorded and reported descriptively, because no research question asks for it.

Neither p99.99 nor jitter enters the family of paired tests described below, and that follows the same rule as the rest of the design rather than any shortage of data. Both are computed from precisely the operations that interference lands on, which is the class of statistic pairing does not rescue, so a paired interval on them could not be read as a

language effect. They belong in a descriptive column, and the family stays the size the design committed to.

Interval estimates alone, however, are a weak basis for a comparison. Two intervals can overlap while one variant is ahead in every single repeat, and they can separate on a difference that does not reproduce. The study therefore also tests the comparison directly, and the interleaved design is what makes this possible: because the four variants are measured back to back inside each repeat, the runs form a matched design in which the interference common to a block cancels in the pairwise difference. Language is compared against language with the allocator held fixed, using an exact two-sided paired permutation test on the repeat-wise differences. The null hypothesis is that a pair differs only by run-to-run noise, under which the sign of each paired difference is exchangeable. For the repeat counts used here every one of the 2^n sign assignments is enumerated rather than sampled, so the resulting p-value is exact and, like the bootstrap, assumes nothing about the shape of the distribution. Because the same comparison is made for several metrics and both allocators, the family of tests is corrected with the Holm step-down procedure [62], which controls the family-wise error rate without assuming independence between the tests.

Two decisions about how that family is read are taken here rather than after the results, because either could otherwise be made to suit whatever the data turned out to be.

The first is which comparisons enter it, and the answer turns on the operating point rather than on the metric. Below saturation every variant completes work as fast as it is offered, so achieved throughput there is pinned to the offered load by construction and cannot express a language effect whatever the languages do. It is reported with its interval instead of tested. Above saturation that reasoning inverts, because what each variant still sustains is pinned to nothing and becomes the one quantity that can separate them, so a second operating point is fixed in the overload region and sustained throughput and CPU utilisation are tested at it (Section 6.4.2). Two commitments go with that. The overload comparisons join the same Holm family rather than forming a second one, so the division into two operating points cannot be read as a split chosen once the p-values were known. And no latency percentile is tested above saturation at all, because a producer that cannot keep up with a fixed intended schedule falls behind cumulatively, which makes the percentiles there a lower bound and not an estimate. Appendix C sets out the reasoning behind both.

The second is what to conclude when the interval and the p-value disagree. At modest repeat counts they routinely will: a paired interval can sit wholly on one side of zero while the corrected p-value stays far above any conventional threshold. Which instrument to believe is not a free choice, and it does not depend on whether the metric happens to be interesting. It depends on whether pairing removes the interference that particular measurement is exposed to. Pairing cancels whatever an interleaved block shared. It cannot cancel interference that landed on one run of a pair and not on the other. A run-level summary such as peak resident set size is unmoved by a handful of descheduled operations, and so is a central order statistic such as the median of tens of thousands of per-operation samples. For those, a paired difference that keeps its sign across repeats is a difference in the software. A high percentile is the opposite case: p99 and p99.9 are

computed from precisely the operations that were disturbed, so an interval excluding zero is as consistent with one run of a pair having been unlucky as with a real effect. Table 6.3 is therefore blocked by that distinction, and intervals in its second block are not read as findings however far from zero they sit. Applying the rule to the block that suits the argument and quietly dropping it for the other is the specific failure this pre-commitment exists to prevent.

The resolving power the design can reach is bounded by the number of repeats, and is stated rather than left implicit. With 20 repeats and 14 comparisons the exact test's two-sided floor is 0.0000019, and 0.000027 after correction, so a null reported here reflects the data and not the arithmetic. Quoting both is what lets a reader tell a null result from an unresolvable one. Choosing the repeat count so that the corrected floor sits below the conventional 5% is therefore part of the measurement protocol, alongside pinning the clock and running the pinned cores tickless (Section 6.1). Appendix C derives the two floors.

4.5 Reproducibility and research artifacts

The study is designed so that its results can be regenerated by a third party without access to Nokia's proprietary code. Three things make that possible. The implementations are driven entirely by the public frozen specification. The benchmarking harness, the fault catalogue and its runner, and the FFI prototype are published together with the thesis. And every result table, every plot of measured data, and every quoted number in the evaluation is rebuilt by one regeneration script in the published package (`bench/refresh_thesis.sh`) directly from the raw logs. The scope of that claim is worth stating precisely, because two things it might be read to cover are outside it. The hand-drawn schematics that explain a concept rather than report a result are authored and not generated. And running the same scripts over another host's campaign regenerates the artifacts by the same procedure, not with the same values, which Section 6.4.4 demonstrates on a campaign of its own design. What the script guarantees is that no figure, table, or sentence can hold a value the logs no longer support.

The logs that script reads are part of the package. A performance campaign is an append-only newline-delimited JSON file, named by the timestamp at which it started, and the reported sweep and the virtualised campaign it replaced both sit under `bench/results/` while the second bare-metal campaign has a directory of its own. What keeps an analysis from pooling two hosts is therefore the generator and not the layout: each campaign has its own script and its own macro prefix, so a figure from one cannot reach a sentence about another. The two bare-metal campaigns also write a sidecar holding every run's complete histogram buckets, which is what the far-tail percentile is computed from and which the virtualised campaign predates. Each run's environment is recorded by a companion capture script (`bench/capture_env.sh`) and stored beside the campaign data. Three runners write three machine-readable outcome files under `faults/results/`: `taxonomy.csv` for the sanitiser and compile outcomes, `model_checking.csv` for the two boundary cases checked formally, and `miri.csv` for the `unsafe` interpreter run. Nothing in the evaluation chapters is transcribed from those files by hand.

The execution environment therefore travels with its data: CPU model, kernel, compiler and toolchain versions, and the frequency and scheduling settings of the testbed. That matters for one distinction in particular. Separating environment-sensitive results (the performance sweep) from environment-insensitive ones (parity testing, fault injection, and the FFI structure) is what lets the thesis state, per result, which of the two a reader is looking at. Nothing on the path requires proprietary source, which is what makes third-party reproduction possible at all rather than merely permitted.

5 Design and Implementation

5.1 Shared specification: the fairness baseline

Before any code was written, one specification was frozen, and nothing in it names a language. It fixes the public API, the UE state machine, the data structures and the algorithm behind them (a sharded hash map keyed by UE identifier, for instance), the granularity of locking and sharding, and the concurrency model. Both implementations follow it. That is what keeps the comparison a comparison of two languages, and not of two designs or two levels of skill. The specification is a deliverable in its own right, and it is the main control against implementation bias.

5.2 Conceptual model and concurrency design

Two implementations are built against the shared specification, and they do the same job: a modern C++ version (C++23, STL, RAII, standard threading) and a Rust version (stable toolchain, ownership-based memory model, safe concurrency). Both use one concurrency model, described as *shared-nothing, thread-per-shard*, which is how real RAN L3 divides UE state. Async and thread-based models are never mixed, and that is deliberate.

The UE-identifier space is split into N_{shards} shards, and each shard belongs to exactly one worker thread. Eight is the specification's default. The reported campaign runs seven workers against seven producers, which fits inside the 16 pinned cores of the host in Section 6.1 with room to spare and was chosen on the earlier host, where fourteen threads and fourteen pinned cores met exactly. What the fairness controls of Section 4.3 require is that the count be identical in the two languages, which it is in every campaign, and not that it be identical across machines: the second bare-metal campaign raises it to 8 deliberately, to test whether the findings depend on the choice (Section 6.4.4). Only the owning worker reads or changes its shard's contexts. Workers therefore share no mutable state, and context data needs no locks. Callers (the load-generator threads) never touch context state directly. They submit request messages to a shard's bounded multi-producer, single-consumer (MPSC) queue, then wait on a single-use completion handle that the worker writes the result into. The queue is *bounded and blocking*. When a shard's queue is full, submission blocks until space frees up. That is what produces realistic queueing latency under overload, and it is also what makes tail latency observable at all, because a policy that dropped requests instead of blocking would hide the overload. Requests to the same shard run strictly first-in-first-out.

This structure is what gives the safe-concurrency comparison something to measure. The machinery under test is the queue, the completion handoff, and the ownership transfer of each request, not lock contention. One piece of shared mutable state is allowed: an atomic counter that stamps a per-context activity sequence number. It is write-only bookkeeping, and no control flow branches on its value. Latency is measured across the whole path, from intended send time to completion, so it includes queueing delay under load and covers the

same cross-thread round-trip on both sides. Section 6.4.1 later splits that figure into the part the language executes and the part the operating system does. Figure 5.2 shows the data flow, and Figure 5.1 traces one operation through it as a sequence.

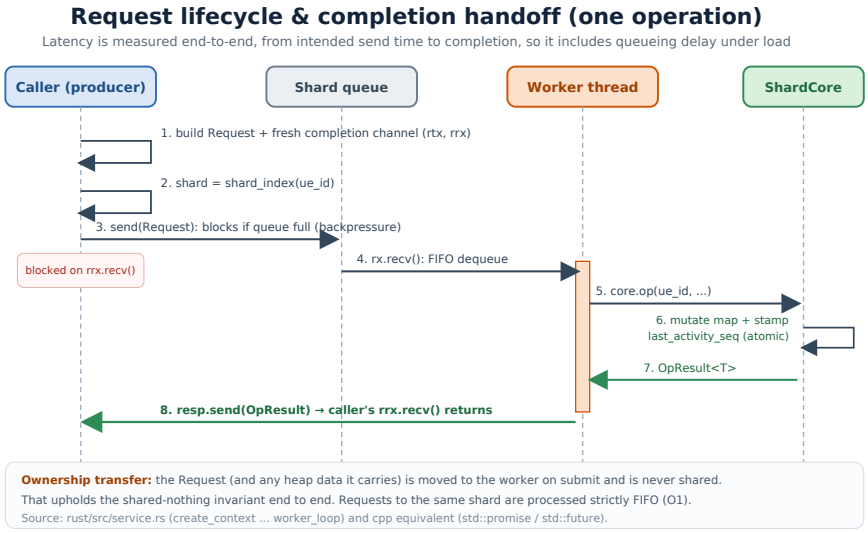


Figure 5.1: The end-to-end path of a single operation and the completion handoff. The caller builds a request with a single-use completion handle, routes it by `shard_index(ue_id)` to the owning shard's bounded queue (blocking if it is full), and waits. The shard's single worker dequeues in FIFO order, executes the operation against its exclusively owned store, and writes the result back to the completion handle. Ownership of the request is transferred to the worker on submit and never shared. Latency is measured across the whole path.

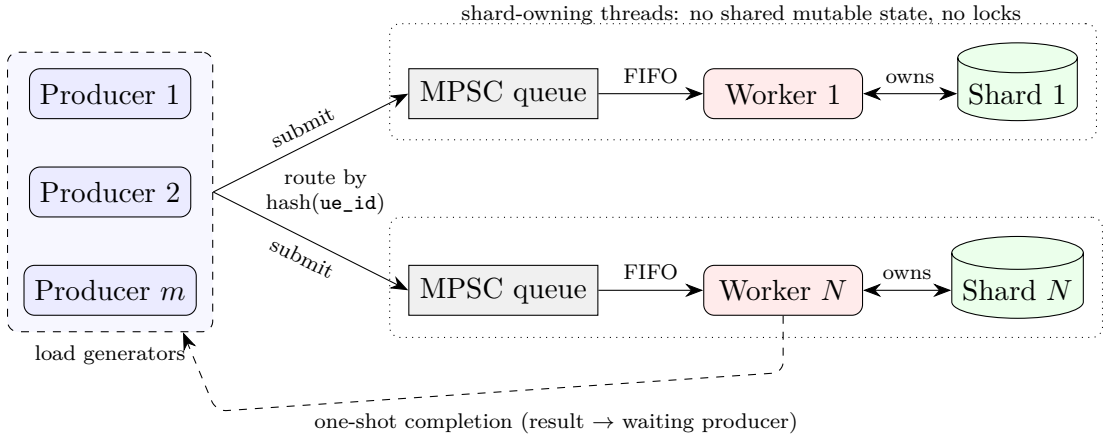


Figure 5.2: Shared-nothing, thread-per-shard architecture. Producers submit request messages to a shard's bounded, blocking MPSC queue (selected by a pinned hash of the `ue_id`) and wait on a single-use completion handle. Each shard is read and written by exactly one worker thread, so there are no locks on context state. Blocking on a full queue is what makes overload observable as tail latency.

5.3 Architectural choices and technology stack

The C++ side uses CMake with Ninja, the standard library, and `std::thread`. The Rust side uses Cargo and the stable toolchain. Both link the same memory allocator, which keeps the comparison about the two languages and not about two allocators: C++ links the system `jemalloc`, and Rust installs a `jemalloc` global allocator (from

`tikv-jemallocator`). Each side also keeps a system-allocator build as a control. Latency passes through the same high-dynamic-range (HDR) histogram algorithm [63] on both sides, configured the same way (range $[1\,\text{ns}, 3.6\,\text{s}]$, three significant figures). Rust uses the standard `hdrhistogram` crate. C++ uses a header-only port of the same bucketing algorithm, checked against a known distribution. Build optimisation is matched as closely as the two toolchains allow, so the Rust release profile is pinned to `opt-level = 2` against the C++ `-O2` build.

5.4 Implementation details

Both implementations follow the frozen specification field for field, and both are small enough to read end to end. The C++ core is 674 lines across eight headers and two translation units. The Rust core is 762 lines across eight modules. Dropping blank lines and comments leaves 517 and 611. Both counts describe the build the campaign actually measures. A build flag switches on the two attribution probes of Section 5.4.4, and the conditional blocks that select them are left out of the count, because counting them would describe neither the compared build nor the probe.

The Rust version is therefore the larger of the two, by 13% in raw lines and 18% in code lines. That belongs in the text and not in a footnote, for two reasons. Line count is one of the software-quality proxies this study set out to collect. And the difference is not stray verbosity: it is the net of several forced idioms pulling in opposite directions, which Section 5.4.4 takes apart file by file.

The rest of this section works outward from the data model to the concurrency machinery. Three of the divergences it describes carry a measured consequence, and those are the ones to read closely. The record layout differs by 8 bytes per context, the request representation differs in both size and type safety, and the bounded queue is a primitive Rust supplies outright where C++ has to assemble one by hand. Everything else is a matter of naming or mechanism. Several of those cost source lines, which Table 5.3 accounts for, but none changes what the two cores do.

5.4.1 The UE context and the state machine

Each UE context stores its identifier, a four-value lifecycle state (`Idle`, `Connected`, `Reconfiguring`, `Released`), a fixed-size dummy security context, and a dynamic list of data radio bearers, alongside the cell, transaction and counter fields that Figure 5.3 sets out in full. The bearer list is deliberately a growable container (`std::vector` / `Vec`) and not a fixed array, so that the allocator study (RQ3) observes realistic per-context allocation. Reads return an immutable snapshot (a `UeContextView`) copied out of the worker, not a reference into live shard state. That keeps the shared-nothing guarantee intact and keeps the two implementations semantically identical.

Identical field declarations do not, however, produce identical objects. Measured on the evaluation host, a `UeContext` takes 96 bytes in C++ and only 88 bytes in Rust. Layout policy accounts for it. C++ guarantees that non-static data members sit in declaration order, so the narrow fields the specification places between wider ones open two holes of alignment padding. Rust carries no `#[repr(C)]` annotation here, so it is free to reorder

fields, and it does. It packs the small scalars together and both holes close.

The specification does not try to erase that difference, and the reason is worth stating. Pinning the Rust layout with `#[repr(C)]` would penalise Rust for nothing but obedience to a C++ padding rule. Hand-reordering the C++ declarations would be an optimisation the specification asks of neither side. The gap is therefore recorded as a language-level effect and then bounded. At the 5 000 active UEs the sweep uses, the 8-byte gap comes to roughly 40 kB. Resident set sizes are measured in megabytes, so this does not matter to the RQ3 comparison.

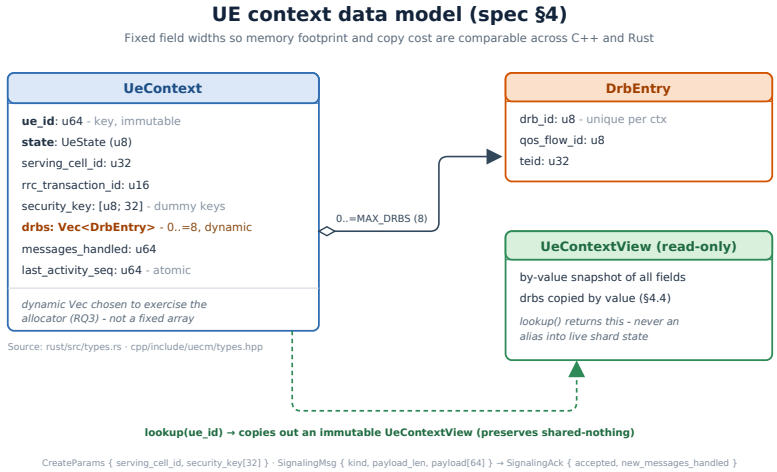


Figure 5.3: The UE-context data model shared by both implementations. Field widths are fixed so that memory footprint and copy cost are comparable across languages. The data-radio-bearer list is a dynamic `Vec`/`std::vector` on purpose, to exercise the allocator for RQ3. `lookup` returns an immutable `UeContextView` copied out by value rather than an alias into live shard state, preserving the shared-nothing guarantee.

5.4.2 Sharding, the store, and the bounded queue

Both sides pin their hashing, so that neither standard library's hasher can become an accidental language advantage. Shard assignment runs the `ue_id` through the SplitMix64 finalizer, then reduces modulo the shard count. The finalizer is three rounds of xor-shift and multiply against constants the specification pins. It matters because the workload draws sequential identifiers, and a naive modulo would map those onto shards in a fixed repeating pattern. Scrambling the identifier first makes the offered load per shard even in both languages by construction, not by luck.

The per-shard container is each language's standard hash map (`std::unordered_map` / `std::collections::HashMap`), but the default hashers are bypassed. They differ too much to leave in place. C++'s `std::hash<uint64_t>` is usually the identity function, while Rust's default `SipHash-1-3` is cryptographically strong and correspondingly slower. Keeping both would have measured a standard-library choice, not a language. Both maps are therefore keyed through a pinned FNV-1a 64-bit hash over the eight little-endian bytes of the identifier, and a test checks that the Rust `BuildHasher` produces a digest bit-identical to the reference `fnv1a_64`.

Both maps reserve capacity at start-up so that steady-state measurements are not dis-

turbed by rehashing. Every other service parameter is shared in the same way, fixed by the specification and listed in Table D.2. Only the shard count is varied from those defaults, and only by the campaign, which sets it to match the cores it is given.

The bounded blocking queue is where the two standard libraries part company most sharply. Rust supplies the primitive the specification asks for outright. `sync_channel(capacity)` is a bounded multi-producer, single-consumer channel whose `send` blocks when the queue is full and whose `recv` blocks when it is empty, which is exactly the required backpressure in one call. The C++ standard library, through C++23, offers no bounded blocking queue at all. The same semantics are therefore assembled by hand from a `mutex`, two `condition_variable`s and a `std::queue`, in a class of 51 lines.

The point of the comparison is not that the C++ code is badly written. It is short, it uses the predicate form of `wait` so that spurious wakeups are handled correctly, and it releases the lock before notifying. The point is that it exists at all. Every one of those lines is a place where a concurrency defect could enter. And the correctness argument for it rests with the author, not with the compiler or the library: that the two condition variables are never confused, that the predicates match the notifications, that no wakeup is lost. This is the same asymmetry the fault-injection study of Chapter 6 measures, showing up here in ordinary code with no defect in it instead of in a bug put there on purpose.

5.4.3 Requests, responses, and the service lifecycle

The message type carried through that queue is the second forced divergence. Rust models a request as an `enum` with one variant per operation. Each variant carries exactly its own arguments plus a `Sender` parameterised by exactly that operation's result type. C++ has no payload-carrying enumeration in the standard idiom used here, so a request is a tagged struct instead. It holds an `OpKind` discriminant alongside *every* argument field any operation might need, paired with a single `std::promise`. That promise's value type must therefore be a `std::variant` over all four possible result types.

This has three measurable consequences. The first is size. A `Request` is 160 bytes in C++ against 96 in Rust, because the C++ struct always carries the create parameters, the 68-byte signalling message, the bearer entry, and the reconfiguration parameters, whether the operation in flight uses them or not. The Rust enum is sized by its largest variant alone. With the default eight shards holding up to 1 024 requests each, full occupancy means roughly 1.3 MB of queue-resident request storage against 0.8 MB. The second consequence is a copy cost paid on every submission, because the unused fields are default-initialised and moved along with the used ones.

The third matters most to the argument of this thesis, and it is a type-safety gap. The C++ caller has to recover its typed result from the variant with `std::get<OpResult<T>>`, which throws `std::bad_variant_access` if the worker ever writes an alternative the caller did not expect. That exception cannot fire in the code as it stands, because each `OpKind` is handled by exactly one branch and that branch stores the matching alternative. But the invariant behind it is upheld by reading two files, not by the compiler. Rust settles the same invariant by construction. The sender embedded in each variant is already typed, so there is no response union, nothing to unwrap, and no runtime dispatch on the way

back.

Starting the service allocates the shards, spawns one worker thread per shard, and hands each worker exclusive ownership of its store. The request path is then the one described in Section 5.2. Shutdown is the more delicate half, because it has to stop workers blocked on an empty queue without abandoning requests still in flight. Both implementations solve it the same way. An in-band `Stop` sentinel is pushed onto each shard's queue, and strict FIFO order means every request ahead of it completes first, so no closed flag or drain API is needed and no completion handle is left unfulfilled. The worker threads are then joined. Both languages tie all of this to object destruction, which is one of the few places the two are near-identical, and both make it idempotent, so a service that goes out of scope can neither leave detached threads behind nor stop twice.

The window between `start()` and `shutdown()` is a precondition on every operation, and it is written as one. An operation submitted outside that window is a caller error and not a runtime condition, so it is reported out of band and does not widen the closed `OpError` set of Section 5.6. C++ throws `std::logic_error` and Rust panics. Saying so explicitly was not a formality. The specification forbade the call from the start, neither implementation checked it, and in that gap the two languages did not fail alike. The episode is reported with the safety results in Section 6.4.5, because a defect found in code written to be correct carries evidence an injected one cannot.

One more piece of ownership discipline holds the shared-nothing invariant together end to end. A request is moved into the queue and not copied, and it carries its completion handle with it, so the producer and the worker never both hold a usable reference to the same request. C++ enforces this by making `Request` move-only, a property it picks up automatically from the move-only `std::promise` it contains. In Rust it follows from ordinary move semantics, and the compiler rejects any attempt to use the request after submission. The only shared mutable state in either implementation is the atomic activity counter. C++ holds it by reference, Rust as a cloned `Arc<AtomicU64>`, and both increment it with relaxed ordering because nothing branches on its value.

5.4.4 Forced idioms, and what the two cores cost

Where the two languages must diverge to satisfy the same requirement, the deviation is recorded in the specification's forced-idiom log so that it can be weighed rather than silently absorbed into the results. Table 5.1 collects the log in full, and the preceding subsections have already quantified the two entries with measurable consequences.

Table 5.1: The forced-idiom log: every point at which the two implementations diverge in order to satisfy the same specification requirement, with the measured consequence where there is one.

Concern	C++	Rust
Result type	`std::expected<T, OpError>`	`Result<T, OpError>`
Request	tagged struct: discriminant plus every argument field (160 B)	`enum`, one variant per operation, carrying only its own arguments (96 B)
Response	`std::variant` of four result types, unwrapped by the caller with `std::get`	none needed: each variant carries a sender of its own result type
Bounded queue	hand-written `mutex` plus two `condition_variable`s (51 lines)	`sync_channel(capacity)`
Completion	`std::promise` / `std::future`	a fresh `mpsc::channel()` per request
Per-shard map	`std::unordered_map` with an `FnvHash` functor	`HashMap` with an `FnvBuildHasher`
Activity counter	`std::atomic<uint64_t>` held by reference	`Arc<AtomicU64>` cloned into each worker
Teardown	destructor calls `shutdown()`	`Drop` calls `shutdown()`
Optimisation level	`-O2`	`opt-level = 2`

The log also explains the line-count difference reported at the head of this section, and it says more than the net figure does, because the contributions do not all point the same way. Table 5.3 breaks the 88-line gap down by concern. Rust is much *shorter* in exactly one place, the bounded queue, where it needs no code at all against 51 lines of C++. It is longer almost everywhere else, and three entries account for most of that excess.

The largest is the service module, for the reason given above. Because the C++ response is a single `variant`, all ten public operations funnel through one shared `submit` helper, and each typed wrapper shrinks to two or three lines. Rust's per-variant typed sender admits no such helper without generics, so every operation repeats the same sequence: create a completion channel, construct its variant, send, receive. The next entry shows the trade-off most clearly, and it is hashing. C++ meets `std::unordered_map`'s hasher requirement with a five-line functor wrapping `fnv1a_64`. Rust has to supply the whole trait apparatus, every line of it boilerplate, and what it buys is a statically dispatched, type-checked hasher instead of a callable of the conventional shape. The third entry is the module tree and public re-export list, work C++ gets for free from its header structure.

None of this is a defect on either side, and none of it shows up in the performance figures. It is the kind of engineering cost the forced-idiom log exists to expose. Rust spends source lines on trait machinery and repetition at the call site, and what it gets back are guarantees the compiler can check. It then wins much of that spend back wherever its standard library happens to supply a primitive that C++'s does not.

Counting lines, however, answers only half the question a reviewer of either codebase would ask, because it treats a line of boilerplate and a line of branching as the same thing. Table 5.2 therefore adds cyclomatic complexity, measured by `lizard` [64] over both cores. One tool applied to both languages is what makes the figures comparable, because it uses the same definition of a decision point on each side. Each language's own

tooling would count somewhat different things. The two measures then point in opposite directions. Rust is 18% longer than C++ by code lines, but its total cyclomatic complexity is 15% *lower*, 81 against 95, spread over slightly more functions (40 against 37). Both cores are undemanding by the usual thresholds, and both have a median per-function complexity of one, so a small number of functions at the top carry the aggregate.

That top is where the difference sits, and it traces back to a divergence already described, not to anything spread thin. The most complex function in the C++ core is `worker_loop` at 13, more than twice its Rust counterpart's 6. It switches on the `OpKind` discriminant and then stores the matching alternative into the response `variant`. The tagged-struct design of Section 5.4.3 therefore gathers the whole dispatch, and the correctness argument that goes with it, into one branch-heavy function. Rust's per-variant typed sender removes the return-path dispatch altogether, which is why its worker has nothing comparable to unwrap. The trade is sharper than the line count alone suggested. The extra lines Rust spends buy back branching in the one place where branching is hardest to review. And the `std::bad_variant_access` invariant that Section 5.4.3 described as upheld by reading two files lives inside exactly the function this measurement flags.

Table 5.2: Size and cyclomatic-complexity proxies for the two cores, measured with `lizard` [64] applied identically to both languages. Benchmark drivers, tests, conformance runners, the FFI seam, and the probe-conditional blocks of Section 5.4.4 are excluded. Only the compared code the parity contract constrains is counted. Complexity is one plus the number of decision points in a function. The line count is NLOC as `lizard` defines it, which also discounts brace-only and preprocessor lines. It is the same measurement quoted at the head of Section 5.4.

Measure	C++	Rust
Files	10	8
Functions	37	40
Lines of code (blank and comment lines excluded)	517	611
Total cyclomatic complexity	95	81
Mean complexity per function	2.6	2.0
Median complexity per function	1	1
Most complex function	13 (`worker_loop`)	6 (`add_drb`)

Table 5.3: Where the 88-line difference between the two cores comes from, by concern (raw lines, including comments. C++ figures merge header and translation unit. Probe-conditional blocks excluded as in Section 5.4). Rust is shorter only where its standard library supplies a primitive C++ lacks.

Concern	C++	Rust	Δ
Service (queues, workers, API)	234	278	+44
Hashing (functor vs. trait pair)	36	70	+34
Module tree and re-exports	0	26	+26
Data model	73	85	+12
Message types	48	60	+12
Configuration	18	29	+11
Shard core (state machine)	182	188	+6
Error model	32	26	−6
Bounded queue	51	0	−51
Total	674	762	+88

Two properties of the Rust core are not costs at all, and they close this section because

they are what the costs above bought. The Rust library contains *no* `unsafe` code, and it resolves no third-party crate: the core depends on the standard library alone, so nothing outside it is compiled into the artefact this study measures. Both are checked mechanically over the crate rather than asserted, and Section 6.4.6 reports the counts. Safe Rust therefore guarantees data-race freedom for this core at compile time, through the `Send` and `Sync` traits. The shard store is owned by a single worker and never shared, the request type is moved across the thread boundary, and the one piece of genuinely shared state, the activity counter, is an atomic. None of this needs a runtime sanitiser to establish, and none of it needs test coverage to hold. The C++ implementation does no manual memory management beyond STL containers and no explicit locking outside the queue class, and it was checked data-race free under ThreadSanitizer.

The two statements are not, however, evidence of the same kind, and that difference is the thesis's central safety claim in miniature. The Rust property is a theorem about the program, discharged by the compiler on every ordinary build and covering every execution. The C++ property is an observation about the executions a particular instrumented binary happened to take, and it would have to be re-established after any change, on a build most projects do not run by default. Quantifying exactly how far apart those two kinds of assurance are is the purpose of the fault-injection study in Chapter 6.

5.5 Public API and operation semantics

Both implementations expose the same small, lifecycle-oriented API against the shared specification, so the surface a caller sees is identical apart from each language's naming conventions. A service is created with a fixed shard count and a per-shard queue bound. Callers then invoke context operations, each identified by the target `ue_id`: create, setup, handle-signalling, add-bearer, reconfigure begin/complete/abort, release, and state lookup. Every operation is *total* and *typed*. It returns either a success value or a typed `OpError` drawn from a closed five-value enumeration (`NotFound`, `AlreadyExists`, `InvalidState`, `CapacityExceeded` and `InvalidArgument`). An operation that is illegal for the context's current state (Figure 3.1) returns the matching error and leaves the context untouched. It does not mutate the context, and it does not signal failure out of band.

That choice matters for fairness as much as for correctness. Neither implementation uses exceptions or error codes as a hidden control-flow channel, so the cost and the control flow of the failure path are visible and identical on both sides. It also means the same conformance vectors (Section 5.6) can assert op-for-op agreement on both the returned value and the resulting state.

Two points in that semantics need stating, because the state machine as drawn hides them. `Released` is never stored: `release_context` erases the entry, so the state is a passing label rather than one an implementation can rest in, and a second release returns `NotFound` and not `InvalidState`. And `abort_reconfigure` restores the *state* to `Connected` without restoring the serving cell `begin_reconfigure` overwrote, because the model keeps none to roll back to. That is a deliberate simplification and a semantic commitment either way, and the parity suite of Section 5.6 holds both languages to it.

5.6 Parity verification

The whole comparison rests on the two versions being functionally identical, so parity is tested and not assumed. A language-neutral conformance suite writes test vectors as sequences of operations with their expected results and an expected post-state summary. A runner in each language executes the vectors and emits a transcript in one fixed format, and the two transcripts are diffed for op-for-op agreement. The seeded suite covers the full error model, the reconfiguration begin/complete/abort paths, data-radio-bearer fill and overflow, and shard-assignment determinism. Both implementations currently pass and agree. The benchmark workloads add a second parity signal, independent of the first. For a given seed and active-UE population, the mixed workload's per-operation-kind counts are checked to be identical across languages, which confirms that both implementations are driven along exactly the same code paths.

Two things bound what that establishes, and both are answered rather than conceded. A cross-diff of two implementations cannot tell "both correct" apart from "both wrong in the same way", and that is the failure mode a shared specification and a single author make most likely. The expected results in the hand-written vectors also came from the specification by way of the person who wrote the implementations. A third implementation of the same written specification therefore acts as an independent oracle. It is written in Python, shares no code with either language, and has to reproduce all 223 hand-derived expectations before it is allowed to generate any of its own. It then generates randomised operation sequences under several profiles: realistic lifecycles, adversarial sequences weighted towards illegal transitions, sequences that sit on the capacity limits, and sequences that stress shard assignment. The model computes every expected result. A campaign of roughly 793 000 generated operations produced no divergence at all. The two implementations agree with each other, and with an independent reading of the specification, on sequences nobody chose. The generator is seeded and deterministic, so any future divergence is a permanently reproducible regression and not an anecdote.

Three further checks bound the suite from other directions. The specification's transition matrix is walked exhaustively and not sampled: every operation applied to a context in every state it can occupy. Each case asserts the exact error, the resulting state on success, and the absence of any field change on failure, including the error precedences the specification leaves unordered. The C++ implementation is then run under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer across that whole suite, and reports nothing. That matters for fairness. Safe Rust carries data-race and memory-safety freedom by construction, so without the sanitiser run the C++ arm's equivalent property would rest on not having failed yet instead of on having been checked. Miri, run over the Rust state-machine suite, likewise reports no undefined behaviour. Third, the suite's reach is measured and not asserted, because a negative result from a sanitiser is only as broad as the code that ran. Line coverage of the C++ library is 94.6% and of the Rust library 98.2%, and both implementations' core operation processors and service facades are fully covered. The two figures are not strictly comparable, because `gcov` and `llvm-cov` do not resolve an executable line alike. On the finer-grained regions `llvm-cov` counts, the Rust figure is 99.3%. Each bounds the claims made about its own implementation.

6 Experimental Setup and Evaluation

6.1 Testbed configuration

This chapter reports two kinds of result. Some follow from the compilers and the source. Others depend on the machine. Both kinds are now measured on a host that meets the protocol of `bench/TESTBED.md`, which was not true of the campaign this study ran for most of its life. Table D.1 describes the machines and keeps the split clear.

The environment used. Every performance number in this chapter comes from one dedicated bare-metal server: a single-socket AMD EPYC 7443 with 24 cores on 1 NUMA node, simultaneous multithreading off, kernel 6.17.0-23-generic. The frequency governor is fixed at performance with boost disabled, so the cores hold their base clock rather than a thermally negotiated one, and cores 2–17 carry the benchmark. The environment capture records virtualisation as none. Every run records the environment metadata of Section 4.5.

The toolchain is not the host's. Both implementations are built and run inside a pinned container image carrying Rocky Linux 8.10 (Green Obsidian) with `g++ 15.2.1` and `rustc 1.97.1`, which is the same userspace and compiler pair the earlier virtualised campaign used. Fixing the toolchain in an image is what allows a change of machine to be a change of machine only: the container shares the host's kernel, scheduler and clocksource, so it buys toolchain identity and nothing else, and it is the reason the two campaigns compared in Section 6.4.4 differ in the hardware alone.

Two properties of this host are worth stating before any tail number is read, because they are what the previous host lacked. There is no hypervisor between the benchmark and the cores, so no timeslice is lost to a machine this study cannot observe. And the kernel runs tickless on the pinned cores (`nohz_full`), which removes the periodic timer interrupt from the measured path. What the host does *not* have is `isolcpus`, and that omission is deliberate rather than an oversight: isolating the cores also removes them from every scheduling domain, and because the benchmark pins its threads to a range and relies on the kernel to spread them, an isolated set collapses all of them onto one core. Section 7.6 records the cost.

The host this study measured on first, and why it appears at all. Until late in the work the campaign ran on a virtualised host: a Intel(R) Xeon(R) CPU E5-2680 v4 guest, 16 vCPUs, kvm virtualisation, no core isolation. Three of this chapter's results were already settled there, and one was not settleable there at all. The obstacle was not a shortage of repeats. On that host the 95% confidence interval on a single variant's p99 spans 1480 μs while the four variants' p99 values differ by 293 μs, so the uncertainty about one swamped the gap between all of them.

That campaign is not discarded, and Section 6.4.4 reports it against the present one. The two share a design exactly, the same 7 shards and 7 producers over 14 load levels at 20 repeats, so what separates them is the machine. A comparison of that kind is the only direct evidence this study can offer that its instrument is now adequate to the questions it asks, and one of its findings is that a quantity this thesis once reported as a language-independent property of allocators was in part a property of that machine.

Measurement discipline common to both. On any host, Section 4.4 applies: each sweep point is a median over 20 interleaved repeats with a bootstrap 95% confidence interval. Interleaving shares interference across variants. Absolute latency suffers, but comparative C++/Rust conclusions remain valid. Throughput is counted over the measured phase only, warm-up excluded.

Table D.1 does not rank results by importance. It separates what compilers and source settle from what only the machine can settle.

6.2 Workload and evaluation measurements

A synthetic but spec-justified telecom workload is generated by a deterministic driver whose structure both implementations share, so the two versions are exercised along identical code paths. Two workloads are provided. The *signal* workload preloads a population of connected UEs and then hammers the `handle_signalling` hot path, which isolates steady-state per-message latency. The *mixed* workload is a pinned RAN lifecycle generator. It walks each UE through a realistic procedure sequence, context creation, connection setup, signalling, bearer setup, reconfiguration/handover (begin/complete/abort), and release, using a fixed SplitMix64 draw order and a per-slot expected-state model so that every operation it emits is valid. The fixed draw order means the per-operation-kind counts are identical in C++ and Rust for a given active-UE population and seed, which doubles as the parity signal described in Section 5.6. The control parameters are the target event rate and the number of active UEs, swept from nominal load through saturation and overload. The send schedule is fixed and the histogram is corrected, so a stall introduces no coordinated-omission bias [31, 63]. Three measurement campaigns are run: a load sweep for performance, the fault-injection study, and an FFI migration prototype. Figure 6.1 shows the two workloads and the procedure sequence the mixed one walks.

What "spec-justified" claims, and what it does not. The term is used precisely, because the distinction bounds what the workload can support. The *sequence* of operations is justified. It is the RRC connection lifecycle the 3GPP specifications define for a UE, from establishment through in-session signalling and bearer setup to reconfiguration and release [1, 4]. The per-slot expected-state model then guarantees that every operation the driver emits is legal for the state its context is actually in, so the workload never drives the service down a path the specification forbids. The *proportions* are not justified in the same way. How often a real cell hands over, against how often it establishes a bearer, depends on deployment, mobility, and time of day, and no public figure fixes it. The mix used here is therefore plausible and not validated. Section 7.6 reports that as a threat to

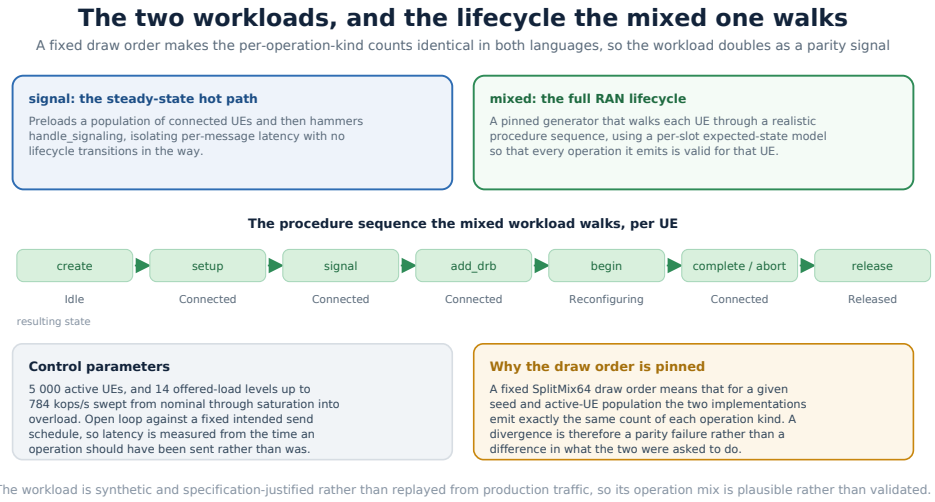


Figure 6.1: The two workloads. The *signal* workload isolates the steady-state hot path. The *mixed* workload walks each UE through the full RAN procedure sequence, with a per-slot expected-state model ensuring that every operation it emits is legal for the state its context is actually in. Because the SplitMix64 draw order is pinned, a given seed and active-UE population produce identical per-operation-kind counts in both languages, so a divergence is a parity failure rather than a difference in what the two implementations were asked to do.

external validity, and trace replay heads the future-work list for exactly this reason.

For a *comparative* question the distinction costs less than it would for an absolute one, and saying why is part of arguing that the data suits the task. Both implementations execute the same sequence in the same order for a given seed and active-UE population, and the per-operation-kind counts verify it. An unrepresentative mix therefore shifts both languages equally and cannot manufacture a difference between them. An unvalidated operation mix bounds how far the numbers travel. It does not compromise the fairness of the comparison they are used to make.

Table D.2 gathers the configuration of the service, the workload and the campaign in one place. Those values are otherwise spread across the specification, Chapter 5, and this chapter. That is convenient while reading each in turn, and unhelpful to anyone trying to reproduce the campaign or to judge whether a parameter was chosen to suit the result. Every entry in the campaign block is either fixed by the specification or generated from the measurement logs, so none of it can drift out of step with the data reported below. Two operating points are used throughout what follows, and the first of them is fixed rather than derived. The representative point sits at 56 kops/s, against a measured throughput plateau of 392 kops/s, and saying why it is fixed is worth a short paragraph because the alternative was tried and failed.

The rule this study began with took the busiest swept load at which all four variants still held their p99 below 5 ms. It is retained as a diagnostic and is no longer the operating point, because it turns out to measure the machine and not the service. On the virtualised host it returned 56 kops/s, where p99 sat just under the deadline, so it was locating where that host's interference reached 5 ms. On a quiet host p99 stays near 74 μs until capacity binds, so the same rule climbs until it collapses the representative point onto the saturation knee. One rule, two quantities, decided by how noisy the machine was. Fixing

the point costs the appearance of derivation and buys two things worth more: the hosts are compared at the same offered load, and the point stays below saturation on all of them, which every below-saturation claim in this chapter already assumes.

6.3 Evaluation data

The campaigns behind this chapter produce datasets that differ in kind and not only in content: one load sweep, three small paired experiments, a categorical fault catalogue, a build and footprint measurement of the two toolchains, and two further sweeps kept for comparison. What follows describes each as captured, because judging whether the analysis is sound takes knowing what was recorded and at what level of detail. Every measured quantity this chapter quotes reaches the page as a generated macro. That script (Section 4.5) rebuilds all of them, along with every figure and table body, from the raw logs, so a rerun moves the prose and cannot leave a headline number behind. The software-quality proxies are not gathered here but reported where they read most clearly: beside the forced-idiom log they explain (Table 5.2), and beside the FFI seam, the only place either implementation contains `unsafe` (Table 6.12).

The performance sweep. Each of the 1120 runs of the factorial design in Table D.2 contributes one record, written as newline-delimited JSON. A campaign is therefore an append-only log and not a summary table. Nothing is aggregated at capture time, and every statistic in this chapter is recomputed from the raw records. A record carries the run's configuration (language, allocator, workload, shard and producer counts, target rate), its outcome (operations completed, errors, wall-clock duration, achieved throughput), the latency distribution read out of the HDR histogram (mean, standard deviation, the quartiles, p50, p90, p99, p99.9, p99.99 and the maximum, in nanoseconds), a shorter block for inter-operation delay variation carrying the mean, the standard deviation, p50, p99, p99.9, the maximum and a sample count, the resource counters taken through `getrusage` (peak resident set size, user and system CPU seconds, CPU utilisation), and the per-operation-kind counts. Runs on a `jemalloc` arm add that allocator's own accounting of the process, six fields that decompose the resident set, and the system-allocator arms store those as unavailable rather than as zero, on the same principle as the governor and turbo state. Keeping the kind counts in every record is what makes a divergence between the two languages show up as a parity failure (Section 5.6) instead of being mistaken for a performance difference.

Two files travel beside the campaign. Each run persists its complete HDR bucket contents to a sidecar, so the repeats can be pooled into one distribution before a far-tail percentile is read off (Section 4.4), and a separate file records `perf` counters at two scopes, the end-to-end path and the operation on its own.

The component decomposition and contended primitives. The second measurement level of Section 4.2.1 yields a much smaller dataset of a different shape. For each of 15 interleaved repeats it records the batch-mean cost in both languages, once for the operation called directly and once for the same operation called through the threaded

service. A third dataset, of the same paired shape over 20 repeats, records sustained throughput through the bounded queue and completion primitives alone (Section 6.4.2). The FFI comparison of Section 6.4.6 has the same structure over 15 repeats. All three are paired by construction, which is what the exact paired test of Section 4.4 requires, and all three predate the environment record described below, which is why each is read as a ratio between its two arms.

The adoption-cost measurement. Only one of the RQ4 axes beyond the seam is a timing measurement, which is what makes this a dataset of its own kind. Build time is recorded over 15 interleaved repeats at 14 parallel jobs on the development machine, alternating the toolchains so the ratio survives a change of machine even though the seconds do not. The static footprint and the resolved dependency graph are read from the linked binaries and the manifests, so the same inputs give the same figures anywhere. Section 6.4.6 keeps the two apart for that reason.

The fault-injection catalogue. This dataset is categorical and not numeric. Each of the 14 defect cases yields four tool logs: the AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer output for the C++ program, and the `rustc` output for the safe-Rust counterpart. Each case also yields one row in a machine-readable taxonomy, recording the three sanitiser outcomes, the detection stage, the compiler result with the diagnostic codes that fired, and the expectation registered before the study. The two original boundary cases yield a second table from the model checkers, and the `unsafe` one a third from the interpreter. Carrying the expectation in the data instead of in the prose is what lets the runner report a mismatch in either direction. The table and figure in Section 6.4.5 are generated from those logs (Section 4.5), not transcribed from them.

What travels with the data. Every load sweep writes the environment record of Section 4.5 beside its results, *including* the fields a given host does not expose. That last detail matters more than it sounds. The governor and turbo state are stored as unavailable rather than left out, so the limitation documented in Section 6.1 is a property of the dataset itself and not only of the prose describing it, and a reader can check it without taking it on trust. The three small paired experiments predate that discipline and do not carry one, which is why each is read as a ratio between two arms measured alternately and never as an absolute.

The two comparison campaigns. Two further datasets exist and neither is merged with the sweep. The virtualised campaign shares the sweep's design exactly and predates all of the instrumentation above: its records carry the percentiles and the resource counters and nothing else, no quartiles, no p99.99, no jitter, no allocator accounting, and no sidecar or counters file beside them. That absence is itself the measurement-validity argument of Section 6.4.4, which is why the campaign is kept rather than deleted. The second bare-metal campaign carries the same record shape as the reported sweep, including the allocator accounting that lets Section 6.4.4 decompose the resident set, but runs a different shard count, load grid, repeat count and toolchain. The same scripts read all three, so

nothing in the analysis changes between them, and each has its own generator and macro prefix so that no figure from one can reach a sentence about another.

6.4 Evaluation results

The safety results do not depend on the measurement host at all, and the migration results depend on it along one axis only. The performance and resource results do, and Section 6.1 states what that host can and cannot carry. Every figure and table below is measured on it unless stated otherwise.

Resolved and unresolved outcomes are reported side by side, because saying at the outset which is which keeps a null result from being read as an absent one. Every comparison below resolves on this host, including the far tail *below* saturation, which was the one open question of this study for as long as it measured on a virtualised machine. Section 6.4.4 sets that change beside the earlier host, because the difference between the two is a result about measurement rather than about either language, and it is reported last among the performance results for that reason. One resolved outcome is resolved only as a measurement and not as a mechanism, the divergence past saturation, and Section 6.4.2 reports how far the mechanism could be narrowed.

6.4.1 Below saturation (RQ1, RQ3)

The hypothesis under test is H1 (Section 1.3): equivalence below saturation, with any difference confined to the far tail. The absolute figures belong to the host of Section 6.1, each point aggregates 20 interleaved repeats, and reported values and shaded bands are medians with nonparametric bootstrap 95% confidence intervals.

Table 6.1 summarises the per-operation latency distribution at the representative load of Section 6.2, 56 kops/s, a point at which all four build variants are simultaneously healthy and so a fair place to compare them.

Two readings of that table are worth keeping apart, because they answer different questions. The first is magnitude, and it is negligible: the four variants sit within $2\,\mu$s of one another at the median, 2.1%, which no deployment would notice. The second is *ordering*, and it is legible: the two Rust intervals sit below the two C++ intervals with no overlap between the languages, while the two allocators of one language overlap each other. So the table already suggests what the paired test below confirms. On the previous host it could not, since every interval at this percentile overlapped every other, and the ordering had to come from the test alone.

That agreement extends up the distribution, and stating why takes one comparison. The p99 values differ by $2\,\mu$s across variants while the widest 95% confidence interval on a single variant's own p99 spans $1\,\mu$s, so the uncertainty about one variant is smaller than the gap between them. That relation is what licenses an ordering, and it is the relation this study could not obtain for most of its life: on the virtualised host the same two quantities were $293\,\mu$s and $1480\,\mu$s, the wrong way round by a factor of five. Section 6.4.4 sets the two hosts side by side, because the difference between them is a result about measurement rather than about either language.

The tail rows of Table 6.3 therefore carry a finding rather than a shrug, and the instrument that carries it is the corrected p-value and not the interval. Both p99 rows and both p99.9 rows clear the Holm correction, 4 rows in all, with the same direction as at the median. Their intervals also exclude zero, and Section 4.4 fixes in advance that an interval in this block is not read as a finding, so those are reported and left to one side. A rule dropped once the machine improves was never a rule, and keeping it costs nothing here: the correction resolves these rows on its own.

Predictability is evidenced as far as p99.99, and that column is a different statistic from the rest of its table. It is one percentile of the 20 repeats' histograms concatenated, between 3 360 051 and 3 360 084 operations per variant, which is what Section 4.4 means by pooling: a single run holds too few observations past p99.99 for the percentile to mean anything, and every run persists its full histogram buckets so the repeats can be combined after the fact. Pooling takes the operation as the unit of analysis, so the column has no across-repeat interval and none is printed. Read that way the far tail separates the languages outright: the widest gap between variants is 87 μs, 97% of the smallest, and both Rust variants sit below both of the others.

Table 6.1: Per-operation latency at a sustained offered load of 56 kops/s (mixed RAN-lifecycle workload, 7 shards). Values in microseconds: the median over 20 repeats above its nonparametric bootstrap 95% confidence interval. The interval is printed in full rather than as a symmetric half-width because it is strongly asymmetric at these percentiles, and a half-width wide enough to cover the upper bound would place the lower one below zero. †The p99.99 column is the exception, and deliberately so: it is one percentile of the 20 repeats' histograms concatenated, 3 360 051 to 3 360 084 operations per row, because a per-run p99.99 rests on too few observations to read (Section 4.4). Pooling makes the operation the unit of analysis, so that column has no across-repeat dispersion to quote and carries no interval. Together with the maximum, it is one of the two columns here that the paired family of Section 4.4 does not test, and for the same reason: the maximum is the limiting case of a statistic read off exactly the operations interference lands on, which is the class pairing cannot rescue.

Variant	p50	p99	p99.9	p99.99†	max
C++ (system alloc)	73.7	76.4	81.2	142.0	243.1
	[73.5, 73.7]	[76.2, 77.0]	[80.8, 81.6]		[239.2, 275.8]
C++ (jemalloc)	73.5	76.6	80.2	176.4	246.3
	[73.3, 73.6]	[76.4, 76.9]	[79.5, 82.0]		[228.8, 259.5]
Rust (system alloc)	72.4	75.0	77.3	92.1	145.2
	[72.3, 72.5]	[74.7, 75.2]	[77.0, 78.7]		[115.9, 178.2]
Rust (jemalloc)	72.1	75.6	77.8	89.5	121.4
	[72.1, 72.3]	[75.6, 75.7]	[77.7, 77.9]		[101.8, 179.6]

Variation from one operation to the next. The percentiles above describe the distribution. They do not describe how far consecutive operations differ from each other, which is the quantity a scheduler downstream of this service would feel, and RFC 3393 names it inter-operation delay variation (Section 4.4) [61]. Table 6.2 reports it, descriptively, because no research question asks for it and adding it to the tested family would spend two slots of the Holm budget on a metric no claim rests on. The reading is short. Median jitter across the four variants runs 1.37 to 1.72 μs and its p99 runs 19 to 25 μs, so at the median the widest gap between variants is 26% of the smallest. Against the operation it perturbs, median jitter is 2% of the median latency and its p99 is 33%, so consecutive operations differ by a small fraction of one of them and not by a whole one.

What separates the four variants is smaller than what separates one load level from the next, which is the same conclusion the second bare-metal campaign reaches independently (Section 6.4.4).

Table 6.2: Inter-operation delay variation at a sustained offered load of 56 kops/s, in microseconds, as a median over 20 interleaved repeats with a bootstrap 95% interval. Measured within one producer and never across producers, since two producers share no schedule and a difference between them is not a variation either one experienced. Reported for completeness rather than as evidence: no research question names this metric, and it is absent from the paired family of Table 6.3.

Variant	p50	p99	p99.9	max
C++ (system alloc)	1.5	19.4	44.3	169.9
	[1.3, 1.6]	[13.9, 23.1]	[42.7, 47.3]	[165.8, 202.9]
C++ (jemalloc)	1.5	23.4	44.0	173.8
	[1.4, 1.6]	[18.1, 31.3]	[41.7, 46.0]	[154.9, 185.9]
Rust (system alloc)	1.4	25.1	45.1	106.7
	[1.2, 1.5]	[17.8, 31.7]	[42.4, 46.6]	[60.7, 113.0]
Rust (jemalloc)	1.7	25.4	43.8	74.7
	[1.6, 1.8]	[19.7, 29.7]	[41.1, 46.2]	[57.8, 115.3]

Testing the comparison rather than eyeballing it. Overlapping intervals are a weak instrument in both directions, so the comparison is also tested directly, by the procedure fixed in Section 4.4 and reported in Table 6.3. What licenses the pairing is a property of this campaign and not of the statistic: the four variants are interleaved inside every repeat, so the i-th run of each met the same ambient conditions and whatever interference that block carried cancels in their difference. That is the second thing the interleaving discipline buys, beyond the fairness argument it was adopted for.

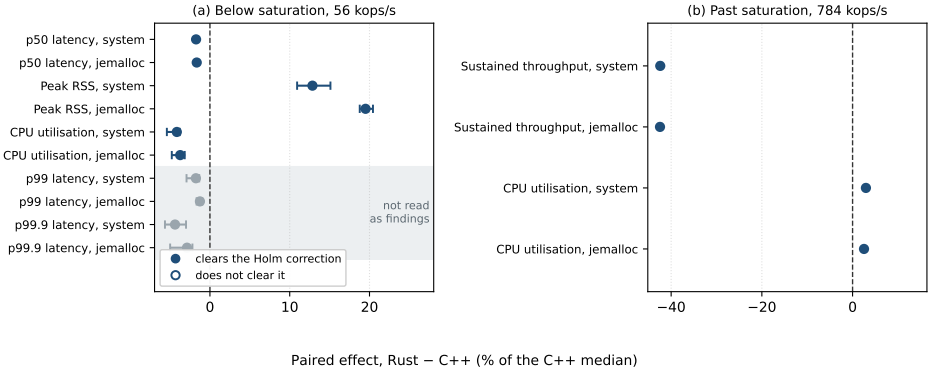


Figure 6.2: Every test in Tables 6.3 and 6.7 as a paired effect with its bootstrap 95% interval, normalised by the C++ median so that microseconds, mebibytes, percentage points and kops/s can share an axis. Filled markers are the 14 of 14 tests that clear the Holm correction, which on this host is all of them. The two allocators of a metric are drawn adjacent, so an effect that reversed sign between them would appear as a pair straddling the zero line. The shaded rows are the high percentiles, computed from exactly the operations interference falls on, which Section 4.4 fixes in advance as rows whose *intervals* are not read as findings. Their corrected p-values are read, and the shading marks the instrument that is set aside, not the rows. The metrics keep their own units in the tables.

Figure 6.2 draws the same family on one axis, and three things it contains are easier to see there than in a table of fourteen rows: that every effect survives the correction, that no effect changes sign when the allocator changes, and how much wider the tail intervals are than the rest.

Table 6.3: Exact paired permutation test of C++ against Rust at 56 kops/s, with the allocator held fixed. The effect is the median of the 20 paired differences (Rust − C++), shown above its bootstrap 95% confidence interval. The test statistic is their mean. p is exact over all 2^{20} sign assignments, and the final column applies the Holm correction across the whole family of 14 tests. The attainable floors are 0.0000019 uncorrected and 0.000027 after correction. The rows are blocked by the criterion set out in Section 4.4: pairing rescues the statistics in the upper block, so a consistent interval there is evidence, whereas the high percentiles below are computed from exactly the operations interference falls on, so an interval excluding zero is not read as a finding. On this host both blocks clear the correction, so the rule costs the lower block nothing and is applied anyway. Achieved throughput is not tested, for the reason given in the same section, and appears in Table 6.9 instead. Measured on the bare-metal host of Section 6.1.

Metric	Allocator	Effect (Rust − C++)	p	Holm
Robust to interference on single operations				
p50 latency (µs)	system	−1.3 [−1.3, −1.2]	0.0000019	0.000027
p50 latency (µs)	jemalloc	−1.2 [−1.3, −1.2]	0.0000019	0.000027
Peak RSS (MiB)	system	+1.4 [+1.2, +1.7]	0.0000019	0.000027
Peak RSS (MiB)	jemalloc	+2.5 [+2.4, +2.6]	0.0000019	0.000027
CPU utilisation (%)	system	−5.1 [−6.7, −4.7]	0.0000019	0.000027
CPU utilisation (%)	jemalloc	−4.6 [−5.9, −3.9]	0.0000019	0.000027
Determined by it: high percentiles				
p99 latency (µs)	system	−1 [−2, −1]	0.0000057	0.000027
p99 latency (µs)	jemalloc	−1 [−1, −1]	0.0000038	0.000027
p99.9 latency (µs)	system	−4 [−5, −2]	0.00016	0.00016
p99.9 latency (µs)	jemalloc	−2 [−4, −2]	0.0000095	0.000027

All 14 of the 14 tests in the family survive the Holm correction, and the smallest adjusted value sits at the family's own floor of 0.000027. A family in which nothing fails invites one obvious objection, that the threshold is doing no work, so it is worth naming what would have failed. Peak resident set size under `jemalloc` did not clear the correction on the virtualised host, despite an interval nowhere near zero, and Section 6.4.3 takes up why it does here. Neither did any tail row.

14 paired intervals exclude zero, in both blocks, so the reading rule of Section 4.4 changes no verdict here. Fixing it in advance was still the whole precaution, because that outcome was not guaranteed: an earlier seven-repeat pass of this same campaign produced tail intervals excluding zero, and more repeats then removed them again.

The allocator control supplies a second check, independent of the first. Every comparison is run twice under conditions that should not change a language effect, so an effect that really is the language ought to keep its sign whichever allocator is linked. Here p50 latency, p99 latency, p99.9 latency, Peak RSS and CPU utilisation all do, and 0 reverse. That the control confirmed here and did not veto is no evidence it was unnecessary. Section 7.6 records the earlier campaign on which it did veto, and a control earns its place by what

Table 6.4: How large each effect could be. For every comparison at 56 kops/s, the observed effect against the tightest margin within which the two variants could be called equivalent, and the verdict Table 6.3 reaches. The two questions are kept in separate tables on purpose: one asks whether a difference is there, this one asks how large it could be, and a row can matter in one and not the other. On the host reported here every row resolves, so the last column reads the same throughout and the margins are what carry the information: a resolved difference of a microsecond and a resolved difference of a millisecond mean different things to a deployment. On the virtualised host of Section 6.4.4 5 of these rows did not resolve, which is what the table was built to report. Latency margins are in microseconds, peak resident set size in mebibytes, CPU utilisation in percentage points.

Metric	Allocator	|Effect|	Equivalent within	Verdict
p50 latency (µs)	system	1.3	1.3	significant
p50 latency (µs)	jemalloc	1.2	1.3	significant
p99 latency (µs)	system	1	2	significant
p99 latency (µs)	jemalloc	1	1	significant
p99.9 latency (µs)	system	4	5	significant
p99.9 latency (µs)	jemalloc	2	4	significant
Peak RSS (MiB)	system	1.4	1.6	significant
Peak RSS (MiB)	jemalloc	2.5	2.6	significant
CPU utilisation (%)	system	5.1	6.6	significant
CPU utilisation (%)	jemalloc	4.6	5.7	significant

it would have caught.

Resolving a difference is not the same as sizing it, and Table 6.4 is where the size goes. Every row is significant here, so the verdict column carries no information and the margins carry all of it: a resolved difference of a microsecond and a resolved difference of a millisecond mean quite different things to a deployment. At the median the effect could be no larger than 1.3 μs, and in the tail block no larger than 5 μs. Both are below anything a procedure budget of a few milliseconds would notice, which is the useful form of this result: the differences are real, reproducible, and operationally negligible. That table was built to report the opposite case, and on the virtualised host it did, with 5 of its 10 rows unresolved and a tail margin running out to 1298 μs, which is wider than any difference a deployment would care about and was the campaign declining to answer.

The median itself is a resolved difference and not a null: 1.3 μs in Rust's favour, consistent under both allocators. The interval on it reaches 1.3 μs against a median of 70 μs, and the temptation is to report that as a bound on the language effect. That reading would be wrong, for the reason set out next.

What the end-to-end number is made of. Every latency the sweep reports is an end-to-end round-trip, and Section 4.2.1 set out why that bundles two costs of quite different character: the operation itself, executed inside a single thread and generated by the compiler, and the cross-thread submit-and-complete handoff around it, charged to the operating system. Only the first can plausibly differ between the languages, and the sweep cannot separate the two.

That matters for the bound just quoted, because a bound says only as much as the quantity it is a fraction of. Both costs are therefore measured directly by the two-level method described there, in both languages and at the sweep's own shard count and context population. `ShardCore` with no queue in the path gives the operation, `Service` gives the

Table 6.5: Where an end-to-end operation's cost actually goes, measured at 7 shards and 5 000 active UEs with a single uncontended caller. Each row is the median over 15 interleaved repeats above its bootstrap 95% confidence interval, in the unit its own magnitude calls for: the two terms differ by more than two orders of magnitude, which is the point of the table. p is the exact two-sided paired permutation test of C++ against Rust on the repeat-wise differences. The handoff is obtained as the difference between the two measured rows, so it carries the uncertainty of both. It predates the move to bare metal and carries no environment record, so the absolute nanoseconds name no host. Both arms ran alternately in one session, which is what the share and the ratio rest on.

Component	C++	Rust	p
Operation alone, no queue in the path (ns)	34.5	21.3	0.000061
	[32.4, 35.2]	[21.0, 21.5]	
Cross-thread submit-and-complete handoff (µs)	13.58	13.59	0.439
	[13.38, 13.83]	[12.69, 14.52]	
End-to-end round-trip, uncontended (µs)	13.62	13.61	0.421
	[13.41, 13.87]	[12.71, 14.54]	
Operation's share of the round-trip	0.250 %	0.160 %	

operation plus the handoff, and their difference is the handoff. Table 6.5 gives the result, and it reframes the whole performance comparison.

The operation is 34.5 ns in C++ and 21.3 ns in Rust. The handoff is 13.58 μs and 13.59 μs. The work the languages actually perform is therefore at most 0.250% of what the sweep measures. The complementary 99.750% of every latency in Table 6.1 is the operating system moving a message between two threads.

That shape is not peculiar to this system or to this pair of languages. On a shared-nothing database partitioned one per worker thread like this service, Zhou et al. report that once the per-operation work is optimised what dominates the rest is communication across isolation boundaries and not the operation [65]. The result below is therefore a general property of the design and not an artefact of this measurement.

Two conclusions follow, and they pull in opposite directions.

The first is that the sweep cannot bound the language effect, and the bound quoted above must not be read as though it does. An interval of 1.3 μs on the paired median difference is 38 times the operation's own cost, so a language effect could be several hundred percent of the work being done and still sit comfortably inside it. What the sweep does establish is weaker and still useful: neither language disturbs the scheduler-dominated round-trip, which is what a deployment of this shape would actually experience. The limit comes from how the measurement is built and not from the host, so unlike the tail result it will not be fixed by moving to bare metal. It is fixed by measuring the component on its own, which is what this decomposition does.

The second is that once the component *is* measured on its own, the languages are not equivalent. Rust runs the bare operation about 1.6× faster, the difference shows up in the great majority of repeats, and the paired test puts its p nearly three orders of magnitude below the conventional threshold. This decomposition is a separate experiment from the sweep and no part of its Holm family. It runs 15 repeats instead of 20, so its own attainable floor is 0.000061 and not the 0.000027 quoted in Section 4.4, and no correction is applied to it. The same holds for the interoperability comparison of Section 6.4.6. The sweep does resolve language differences in *time* of its own, at the median and at both tested tail

percentiles, but every one of them is a difference in a quantity that is 99.750% handoff. This decomposition is the only place where the language's own work is timed, and the sweep is blind to it by more than two orders of magnitude.

Figure 6.3 draws both conclusions on one page, and its second panel is the argument in one image: the instrument sits entirely above the quantity it would have to constrain.

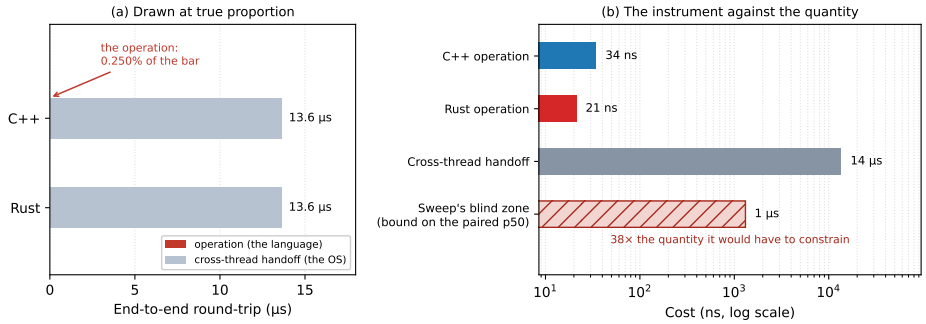


Figure 6.3: Where an end-to-end operation's cost goes, and why the sweep cannot bound a language effect. **(a)** The round-trip at true proportion: the operation the language performs is 0.250% of the bar, the remainder the cross-thread handoff. **(b)** The same quantities on a log axis against the smallest paired difference the sweep can resolve, shown as a blind zone. Both operation measurements fall inside it by a factor of 38, which is why the differences the sweep does resolve in time are differences in the round-trip and not in the work the language performs.

The attribution needs care, because the obvious reading is the wrong one. Nothing here implicates the borrow checker, bounds checking, or ownership. The most likely source is already recorded in the forced-idiom log of Table 5.1. The per-shard container is each language's standard hash map, and those are different data structures. C++'s `std::unordered_map` has to be node-based to keep its iterator- and reference-stability guarantees, so a lookup chases a pointer out of the bucket array into a separately allocated node. Rust's `HashMap` is an open-addressed table that keeps entries inline and usually resolves a lookup within one cache line. The hash function is pinned to FNV-1a on both sides (Section 5.4) precisely so that hashing could not be the explanation, which leaves the container layout. That is a real and reportable difference between the two languages as they are actually used, because a developer reaching for the standard hash map gets whichever one their language ships. It is not evidence about Rust's safety model and should not be quoted as such. Confirming the mechanism would need the cache-miss counters Section 7.6 lists as the clearest instrumentation gap.

Two limits keep the finding the size it is. It is a single-threaded, uncontended microbenchmark, so it says what the operation costs and not what a loaded system does with it, and it is one operation on one data structure. Generalising from it to "Rust is 1.6× faster" would repeat exactly the error this thesis criticises in the average-case literature.

Latency, throughput and the knee across the sweep. Figure 6.4(a) plots p99 latency against offered load, and the shape it shows is the one Figure 3.2 sketched, with one feature sharper than the sketch. Through the whole tracking region p99 is *flat*: it reads 78 μs at the lightest load and 74 μs at the top of the region, a drift of -4 μs across a fifty-fold increase in offered load. A queueing model would predict a gentle climb, and

there is none to see, because until a worker saturates its shard the queue never holds more than a request or two and latency is service time, not waiting time. Then, in the single step of the rate ladder that follows, p99 rises by four orders of magnitude at once, from 74 μs to seconds. That is the overload cliff, and how abruptly it arrives is the finding. One increment of offered load separates a service meeting millisecond deadlines from one missing them by three orders of magnitude. A design with bounded, blocking queues has no graceful degradation to offer in between, and the flatness before the cliff is why: there is no rising-latency region to warn an operator that capacity is close.

Section 4.4 tests no percentile past the knee, for two reasons. Latency there is accumulated schedule backlog and not service time, and the measured phase is cut off at a fixed width, so the reported percentiles saturate near it and every one above the knee is a lower bound. What stays interpretable is how much work each variant still completes, which Section 6.4.2 takes up.

Panel (b) shows achieved throughput against the same axis, and it is the more informative of the two, because an average over a whole run is far less sensitive than a percentile to interference landing on individual operations. All four variants track the ideal offered-equals-achieved line up to an offered 392 kops/s, and the lowest plateau any of them reaches is 392 kops/s. Each knee is bracketed by the rate ladder and not pinned by it: the next rung is where an arm stops tracking, so its true saturation point lies between the two and the sweep steps over it. Achieved throughput matches offered load below the knee to within 0.0%, so the operating point of Table 6.1 is a genuinely *sustained* load and not a nominal one, and the latencies reported there are not those of an already-queueing system.

The knee does *not* sit at the same place for all four variants, and that is worth stating plainly because the identical thread-per-shard structure predicts that it should. The two C++ arms stop meeting their offered rate after 392 kops/s and the two Rust arms carry one rung further, to 448. The separation is by language and not by allocator, as it is everywhere else in this regime. At the rung between the two limits, 448 kops/s, the consequence is the largest throughput gap anywhere in the sweep: Rust still serves 448 kops/s while C++ has already collapsed to 162, a factor of 2.8, in 40/40 paired repeats.

Three things bound what that rung licenses. It is *not* added to the tested family of Section 4.4, because the operating point was identified after seeing the sweep and the family is fixed in advance precisely so that a point chosen that way cannot enter it, whatever its p-value would be. It is reported descriptively instead, with its per-repeat agreement, and the tested comparison remains the pre-committed overload point of Section 6.4.2. One rung of a geometric ladder is also coarse: the true C++ limit lies between 392 and 448 kops/s, so the difference in *capacity* is bounded by the ladder's spacing and not measured to better than that. And the direction here is the opposite of the direction deeper into overload, where C++ sustains more (Section 6.4.2). Both are real, they belong to different loads, and neither cancels the other.

Figure 6.5 holds the load fixed instead and sweeps the percentile. The four variants are close at the median and fan out as the percentile rises, which is the expected shape for a heavy-tailed distribution and is why a percentile-aware methodology is used throughout.

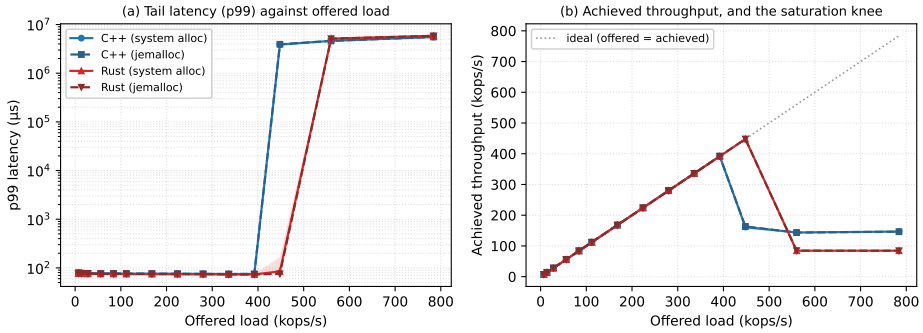


Figure 6.4: The load sweep, both panels against offered load. **(a)** Tail latency (p99) for the four build variants on a log axis. It is flat across the whole tracking region and then rises by four orders of magnitude in one step of the ladder. **(b)** Achieved throughput, with the dotted line marking the ideal offered-equals-achieved regime. The C++ curves leave that line after an offered 392 kops/s and the Rust curves after 448, so 448 kops/s is the only load in the sweep at which the two languages differ in *throughput* with Rust ahead. Lines are medians over 20 repeats and shaded bands are bootstrap 95% confidence intervals in both panels, which share the variant styling of the legend in (a). Throughput is counted over the measured phase only, so the warm-up iterations bias neither the numerator nor the denominator.

Here the fan opens by language: the two Rust curves stay below the two C++ curves across the whole range, and the gap widens with the percentile.

Whether that is a language effect or a measurement artefact is a question the right-hand panel is built to answer, and on this host the answer is the first. It sets the spread between the four variants against the widest interval on any one of them, and the interval stays the smaller quantity at every percentile it covers, reaching at most 0.6 times the spread at p99.9. So the fan marks where the languages differ and not where the measurement runs out. That panel was added when the answer went the other way, and it is retained now that it does not, because a reader has no way to tell a resolved comparison from an unresolved one unless the study shows the same diagnostic in both cases. It stops at p99.9, since the far tail is reported pooled and a pooled percentile carries no across-repeat interval to weigh a spread against.

6.4.2 Past saturation (RQ1)

RQ1 asks how the two languages compare under increasing load *and overload*, and everything reported so far has been below the knee. This section is the other half. The operating point is 784 kops/s offered, defined as the deepest swept load at which every variant has stopped meeting the offered rate, and the metrics are the two that remain interpretable there (Section 4.4). Table 6.6 gives what each variant still sustains and Table 6.7 tests the comparison.

The languages are not equivalent in this regime, and the margin is not marginal. C++ sustains 147 kops/s where Rust sustains 84, a factor of 1.7. The paired difference is at least 62 kops/s under either allocator, it holds in 20/20 repeats, and its Holm-adjusted p sits at the corrected floor of the whole family of 14 tests. This is the largest language effect in the tested family and the most reproducible effect in the study. One larger ratio appears in the sweep, at the rung between the two languages' saturation limits (Section 6.4.1), and it runs the other way and is not tested, for the reason given there.

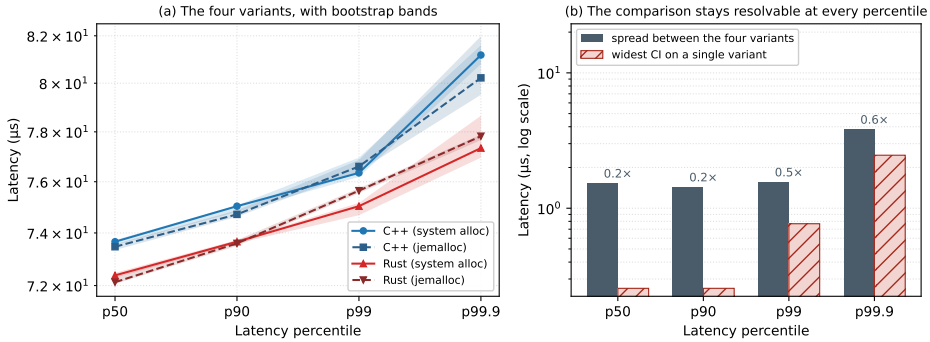


Figure 6.5: Per-operation latency across the percentile range at 56 kops/s. **(a)** The four variants as medians over 20 repeats, each with a bootstrap 95% band. The bands are the point: drawn as bare lines the curves would read as a claim of equivalence, and what the bands show instead is two languages separated at every percentile with the gap widening. **(b)** How far that separation can be trusted. The spread between the four variants is set against the widest confidence interval on any one of them, with the ratio annotated above each pair of bars. Once that ratio passes one, the uncertainty about a single variant is larger than the gap between all four and no ordering can be read off. It does not pass one anywhere here, peaking at 0.6 at p99.9, which is what licenses reading an ordering at the percentiles drawn here. On the virtualised host of Section 6.4.4 it passed one above the median. Both panels stop at p99.9, because both are built from per-run percentiles aggregated across repeats and the reported p99.99 is pooled instead, for the reason Section 4.4 gives. A pooled figure has no across-repeat interval, so it has no place in panel (b)'s comparison, and the far-tail ordering of Table 6.1 is licensed by the millions of operations behind it and not by this diagnostic.

The CPU column rules out the reading that would otherwise be natural, that a variant completing less work is simply doing less, whether throttled, blocked or idle. That is not what happens. Rust draws *more* total CPU than C++ here, reproducibly and under both allocators, having flipped the sign it carried at the representative load in Table 6.3. Divided through by the work that actually completed, its CPU cost per sustained operation is 1.8 times the other's. Neither variant is near the capacity of the machine either, since both sit near six of the 16 pinned cores. So the gap is not a capacity limit and not idleness. It is effort spent without completing operations, which is the signature of a synchronisation path that spins, or parks and wakes repeatedly, where the other blocks once and sleeps.

Figure 6.6 puts both halves of that reading side by side, and its grouping adds a third: the separation falls almost entirely inside the allocator groups, so the effect follows the variable the study manipulates and not the one it controls for.

Where that effort goes is narrowed by measurement. Past saturation the producers spend almost all their time in two places recorded in Table 5.1: the bounded blocking queue and the completion handle. The microbenchmark below isolates those primitives, and they do not reproduce the full-service ordering at significance. That rules them out *alone* as the whole explanation for a 1.7× gap. As in Section 6.4.1, nothing in the safety model is implicated.

The setting bounds what the finding licenses, and Section 7.6 says how far it travels. The depth of the collapse depends on one design, one request outstanding per producer and a queue bound of 1 024, measured with a harness whose producers block per request. What travels is the ordering and its cause: below saturation the language choice did not matter,

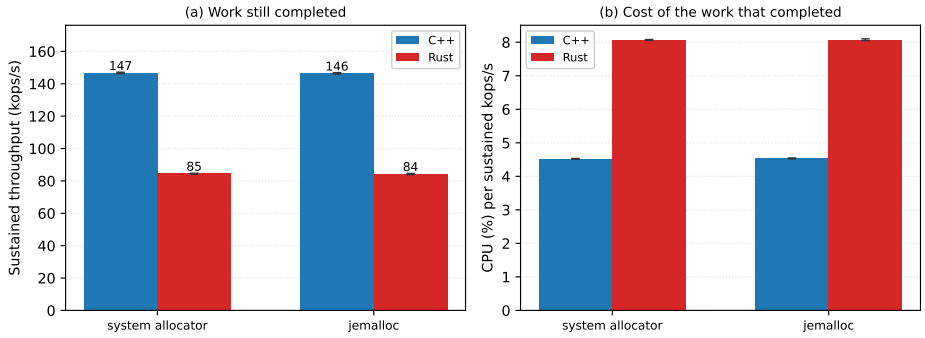


Figure 6.6: Behaviour at 784 kops/s offered, well beyond the 392 kops/s knee. Bars are medians over 20 repeats with bootstrap 95% intervals, grouped so that the two bars inside a group differ only in language. **(a)** What each variant still completes: C++ sustains a factor of 1.7 more than Rust. **(b)** Total CPU divided by the work that actually completed, which is what rules out the reading that the variant delivering less is simply doing less. It spends 1.8 times as much CPU per sustained operation, and the CPU figures are summed across threads.

and above it the standard library's synchronisation primitives did.

The closest prior work reports a comparable degradation past a threshold request rate and attributes it to the asynchronous runtime [55]. Section 7.2 takes up what it means that the same shape appeared here with no such runtime on either side.

Table 6.6: What each variant still sustains at 784 kops/s offered, well beyond the 392 kops/s knee: the median over 20 repeats above its bootstrap 95% confidence interval. Below saturation achieved throughput is pinned to the offered rate by construction and can express nothing about the languages. Here it is the only thing that can. CPU utilisation exceeds 100% because it is summed across threads, so 600% is six busy cores of the 16 pinned.

Variant	sustained throughput (kops/s)	CPU (%)
C++ (system alloc)	146.7	662.8
	[146.3, 147.2]	[662.5, 663.6]
C++ (jemalloc)	146.4	664.3
	[146.1, 146.9]	[663.3, 664.9]
Rust (system alloc)	84.5	681.8
	[84.2, 84.7]	[680.4, 682.5]
Rust (jemalloc)	84.3	680.3
	[83.9, 84.6]	[679.8, 681.5]

Table 6.7: Exact paired permutation test of C++ against Rust at 784 kops/s, the allocator held fixed, Holm-corrected across the same family of 14 tests as Table 6.3 rather than as a family of its own. "Repeats" counts how many of the 20 paired differences agree with the sign of the median, which a statistic computed from their mean cannot show. No latency percentile is tested here, for the reason fixed in advance in Section 4.4.

Metric	Allocator	Effect (Rust − C++)	Repeats	p	Holm
Sustained throughput (kops/s)	system	−62.2 [−62.9, −61.6]	20/20	0.0000019	0.000027
Sustained throughput (kops/s)	jemalloc	−62.2 [−62.7, −61.6]	20/20	0.0000019	0.000027
CPU utilisation (%)	system	+19.5 [+17.3, +19.9]	20/20	0.0000019	0.000027
CPU utilisation (%)	jemalloc	+16.8 [+15.6, +17.5]	20/20	0.0000019	0.000027

Table 6.8: Throughput through the bounded queue and completion primitives alone, with 7 contending producers and queue capacity 1 024. Values are medians over 20 interleaved repeats above bootstrap 95% confidence intervals. The paired test compares C++ against Rust on sustained throughput (kops/s). Like Table 6.5 it predates the move to bare metal and carries no environment record, so the comparison between the two alternately measured arms is what this section uses.

Language	Throughput (kops/s)	Mean ns/op	Paired p
C++	55.0	18178	—
	[53.7, 58.4]	[17110, 18632]	
Rust	58.7	17045	0.226
	[56.8, 61.8]	[16169, 17594]	

The contended queue and completion primitives on their own. The overload result attributes its gap to the two forced-idiom synchronisation constructs, but that attribution was argued from the shape of the full-service measurement. A microbenchmark isolates those constructs alone.

7 producer threads hammer one bounded queue of capacity 1 024 while a single consumer drains it. Each producer submits a no-op job carrying a one-shot completion handle and blocks until the consumer signals. No `ShardCore` work runs in the path. C++ uses the hand-written `BoundedQueue` of Section 5.4 with `std::promise`/`std::future`. Rust uses `sync_channel` and a fresh `mpsc::channel()` per request. Each timed window lasts two seconds after a warmup. 20 interleaved repeats are summarised as medians with bootstrap intervals and tested with the exact paired permutation procedure of Section 4.4. Table 6.8 reports the result.

The isolated primitives do not reproduce the overload ordering. C++ sustains 55 kops/s and Rust 59 kops/s, a gap of only 1.1× with $p = 0.226$, where the full service at 784 kops/s separates by 1.7× with its corrected p at the family floor. So the overload divergence involves extra work the microbenchmark strips away: the larger tagged `Request` on the C++ side, the worker's variant dispatch, and the map operations that still execute in the overload regime. An orders-of-magnitude difference in the primitives alone is ruled out, which moves the search for the mechanism to the rest of the handoff path and not to the safety model.

6.4.3 Resource use (RQ3)

Table 6.9 reports resource use at the same sustained load. Peak resident set size separates the two *languages* cleanly, and it does so whichever way the family is read: its paired intervals exclude zero, its effect keeps its sign under both allocators, and both rows clear the correction. It does not separate all four variants, because the C++ build with `jemalloc` and the Rust build with the system allocator land on the same footprint from opposite directions, which is the arithmetic of two comparable effects and is the subject of the next paragraph. Figure 6.7 plots the same measurement, and its middle panel is where the finding of this section sits.

Two effects are present in it, not one, and they are of comparable size. Linking `jemalloc` costs 1.5–2.4 MiB in either language. The Rust build costs 1.4–2.5 MiB more than the C++ build under either allocator, with a bootstrap interval of 1.2 to 2.6 MiB. Resource use follows both, and neither dominates the other. Part of the allocator figure comes

from the fairness controls rather than from `jemalloc` in general: both builds pin the arena count identically, so that neither language's default could become an accidental advantage, and pinning it high makes the arena metadata a fixed cost on both sides. A deployment tuning `jemalloc` for footprint instead of for comparability would not pay it.

The relative size of these two effects is the one result in this chapter that a change of host overturned. On the virtualised machine this study used first, the allocator effect measured 32.5–34.3 MiB, about 13 times the language effect rather than comparable to it, and the language effect was the same size there as here. Section 6.4.4 reports the comparison and what can and cannot be concluded from it. The lesson for reading this paragraph is narrow and worth stating: the *language* effect on peak RSS is reproducible across three machines, and the *allocator* effect is not a constant of the allocators.

Every row in the upper block of Table 6.3 now clears the correction, including peak RSS under `jemalloc`, which did not on the previous host. That row failed there for a reason in the statistic rather than in the software: a handful of runs reported a resident set some 31 MiB low, and a permutation statistic built on a mean of paired differences is dominated by them. The same measurement on this host spreads by 1 MiB, so the contamination was the machine's. The rule fixed in advance in Section 4.4 for reading such a row against its interval was not needed here, and it stays stated because a design that only declares its reading rules when they rescue a result is not applying them.

CPU utilisation behaves differently again, and by regime rather than by noise. At this below-saturation operating point Rust draws slightly *less* CPU, by at most 5.1 percentage points, consistently under both allocators and clearing the correction under both. Section 6.4.2 finds the sign flipped past saturation and the magnitude much larger. Achieved throughput adds nothing here, for the reason Section 4.4 gives. At a below-saturation point it is pinned to the offered load, and it is reported only to confirm that the point is sustained and not merely nominal, which it is to within 0.0%.

The finding is a difference in fixed cost and not in cost per context. At the 5 000 active UEs used in the sweep, the 8-byte layout difference measured in Section 5.4.1 accounts for roughly 40 kB, two orders of magnitude short of the gap, so most of it is runtime and allocator-arena overhead, which does not scale with the workload and would matter proportionally less in a larger deployment. That is an argument from elimination and this campaign cannot do better, because it reads the resident set only through `getrusage`. Section 6.4.4 reads the allocator's own accounting on a host that also exposes hardware counters, and decomposes the same gap directly.

The RQ3 finding is therefore neither the flat "no resource penalty" that overlapping intervals would suggest nor a general one. Rust costs a few megabytes more of fixed footprint under both allocators, comparable to what the allocator choice costs here though not on every host measured (Section 6.4.4), and its CPU cost changes sign with the load regime.

Table 6.9: Resource use at a sustained offered load of 56 kops/s: achieved throughput, peak RSS, and total CPU utilisation. The median over 20 repeats above its bootstrap 95% confidence interval. CPU separates the two languages outright. Peak RSS separates them at a fixed allocator, which is the comparison the paired test makes, and not across all four variants, because the allocator effect and the language effect are of comparable size and one middle pair therefore coincides.

Variant	throughput (kops/s)	peak RSS (MiB)	CPU (%)
C++ (system alloc)	55.99	11.1	124.1
	[55.99, 55.99]	[10.9, 11.2]	[123.4, 124.4]
C++ (jemalloc)	55.99	12.6	123.4
	[55.99, 55.99]	[12.5, 12.6]	[122.8, 123.7]
Rust (system alloc)	55.99	12.5	118.5
	[55.99, 55.99]	[12.4, 12.7]	[117.8, 118.9]
Rust (jemalloc)	55.99	15.0	118.3
	[55.98, 55.99]	[15.0, 15.1]	[117.6, 119.5]

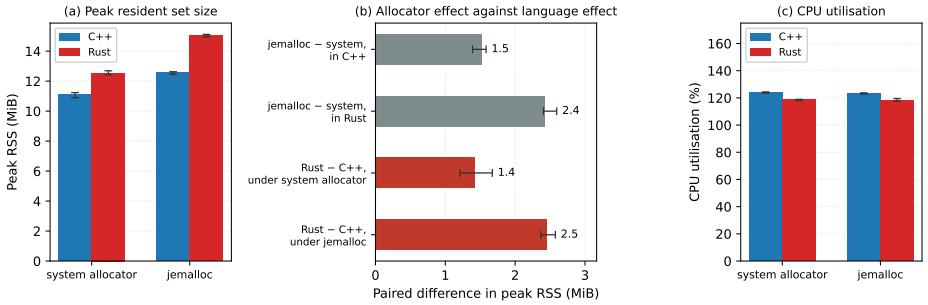


Figure 6.7: Resource use at a sustained offered load of 56 kops/s, grouped by allocator so that the two bars in a group differ only in language and the two groups differ only in allocator. Medians over 20 repeats with bootstrap 95% intervals. **(a)** Peak resident set size. **(b)** The two effects it contains, each as a paired difference over the same repeats: linking `jemalloc` costs 1.5–2.4 MiB, while the Rust build costs 1.4–2.5 MiB more than the C++ one. Both are real and on a shared axis they are of comparable size, which was not true on the host this study used first (Section 6.4.4). **(c)** CPU utilisation, summed across threads, where the difference runs the other way and is small.

6.4.4 What the machine decided, and what the code decided

Every number above was measured on the bare-metal host of Section 6.1. For most of this study they were measured on a virtualised one instead, at the same shard count, the same fourteen load levels, the same 20 repeats and the same toolchain. Two campaigns that differ in the machine alone are an opportunity a single-host study does not get, and this section takes it. The question is not which host is faster. It is which of this chapter's findings are properties of the code and which were properties of the machine, because a comparative study that cannot tell those apart is reporting its testbed.

The instrument, which is where the difference is largest. On the virtualised host the 95% interval on one variant's own p99 spanned 1480 μs while the four variants' p99 values differed by 293 μs. Uncertainty about one variant exceeded the gap between all of them by a factor of five, so no ordering could be read at that percentile however many repeats were run. Computed identically here, the interval is 1 μs against a between-variant range of 2 μs. The relation inverts. That single change is what moves the far tail from an open question to a reported result, and it is worth being precise about its cause.

At the same representative load the median barely moved, from 80 to 70 μs, while p99 fell from 4518 μs to under a hundred. A hypervisor does not slow the common path. It occasionally stops it, and a percentile computed from exactly the stopped operations is the one that records it.

What survived the change of host. The overload divergence did, and it is the strongest claim in the chapter for that reason. The ratio of sustained throughput past saturation was 1.8× on the guest and is 1.7× here, on another vendor's silicon, with the ordering unchanged. The saturation knee moved as capacity should, from 336 to 392 kops/s. The peak-RSS *language* effect survived too, at 1.8–3.5 MiB there against 1.4–2.5 MiB here, which is the same effect within measurement error on two machines, two vendors and two memory subsystems.

What did not survive, and this is the one to read carefully. The peak-RSS *allocator* effect. On the guest, linking `jemalloc` cost 32.5–34.3 MiB, roughly 13 times the language effect, and while that host was the only evidence this thesis reported the gap as an order of magnitude. Here the same comparison costs 1.5–2.4 MiB, which is the same size as the language effect rather than ten times it. The claim that the allocator dominates the footprint was a property of that machine.

The mechanism is not established, and the honest form of the finding is the comparison rather than an explanation of it. One host property is a candidate and is recorded in both captures: transparent hugepages were set to always on the guest and madvise here, and the campaign pins `thp:never` for `jemalloc` while the system-allocator arm inherits whatever the kernel does, so the two arms were not treated alike by a policy neither of them chose. This study did not vary that policy with everything else held fixed, so it cannot attribute the difference to it. What it can say is that a second bare-metal host at a different shard count measured 1.9–3.2 MiB, which agrees with the figure here and not with the guest's, so the guest is the outlier of three rather than one reading of two.

A related artefact went with it. The run-to-run spread that cost peak RSS under `jemalloc` its place in the significance family, a handful of runs reporting a resident set 31 MiB low, is 1 MiB here, its paired repeats agree on the sign 20/20 times, and p50 latency, p99 latency, p99.9 latency, Peak RSS and CPU utilisation keep their sign across the allocator control.

A second bare-metal host, at a different design. One AMD EPYC 7443P, 24 cores on 1 socket and 1 NUMA node, running Ubuntu 24.04.4 LTS with `g++ 13.3.0` and `rustc 1.98`. The governor is fixed at performance, simultaneous multithreading is off, and cores 2–17 carry the benchmark. The environment capture records virtualisation as none. Across 61 s of wall clock, 0 of 60 sampling intervals on the 16 pinned cores showed any CPU steal. The sweep is 480 runs: 15 repeats over 8 offered loads from 8 to 128 kops/s, four variants interleaved inside each repeat as before.

The far tail, measured twice. A percentile this far out is the one most likely to be an artefact of a single campaign's design, so it is worth having two. Pooling this host's repeats gives 360 000 to 5 761 198 operations per group, which is what makes p99.99 a measurement rather than a handful of draws. Between 32 and 96 kops/s the pooled p99.99 of the two C++ variants falls between 147 and 228 μs and that of the two Rust variants between 88 and 115 μs, and the two ranges do not meet. Below 32 kops/s the four are indistinguishable. The direction is the one the main campaign reports, at a different shard count and a different load grid, which is the strongest statement two campaigns can make jointly. The highest swept load is held out of both statements, because its pooled median falls while every percentile above it stays flat and this campaign does not explain that shape.

What survived the change of design. The median difference clears Holm correction at 14 of 16 load-and-allocator points, with Rust lower by 0.5–1.1 μs on an operation of 74 μs. Peak RSS is significant at 16 of 16 points, with the Rust build costing 0.7–1.8 MiB more than the C++ one and `jemalloc` costing 1.9–3.2 MiB more than the system allocator. Those two figures matter beyond this section, and the comparison between them needs stating carefully. Here the allocator effect is the larger of the two and the ranges do not overlap, where on the main campaign the two interleave. What both bare-metal hosts agree on is the order of magnitude, and what they jointly contradict is the virtualised host, where the allocator cost some ten times the language. So the two effects are of the same size class on adequate hardware and their ordering within that class is not established, which is the third of the three measurements Section 6.4.4 weighs against each other. The CPU comparison below saturation is significant at 16 of 16 points with Rust lower at every one of them, by 0.3–7.6 percentage points, the gap widening with offered load. The divergence past saturation reproduces here too. Every arm tracks its offered load to 448 kops/s and the knee falls between 448 and 512 kops/s. At 640 kops/s, where all four arms have collapsed and each one's repeats agree to within 0.4%, C++ sustains 158 kops/s against 96, a ratio of 1.64 against the 1.7 of Section 6.4.2. Three hosts and two shard counts now put that ratio between 1.64 and 1.8, always with the same arm ahead. One load must not be quoted. At 512 kops/s, the first load past the knee, the Rust arms are bimodal across repeats and their throughput varies by 37%, serving the full offered rate in some repeats and collapsing in others. A median there would average a distribution with a mode at each end.

A mechanism, from hardware counters. Section 6.4.1 attributes the operation-level difference to the forced idioms of Table 5.1 on the strength of an argument about container layout. Counters turn that argument into a measurement, and no virtualised host in this study exposed them. Over 10 interleaved repeats at the operation scope, the Rust build retires 11% fewer instructions than the C++ one ($p = 0.002$) and finishes in 15% fewer cycles ($p = 0.0117$), while missing cache 29% more often per thousand instructions ($p = 0.002$). Less work in fewer cycles is a mechanism for the operation-level result, and it replaces an argument from structure with a measurement. It says nothing about the resident set, and the worse locality is not evidence that it does: a larger footprint is a

reason to miss cache more often, so the two are related in the opposite direction. The footprint has its own instrument, one paragraph down.

The resident set, decomposed by the allocator that holds most of it. Section 6.4.3 establishes that the peak-RSS difference is a fixed cost by elimination, since the measured struct layout accounts for orders of magnitude too little. Both benches also read `jemalloc`'s own accounting, and every one of the 240 runs on the two `jemalloc` arms carries it, so the composition can be read instead of inferred. Table 6.10 gives it. Three readings, in the order they change the claim.

The bookkeeping is where the allocator's own difference sits, and it is fixed: arena metadata costs the Rust build +2.07 MiB more, and that figure moves by 0.5% across the whole swept range, so it is a constant of the process and not a function of the workload. Rust nonetheless holds *less* live allocation, by −0.16 MiB, which settles one thing outright: the larger resident set is not more application data.

The second reading is the one that relocates the finding. The allocator's own resident bytes are −1.92 MiB *lower* in the Rust build, while its peak resident set is +1.74 MiB higher. Subtracting the two inside each run, the footprint outside the allocator's mappings is 4.0 MiB for the C++ build against 7.9 for the Rust one, a difference of +3.8 MiB. So the language effect on peak RSS is not an allocator effect at all. It is the standard library and runtime a Rust process carries before it allocates anything, and the allocator comparison runs the other way and partly offsets it. One caveat travels with that subtraction: peak RSS is a maximum over the run and the `jemalloc` figures are sampled once, so the outside-mappings figure is an upper bound rather than an exact quantity.

The third reading is a negative result about the guest. The measurement defect that cost peak RSS under `jemalloc` its place in the virtualised campaign's significance family, a handful of runs reporting a resident set 31 MiB low, does not appear here: no arm's peak RSS spans more than 1.7 MiB across its repeats. That bimodality belonged to the host, which is what the reading rule of Section 4.4 assumed without being able to show it.

Table 6.10: Where the peak resident set goes on the second bare-metal host, from `jemalloc`'s own accounting in each of the 240 runs of the two `jemalloc` arms. Medians in MiB over the steady-state loads, with the paired language difference in the last column. The two rows below the rule are the argument: the allocator holds *less* in the Rust build, and the process holds more, so the difference is outside the allocator. The final row is the quantity the other rows decompose. The derived row is the median of the per-run difference and not the difference of two medians, because the two figures it subtracts come from one process at one moment, so the column does not sum exactly.

Component	C++	Rust	Difference
Live allocation (`allocated`)	0.78	0.62	−0.16
In active pages (`active`)	1.43	0.79	−0.64
Arena bookkeeping (`metadata`)	4.19	6.26	+2.07
Resident in allocator mappings (`resident`)	11.02	9.10	−1.92
Outside those mappings (derived)	4.04	7.86	+3.82
Peak resident set (`getrusage`)	15.08	16.82	+1.74

The allocation profile, and what it cannot attribute. The visit also captured a matched pair of `jemalloc` heap profiles, taken to name the call sites behind the resident-set difference the readings above locate. They do not name them, and the reason belongs in the record because it is a property of the instrument rather than of either language. Peak RSS in the profiled run is 4.9 MiB higher in the C++ build, where the campaign's own `jemalloc` arms put Rust higher by 1.4–3.6 MiB at every load it swept. No effect can be attributed in a configuration that reverses its sign.

Two departures from the campaign could explain that reversal, and a second profile separates them. The first was the operating point: the profile above ran closed loop at 455 kops/s, past the plateau bracketed at 448–512 kops/s rather than below it. Repeating it at a swept load below saturation shrank the discrepancy by most of its size and did not remove it, so the wrong regime was the larger part of the effect and not its cause. What remains is the second departure, and it is the one that cannot be engineered away: profiling requires an allocator built to sample every allocation and record a backtrace for it, so the arm under profile is not the arm the campaign measured. An instrument that perturbs the quantity it reports by more than the difference under test cannot attribute that difference, whatever load it runs at. The dumps are also harder to read than the method anticipated. Each symbolised profile puts almost all of its flat bytes on a frame that allocates nothing, 98.4% on a deallocation symbol in the C++ arm and 100.0% on the allocator's own backtrace routine in the Rust one, so only the cumulative call paths remain and those name subsystems rather than sites.

Two results survive those limits. Both arms allocate 2.81 MiB for the latency histogram, equal to the byte, which is the harness parity control of Section 4.3 working where it can be checked directly. At the same point in each arm's allocation history, the first dump of the series, the worker path holds 7.0 MiB in the Rust build against 2.2 MiB in the C++ one. That is the shape a table reserved in one block gives against a map grown a node at a time, so the container difference Section 6.4.1 argues from the forced-idiom log is visible in allocation as well as in latency. Attributing the footprint itself would need a profile taken at a swept load, on a build whose peak RSS reproduces the campaign's ordering, and that is work for a repeat visit rather than a reanalysis of these dumps.

Inter-operation jitter. Every run recorded inter-operation delay variation, between 23 992 and 383 992 samples of it, and no research question asks for the metric, so it is described here and not tested. Below the top load all four variants sit between 1.50 and 1.94 μs at the median and between 6.7 and 10.8 μs at p99. The widest gap between the four at any single load is 0.36 μs. At 128 kops/s they move together to 45.6–46.5 μs, at least 24 times any value below it. Jitter on this service separates the load regimes rather than the languages.

Why three datasets rather than one, and what each is for. The three campaigns are not interchangeable and the differences between them are what makes each one useful. The reported sweep and the virtualised one share a design exactly and differ in the machine, which is what allows the first half of this section to attribute a change to the hardware. This second bare-metal campaign differs in the design as well: 8 shards and

8 producers against the 7 of Table D.2, 8 loads against 14, 15 repeats against 20, and `g++ 13.3.0` with `rustc 1.98` against the versions Table D.1 records. Shard count is one of the fairness controls of Section 4.3 and the repeat count sets the floor the exact test can reach, so this campaign cannot stand in for the reported one. What it can do is ask whether the findings depend on those choices, and the answer is that the direction of every one of them survives a change of shard count, load grid, repeat count and compiler at once. The compiler half is the only evidence in this study that separates either language from one implementation of it (Section 7.6).

Two caveats belong with this campaign's numbers, and Table D.1 carries the first row by row. It met the protocol on virtualisation, steal, governor and multithreading, and not on tickless operation: `nohz_full` is none here, where the reported sweep runs tickless on its pinned cores. The second is that its resource figures were recorded with the jitter sidecar enabled, which costs both languages the same and lifts both absolute footprints, so paired differences travel between campaigns and absolute peak RSS does not.

6.4.5 Safety via fault injection (RQ2)

This study does not claim that Rust has no bugs. It injects the catalogue of 14 defects set out in Table 3.1, each as a small self-contained program flavoured with the subsystem's domain so that it maps onto a plausible bug in the service. The split into two groups is what the method of Section 4.2.2 turns on: seven classes from inside the region Rust claims to govern, and seven chosen precisely because safe Rust is expected *not* to prevent them. With the second group the study measures a boundary instead of demonstrating a claim, and a case that unexpectedly compiled, or unexpectedly failed to, surfaces against its recorded expectation instead of being absorbed into the conclusion.

Procedure. Each C++ defect is built and run under AddressSanitizer (ASan), ThreadSanitizer (TSan) and UndefinedBehaviorSanitizer (UBSan), and the matching construct is written in safe Rust and compiled. The third sanitiser is not a refinement of the other two but a separate axis. Signed overflow is invisible to both ASan and TSan, so without its own column the integer-overflow class would be recorded as undetected in C++ when in fact it is caught. No static analyser is run, for the reason set out in Section 4.2.2, so the C++ columns of Table 6.11 report the tooling this study ran and not a ceiling on what C++ tooling could achieve. Where the Rust program compiles, it is run under a bounded timeout and its observable behaviour recorded, because "it compiles" alone would not establish whether the defect survives into the running program.

Results. The outcomes are shown in Table 6.11, measured with `g++ 13.3` (`-std=c++23`, `-O0` so the optimizer cannot elide an injected defect) and `rustc 1.97` (edition 2021). That C++ toolchain is the local development machine's and not the measurement host's `g++ 15.1.1`, and the catalogue was run there deliberately. Whether a program compiles, and what a sanitiser reports when it runs, are properties of the toolchain and not of the machine hosting it. That is the concrete content of calling this result environment-insensitive in Section 6.1.

Within its scope the result is unambiguous. Every one of the seven memory-safety and data-race defects is a compile error in safe Rust, so none can be written at all. In C++ each is valid code whose defect shows up only at runtime, and only if a test actually exercises the buggy path. Each sanitiser covers a different part of the catalogue and no single one covers it, so the three between them still leave 4 of the 14 classes untouched, with a fifth reported only intermittently. Runtime detection also means building a dedicated instrumented binary and paying the sanitiser's time and memory overhead, while the Rust errors are reported on every ordinary build whatever the test coverage. The parenthesised codes are the `rustc` diagnostics that fire: conflicting mutable borrows (`E0499`), a mutable borrow while a shared borrow is live (`E0502`), use of a moved value (`E0382`), and a closure outliving borrowed data as it escapes to another thread (`E0373`).

Two rows in that group carry more than the other five. The use-after-move is the only in-guarantee class no C++ sanitiser reports at all: for the reason Section 3.2 gives, all three tools run clean while the acknowledgement goes out under an empty identity. Every other class here is caught by at least one of them, so on this row the instrumented build pays its overhead and reports nothing. The thread-unsafe handle's diagnostic is `E0277`, a trait bound rather than a borrow error, because the rejection comes from `Send` and not from the borrow checker. Where C++ has only a comment to state the thread-affinity rule, in Rust it is part of the type and `thread::spawn` asks for it in its signature, so the compiler refuses without analysing the body at all. One caveat belongs with the use-after-move. A static analyser configured for it would plausibly flag the pattern, and `clang-tidy` ships such a check, whose own documentation records that it follows control flow but not paths and considers only local variables and function parameters [66]. None is run here, for the reason given in Section 4.2.2, so the row records what the sanitisers report and not the limit of what C++ tooling could reach.

All seven boundary cases behave as predicted, and they mark out the edge of the claim. Every one compiles in safe Rust and every one misbehaves. The lost-update case ends in both languages with a silently wrong answer: against an expected total of 40 000 increments the safe-Rust program recorded 25 734 and the C++ program under ThreadSanitizer 27 767, losing 14 266 and 12 233. Neither language is meaningfully better off, which is the point of carrying the class at all. The deadlock hangs in both, the reference cycle leaks in both, the release-build overflow produces a wrong TEID in both, and a failure inside one shard worker takes down more than that shard in both.

The forgotten guard separates two things the phrase "RAII prevents this" runs together. Rust removes the ordinary form of the bug, since the C++ version fails on a validation path that returns before a hand-written unlock and the Rust version has no unlock to skip. It does not remove the class, for the reason Section 3.2 gives. Both programs had to be killed by the timeout and neither language's tooling said anything, because a lock that is never released is not a race and not a cycle, so there is nothing for a lock-order detector to close.

The seam double-free does not report a new defect class at all, which is why it is in the catalogue: it is the double-free of the in-guarantee group, written the way a foreign-function boundary has to write it. Safe Rust rejects the double release with `E0382`, and handing a C caller a raw pointer is precisely the operation that ends the ownership

tracking behind it. `Box::from_raw` then asserts ownership of whatever address it is given, so asserting it twice frees the storage twice and the program aborts on the allocator's own check. The C++ half of the pair is deliberately unremarkable and unchanged, so the only column that moves is the Rust one. The guarantee is therefore not a property of the language considered alone but of the language together with the boundary the component is deployed behind, and that boundary is the one Section 6.4.6 measures.

Two of the seven are cases where the C++ tooling is *ahead*, which is the outcome the two-sided catalogue was built to be able to produce. LeakSanitizer, which ships inside ASan, reports the C++ reference cycle as an unreachable allocation at exit. The safe-Rust cycle leaks in silence, because leaking is safe in Rust by explicit design and nothing in the default toolchain looks for it. That this belongs to the language rather than to the injected example shows in the work being done to change it. Proposals to add garbage collection to Rust are motivated precisely by shared-ownership structures that reference counting expresses "awkwardly, inefficiently, or both" [67], and where `unsafe` marks the memory-safety boundary explicitly the leak boundary is implicit, which is the premise of a static leak checker that found the class in 19 of more than 1,200 published crates [68]. UBSan reports the C++ overflow, because signed overflow is undefined there, while the Rust overflow wraps by definition at `opt-level = 2` and passes unremarked (Section 3.2). That second contrast does not reduce to which language is safer. Defined-but-wrong is easier to debug than undefined-and-wrong and no less incorrect, and neither is what "memory safe" is usually taken to promise. Deliberate wraparound is also common enough in C and C++ that a checker cannot treat every overflow as a defect, and undefined overflow has broken working programs as compilers improved [69].

The deadlock result needs a qualification. ThreadSanitizer does flag the inversion in C++, but on some runs and not others: 5 of 12 repeats in the run behind this table, while the program deadlocks on every one. The rate itself is not stable across runs and should not be read as one, since repeated campaigns on this host have returned figures on both sides of half. What reproduces is the intermittency, and its mechanism generalises beyond this catalogue. TSan builds its lock-order graph from acquisitions that *succeed*, so the cycle closes only when some thread completes an inversion without blocking. A run in which the first inversion blocks produces one edge and never the second, and the detector stays silent at exactly the moment the bug bites. A positive control that inverts the same two locks without ever blocking is reported every time, which establishes that the detector was live and not misconfigured. So the tool catches the inversion the program survives and is unreliable on the one that deadlocks, which is a very different proposition for a continuous-integration pipeline than a clean catch.

The scope all of this exposes is precise. Rust's concurrency guarantee is the absence of *data races*, meaning unsynchronised concurrent access to the same location. It is not a guarantee about *lock-acquisition order*, nor about the *atomicity of a composed operation*, nor about leaks, wrapping arithmetic, or fault containment, and it is suspended wherever a program writes `unsafe`. The lost update stays the quietest case, because no sanitiser reports it either. With every access properly locked there is no race for a race detector to find.

Where the boundary classes are caught, and at what cost. RQ2 asks which defects are prevented, and also at what stage the rest are detected. For the boundary group the sanitiser columns leave that second half largely blank. Two Rust model checkers answer it, in the division of labour a production Rust system has been validated with: Loom explores interleavings soundly under bounded partial-order reduction, and Shuttle samples them probabilistically to reach cases beyond Loom [70]. Both report both of the original boundary defects, the deadlock as a closed cycle and the lost update as a failed assertion. Each is paired with a control in which the defect is fixed, by consistent lock order and by holding the read-modify-write in one critical section, and both controls come out clean, so the reports come from the defects and not from the tools.

The cost is the part worth recording. Neither defect program runs under either checker as written. Each had to be re-imported against the checker's own synchronisation and thread types, because a tool that does not intercept the primitive cannot schedule it, and each had to be shrunk to model size: the lost-update model drops from twenty thousand iterations per thread to one, and the deadlock model loses the sleep that widens its window on real hardware. So the answer for the Rust half of the boundary is *detected at test time, after a rewrite*, while on the C++ half the sanitiser takes the program unchanged and finds one of the two unreliably and the other not at all. Neither language's story here is the simple one, and Chapter 7 develops the consequence for practice.

The seam double-free is answered by a different tool and comes with a sharper limit. Miri interprets the program against the Stacked Borrows aliasing model and reports the second release precisely, as a dangling box reconstructed from a freed allocation, while a control that releases the same handle once is clean. The stage for this class is therefore test time again, and this time without a rewrite. The limit is what a migration has to weigh. Miri interprets Rust and cannot execute a C++ host, so the program it checked uses a Rust stand-in for the host. The tool that finds this defect cannot be pointed at the configuration the defect ships in, which is a seam with C++ on the other side of it. Nothing in the toolchain checks that boundary, and the practical answer is the one Section 6.4.6 argues from the prototype: keep the seam small enough to review by hand, because review is what is left.

A defect that was not injected. Every outcome above comes from a defect written on purpose, and that is the standing weakness of the method. Whoever assembles a catalogue already knows what safe Rust prevents. One case in this study does not carry that weakness. A design review of the specification found that its concurrency section forbade submitting an operation outside the service's running window (before `start()`, or after `shutdown()`), while neither implementation checked for it. The prohibition was addressed to the caller with no mechanism behind it, and in that gap the two languages did not fail alike. The C++ submit path indexes the shard vector that `shutdown()` has cleared, which AddressSanitizer reports as a heap-use-after-free. Called before `start()` it is a segmentation fault on the zero page. In Rust both calls were already a panic, from the ordinary bounds check on the same indexing operation, and nobody had designed for it.

What makes the case evidentially different is where it came from, not how severe it is. It

Table 6.11: Defect-class taxonomy: measured outcomes for the injected catalogue. C++ under `g++ 13.3` with ASan/TSan/UBSan. Rust under `rustc 1.97`, edition 2021. "Compile error" means the safe-Rust program does not build, so the defect cannot be written. The lower block is the boundary group: defect classes chosen because safe Rust is *not* expected to prevent them, and does not. Two of those seven are the ones on which the C++ tooling is ahead. The body of this table is generated from the fault-injection taxonomy log (Section 4.5), so it cannot disagree with the run that produced it.

Defect class	ASan	TSan	UBSan	Safe-Rust outcome
Within the guarantee				
Use-after-free	detected	detected	not detected	compile error (E0499)
Double-free	detected	not detected	not detected	compile error (E0382)
Iterator invalidation	detected	detected	not detected	compile error (E0502)
Data race	not detected	detected	not detected	compile error (E0373†)
Unguarded shared mutation	not detected	detected	not detected	compile error (E0502†)
Use-after-move	not detected	not detected	not detected	compile error (E0382)
Thread-unsafe handle shared	not detected	detected	not detected	compile error (E0277)
Boundary of the guarantee				
Lock-order deadlock‡	not detected	intermittent§	not detected	compiles, deadlocks
Lost update	not detected	not detected	not detected	compiles, wrong result
Reference-cycle leak	detected	not detected	not detected	compiles, leaks
Integer overflow (release)	not detected	not detected	detected	compiles, wrong result
Worker-failure blast radius	not detected	not detected	not detected	compiles, shard outage
Double-free across the seam	detected	not detected	not detected	compiles, aborts
Forgotten lock guard‡	not detected	not detected	not detected	compiles, deadlocks

† Several diagnostics fire for these cases. The first is shown (the data race also raises `E0382` and `E0502`, the unguarded mutation `E0524` and `E0597`). ‡ On these rows the instrumented binary was killed by the harness timeout rather than reporting anything. Nothing is corrupted and nothing is undefined, so the tools that detect corruption have nothing to say. The forgotten guard is silent under all three, and the lock-order case is the one TSan sometimes catches, as the next note records. § ThreadSanitizer reports the lock-order inversion on some runs and not others, measured at 5 of 12 repeats, while the program deadlocks on every one. The detector builds its lock-order graph from acquisitions that *succeed*, so a run whose first inversion blocks never completes the second edge of the cycle.

sat in code written to be correct, in the arm whose safety claim rests on ThreadSanitizer, reachable through the public API by calling documented operations in a documented order. It also survived every check this study runs: 6 C++ test binaries, 27 Rust tests, 223 hand-written and 793 000 generated conformance operations, and three sanitiser configurations reported clean. It survived them because none of them ever called an operation outside the running window. ASan finds this defect only once a test drives it, which is the property the cost argument above assigns to runtime detection, seen here on the study's own artefact instead of a planted one.

The direction in which it must *not* be over-read is the flattering one. Rust did not prevent this at compile time, and nothing in the borrow checker knows that a service must be started before it is used. What turned undefined behaviour into a diagnosable failure was a slice bounds check, which is a runtime guarantee and so the same kind of assurance the C++ sanitiser offers, and not the compile-time kind Table 6.11 records. The honest claim is that checking by default turned a specification gap into a panic, not that a type system caught a lifecycle error. What remains is a panic naming its cause on every build against a heap-use-after-free that needs an instrumented binary and a test that thinks to look. Both implementations now reject the call explicitly, in both directions, and both suites cover it.

Figure 6.8 draws the same 14 outcomes as a matrix, where the change of character at the

boundary rule is the shape of the picture and not something to be read off row by row. Both it and Table 6.11 are generated from the fault-injection taxonomy log of Section 4.5, so neither can drift from the other.

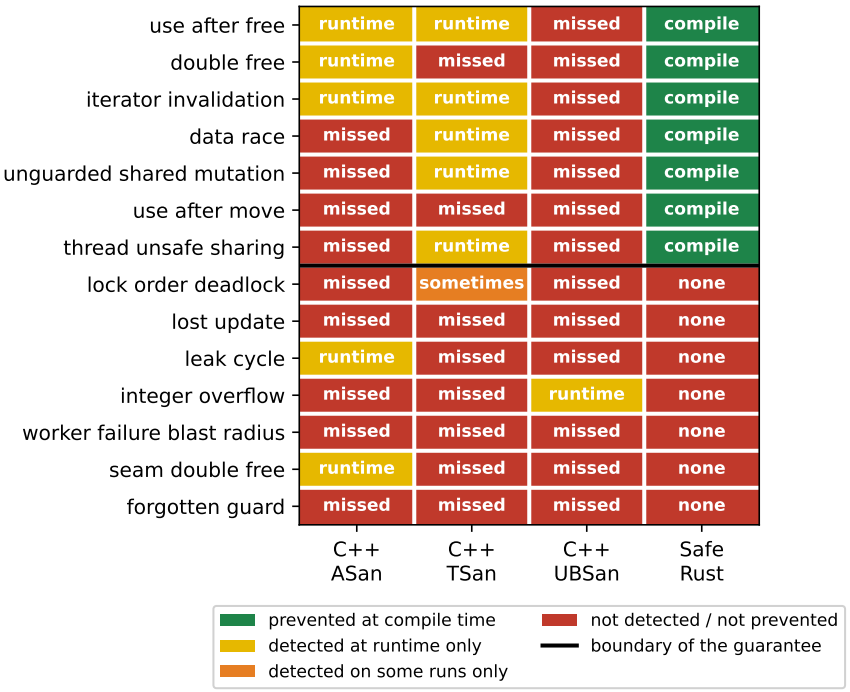


Figure 6.8: The outcomes of Table 6.11 drawn as a matrix, generated from the same taxonomy log (Section 4.5). Above the rule the Rust column is a single colour: every defect is a compile error, so none can be written. The C++ columns there are a patchwork instead, and no one sanitiser covers them. The use-after-move row is the one no sanitiser covers at all. Below the rule the Rust column joins them, and on the reference-cycle leak and the release-build overflow the C++ tooling is ahead. The lock-order deadlock is the one cell where a tool reports intermittently: ThreadSanitizer flags the inversion in 5 of 12 runs while the program deadlocks on every one.

6.4.6 Migration (RQ4)

Incremental adoption is assessed with a working prototype, built in the direction a real migration takes: an existing C++ host keeps a Rust component behind a C ABI. The Rust service is exposed through a thin `extern "C"` seam in a *separate* crate that depends on the unmodified core library, so adopting the component needs no change to the component itself. The C++ host sees only an opaque handle and a set of plain-C entry points (a hand-written header, no generated bindings), and it drives the full UE lifecycle across the boundary. The error model crosses intact, so a duplicate connection setup returns `InvalidState` through the seam. Figure 6.9 shows the resulting structure and where the `unsafe` sits.

Table 6.12 summarises the interop cost. The seam is small: 11 C-ABI entry points in 231 lines of Rust, plus a 66-line C header, with no third-party FFI dependency. The safety boundary is clean, since the core library stays free of `unsafe` and the raw-pointer handling the C ABI requires is confined to the seam crate, in its 9 `unsafe` functions, under a single documented safety contract. The C++ side never sees a Rust type.

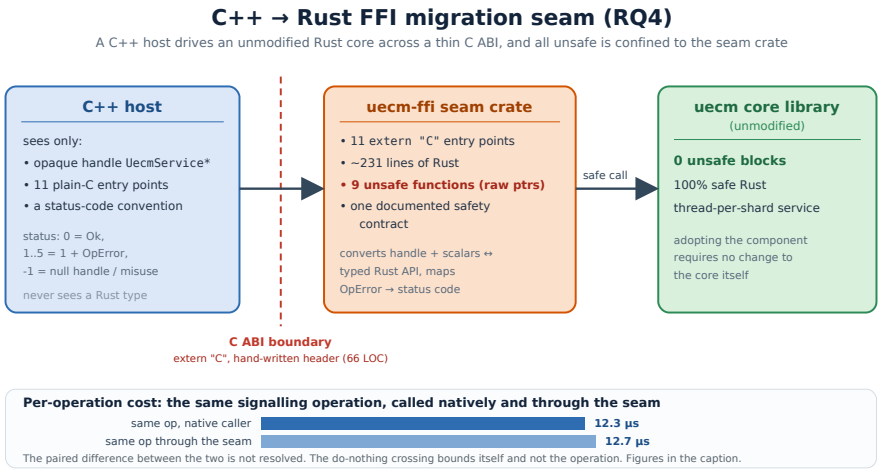


Figure 6.9: The C++-to-Rust FFI migration seam. The C++ host sees only an opaque handle, 11 plain-C entry points, and a status-code convention. A thin `extern "C"` seam crate concentrates all of the raw-pointer `unsafe` (9 functions) and calls the unmodified Rust core, which remains free of `unsafe`. The band along the bottom is the comparison the argument rests on: the same signalling operation called natively at 12.3 µs and through the seam at 12.7 µs, whose paired difference is not resolved and is bounded at 10.91% of one operation. The two bars are meant to look alike. The do-nothing crossing costs 2.54 ns and bounds only itself.

Table 6.12: FFI migration prototype: seam size, safety boundary, and per-call interop cost (C++ host calling the Rust component across a C ABI, `-O2`). The core library is unmodified and `unsafe`-free. All interop is confined to the seam crate. The last three rows are the direct comparison: one signalling operation on the same service configuration and the same loop, invoked natively and then through the seam, so the difference isolates what the boundary adds rather than what the operation costs. Like the other paired experiments it predates the move to bare metal and carries no environment record, so the bound is a ratio between two alternately measured arms and not a claim about a host.

Metric	Value
C-ABI entry points	11
Seam size (Rust `lib.rs`)	231 LOC
Hand-written C header	66 LOC
`unsafe` functions in core library	0
`unsafe` functions in FFI seam	9
Raw FFI crossing	2.54 ns/call
Signaling op, native caller	12.3 µs/call
Signaling op, through the seam	12.7 µs/call
Seam cost per operation (paired)	+192 ns [−954, +1341]
as a fraction of the operation	+10.91% at the upper bound

The per-call cost is measured twice, and only the second measurement answers RQ4's question. The first is the crossing itself: a do-nothing `extern "C"` call costs 2.54 ns, an ordinary indirect call, negligible next to the microsecond-scale round-trip of a signalling operation. That bounds the crossing and says nothing about what the seam does to a real operation. The round-trip is dominated by a cross-thread handoff, and an argument that the handoff "would be paid identically anyway" is reasoning standing in for a measurement.

The measurement is therefore made directly. The same signalling operation, on the same

service configuration and in the same loop, is invoked natively from Rust and then from C++ through the seam, interleaved over 15 repeats so the pair is matched. The paired difference is +192 ns per operation, with a bootstrap interval of $[-954, +1341]$ ns and an exact paired p of 0.239. No cost is detected. More usefully, the interval bounds any cost the measurement could have missed at 10.91% of the operation. The interleaving is not optional here. A single pass of each, run while preparing this section, made the seam look *faster* than calling natively by a third, because the run-to-run spread of the handoff on a shared host is far wider than anything a C ABI could contribute. Figure 6.10 shows the repeats behind those numbers, and its left panel is that spread: the pairs cross, over a range an order of magnitude wider than the difference under test.

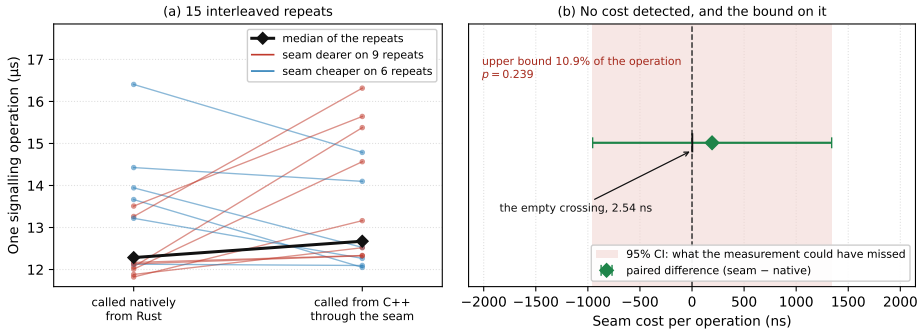


Figure 6.10: The interoperability cost measured on a real operation rather than inferred from an empty call. **(a)** The same signalling operation invoked natively from Rust and from C++ through the seam, one line per interleaved repeat, with the medians in black. **(b)** The pairs reduced to their difference: +192 ns per operation with a bootstrap 95% interval of $[-954, +1341]$ ns and an exact paired p of 0.239. No cost is detected, and the upper end of that interval is what bounds the cost the measurement could have missed at 10.91% of the operation. The empty `extern "C"` crossing, 2.54 ns, is marked for scale rather than as the answer: it is indistinguishable from zero on the axis the operation is measured on.

The main engineering effort therefore goes into *designing* a C-compatible API surface, with an opaque handle, scalar-typed operations and a status convention, and not into runtime overhead. The reverse direction (Rust calling into C++) is symmetric, but it needs a C-ABI wrapper on the C++ side and it brings `unsafe` back to the Rust caller. The prototype and its measurements inform the migration guidelines of Chapter 7.

Three cost axes the seam does not reach. RQ4 asks what incremental adoption costs in effort, interoperability and risk. The seam answers the middle term with a measurement, and it leaves the other two to its own line counts, which describe one component and say nothing about the work of building it. Three further axes are measured here, chosen because a team asks each of them before any of the questions answered so far: how long the toolchain takes, how large the artefact is, and how much code of unknown origin a configuration pulls in. Table 6.13 collects them.

Binary sizes and dependency counts are environment-insensitive in the sense Section 6.4.5 gives the defect catalogue, so they carry from this measurement unchanged. Build time is not, and it is treated accordingly. It was taken on the local development machine over 15 interleaved repeats at 14 parallel jobs, with the first repeats discarded so that a cold page cache is not read as a property of a compiler. The absolute seconds describe that

Table 6.13: Adoption-cost axes beyond the seam. Build time is the median of 15 interleaved repeats at 14 parallel jobs on the local development machine (`g++ 13.3`, `rustc 1.97`), warm-up repeats discarded. "First-party only" rebuilds the same Rust targets with the dependency tree left warm, so the difference above it is what the ecosystem costs and not what the compiler does. Footprint is the `conformance_runner` binary, the tightest matched executable pair in the study, reported at each toolchain's default linkage and then at both matched policies, because the defaults differ and the ordering depends on which policy is held. Crate counts are transitive and come from `Cargo.lock`. The `unsafe` cell is empty for C++ because the language marks no such region, so no count exists to compare.

Measure	C++	Rust
Clean build, core library only (first-party source on both sides)		
Wall clock (s)	1.9	1.2
CPU time (s)	2.6	3.9
Clean build, every default target		
Wall clock (s)	5.7	8.1
CPU time (s)	35.9	64.8
CPU time, first-party only (s)	35.9	29.6
Incremental rebuild after one edit		
Source file changed (s)	1.2	3.6
Core header changed (s)	5.8	3.6
Static footprint, `conformance_runner` (KiB)		
Shipped, each toolchain's default linkage	106	723
Matched: standard library linked statically	1577	723
Matched: standard library resolved dynamically	106	269
Shared standard library left on the host	2531	5295
Third-party code resolved		
Transitive crates, compared component	0	0
Transitive crates, matched-allocator arm	0	8
Transitive crates, benchmark harness	0	17
`unsafe` sites in that third-party code	—	1064

machine. What travels is the paired ratio between two toolchains measured alternately on one host, which is the same reason the campaign interleaves its variants.

Compile time separates the compiler from the ecosystem. On the matched scope, the core library alone with no third-party code on either side, Rust finishes sooner and C++ spends less machine time: 1.9 s against 1.2 s of wall clock, and 2.6 s against 3.9 s of CPU. The orderings disagree because they measure different things, and the wall clock is the weaker: the C++ library is two translation units, so at 14 available jobs only two can ever run, while `rustc` splits its one crate into codegen units and kept about three cores busy. Wall clock at this scope rewards this codebase's file layout as much as either toolchain. CPU time does not.

The full build is where the practical objection to Rust adoption usually lands, and at first reading it holds. Every default target costs 64.8 s of CPU against 35.9 s, a factor of 1.8. Breaking that figure apart moves it somewhere more useful. Rebuilt with the dependency tree left warm, the same Rust targets cost 29.6 s. So 54% of the clean build is third-party code, and the project's own code compiles in 0.8 times the CPU `g++` needs for its equivalent. On this codebase the compiler is not what makes the build slow. Two qualifications bound that reading: the two target sets are close and not identical, and the C++ dependencies are not absent but precompiled into the distribution as shared libraries,

which is a real benefit of a mature ecosystem and not an artefact of the accounting.

The incremental loop is quicker in C++ and less predictable. After one edit to an implementation file, C++ rebuilds in 1.2 s, which is 3.1 times quicker than Rust's 3.6 s. After one edit to a core header it takes 5.8 s. That is 1.6 times slower than Rust and, measured in CPU, indistinguishable from building the tree from scratch. The Rust figure does not move: touching every file in the crate costs 3.8 s against 3.6 s for touching one, because the unit of compilation is the crate and not the file. So the edit-compile loop is faster in C++ where an edit stays inside a translation unit and slower where it does not, and which case dominates a working day depends on a codebase's header discipline rather than on either language. The shape will be familiar from RQ1: the C++ figure is the better one at the median and the worse one in the tail.

The footprint ordering reverses with the linkage policy. Built as each toolchain ships by default, the Rust `conformance_runner` binary is 6.8 times the size of the C++ one, 723 KiB against 106. That comparison is between two linkage policies and not between two languages, because `g++` leaves `libstdc++` to the dynamic loader while `rustc` links its standard library into the binary. Measured at a matched policy the gap narrows and then changes sign. With both standard libraries resolved dynamically, C++ is smaller by 2.5 times. With both linked statically, Rust is smaller by 2.2 times.

The reversal has a measured cause and not an argued one. 47% of the sized symbols in the statically linked C++ binary are locale and stream facets, among them money, date and wide-character facets this program never instantiates, because a static `libstdc++` link brings the facet table in as one set. Neither dynamic configuration is free either, since each leaves a library on the host to be paid for once: 2531 KiB for `libstdc++` against 5295 KiB for Rust's. So the question has three answers and the 6.8× figure is the least informative of them. For a self-contained artefact, which is what a container image or an appliance wants, the ordering favours Rust.

The compared component resolves no third-party code at all. The standard objection to Rust in a regulated codebase is transitive-dependency risk, and on the artefact this study compares it does not arise: 0 crates and 0 `unsafe` sites, which turns the dependency-free claim of Section 5.4.4 into a measurement rather than an assertion. A surface appears in the configurations around that component. Linking the matched allocator resolves 8 crates, 5 of them reaching the artefact and 3 running only while it is built, which is a distinction worth keeping because a build dependency puts no instruction in the binary and still runs code on the build machine. The benchmark harness resolves 17, and counted with one syntactic rule those two trees hold 623 and 1064 `unsafe` sites.

Two readings should be kept apart. The count bounds where an audit would have to look and is not a risk score, since a crate concentrating its `unsafe` in one large block scores below one spreading the same work thinly. The C++ column is empty by construction and not by advantage: the language marks no such region, so the whole program stands where Rust's `unsafe` blocks stand and a zero would be a category error. What the axis

establishes is that the component which would ship carries none of this project's unaudited code, and that the largest surface belongs to the measurement apparatus.

7 Discussion

7.1 Summary of findings

Table 7.1 collects the headline finding for each research question, together with the strength of the current evidence. The safety results (RQ2) do not depend on the measurement environment at all. The migration results (RQ4) depend on it along one axis only, because build time is a timing measurement, which is why it is reported as a paired ratio. The performance and resource results do depend on the environment, and the honest division now falls inside RQ1 and not between chapters. The saturation knee, the overload comparison, and the resource comparisons are resolved on the host of Section 6.1, and they replicate across the allocator control. So is the far tail below saturation, in Rust's favour, which was the standing open question of this study for as long as it measured on a virtualised machine. The division that remains is not between results but between hosts: the same design on the earlier host could not resolve that percentile at all, and Section 6.4.4 reports what changed and what did not.

Table 7.1: Summary of findings per research question and the current strength of evidence.

RQ	Headline finding	Evidence status
RQ1	The answer divides at saturation. Below it the variants agree to 2 μs (2.1%) at the median, and the far tail resolves in Rust's favour, with the pooled p99.99 of the two Rust variants below both of the others by up to 87 μs. The end-to-end number is only 0.250% operation, so it cannot bound a language effect in any case, while the operation measured apart is 1.6× faster in Rust. Past saturation they diverge decisively, and in both directions: the Rust arms hold their offered rate one rung further, to 448 kops/s against 392, while at 784 kops/s, where all four have collapsed, C++ sustains 1.7× the throughput of Rust in 20/20 repeats under both allocators.	Overload, operation and far tail: resolved, and each reproduced at a second shard count. Mechanism: tested. Primitives alone insufficient. The saturation-limit gap is descriptive, not tested
RQ2	All seven injected memory-safety and data-race classes are prevented at compile time in safe Rust, where in C++ each compiles and is caught, if at all, only at runtime and by no single sanitiser. The guarantee stops there: the seven boundary classes compile in both languages, and on the leak and the release-build overflow the C++ tooling reports what Rust's does not.	Final
RQ3	A memory penalty that reproduces: 1.4–2.5 MiB more peak RSS in the Rust build under *both* allocators, in the same direction and order of magnitude on all three machines measured. The allocator's own 1.5–2.4 MiB is comparable here and was an order of magnitude larger on the virtualised host, so their ratio does not travel (Section 6.4.4). CPU inverts across the regimes: Rust draws up to 5.1 percentage points less below saturation and 1.8× more per completed operation above it.	Memory: resolved, and located outside the allocator by its own accounting. CPU: resolved in both regimes, opposite signs, and resolved with one sign on bare metal below saturation
RQ4	The seam is small and keeps the core `unsafe`-free at compile time. Measured against the same operation called natively, no cost is detected and any tax is bounded at 10.91% of an operation. The real cost is API design. The core remains exposed at run time to memory errors in the C++ host it shares an address space with. On the adoption axes beyond the seam, 54% of the Rust clean build is third-party code and the project's own code compiles in 0.8 times the CPU `g++` needs, the footprint ordering reverses with the linkage policy, and the compared component resolves 0 crates.	Final. Build-time absolutes are host-bound, the ratio is not

7.2 Discussion on the research questions

RQ1 (predictability). RQ1 has two halves, below saturation and beyond it, and the campaign answers them differently enough that reporting either alone would misrepresent the result.

Below saturation the languages are equivalent in central behaviour, by a margin no deployment would notice. The small paired difference at the median that does clear the correction is a fraction of a 70 μs operation. The far tail in this regime is a different matter, and one a deployment would also care about. There the result is an ordering rather than a null, and it favours Rust: pooled over millions of operations per variant, the two languages' p99.99 values do not meet, and the gap between the extremes is 87 μs, roughly the width of a whole median operation. So the equivalence claimed at the median does not extend up the distribution, and the direction in which it fails is the one that favours the safer language. That is a difference in the percentile a deployment sizes its margin from, and it is reproduced at a second shard count on a second machine (Section 6.4.4), which is as much corroboration as this study can offer for any timing result. The central result is consistent with the wider evidence that Rust's safety checks are largely absorbed by modern out-of-order CPUs [26, 71]. It is also weaker than it looks, for the reason Section 6.4.1 gives. The operation the languages execute is 0.250% of what the sweep measures. An end-to-end null in a design dominated by a cross-thread wakeup therefore says something about the design and not about the languages. Measured on its own, the operation differs by 1.6×.

Beyond saturation the answer changes in kind, and it changes twice. Where each implementation stops meeting its offered rate differs by one rung of the ladder, with the Rust arms holding to 448 kops/s against the C++ arms' 392. Deeper still, once both have collapsed, the ordering reverses: the C++ build sustains 1.7 times the throughput of the Rust one, reproducibly and at the corrected floor of the whole test family, while the Rust build draws more CPU for each operation it completes (Section 6.4.2). Only the second of those is tested, and Section 6.4.1 says why the first is not. So the practical form of RQ1's answer is not that the language choice is irrelevant to predictability. It is irrelevant *until the service stops coping*, and around that point it decides both when coping stops and how much work still gets done afterwards. A comparison that stopped at nominal load, as most in this literature do, would have concluded the opposite with the same apparatus. That the two effects point in opposite directions is not a contradiction to be resolved but the reason a single verdict here would be false: a service sized to run below its knee should prefer the arm with the higher knee, and one that a signalling storm will drive past every knee should prefer the arm that degrades less steeply.

That reframes the closest prior work instead of merely agreeing with it. Arvidsson and Kaller measure this quantity for 5G control-plane procedures and find Rust's 99th percentile degrading faster beyond a threshold request rate. They trace it to stackless `async/await`, which permits a context switch immediately before an I/O operation begins [55]. Section 7.5 accepts that attribution, and this study was built to exclude the confound behind it: one worker per shard over a bounded blocking queue, and no asynchronous runtime on either side.

Section 1.3 committed in advance to what each outcome would mean, and the second branch is the one that fired. A degradation past a threshold rate appeared with the concurrency model held fixed, so the asynchronous runtime cannot be the only place such an effect starts. The prediction also named where to look first if that happened: bounds and overflow checking [72, 73]. That part of it was wrong. The effect appears only once the handoff path is contended, which points instead at the two standard libraries' blocking primitives recorded in Table 5.1. Section 6.4.2 then tests whether those primitives alone explain the gap, and they do not reproduce the full-service ordering at significance. That narrows the cause to extra work on the C++ handoff path and not to the safety model. Recording that the prediction resolved against its own first hypothesis, and that the follow-up microbenchmark refined the attribution instead of confirming it, is more useful to the next study than quietly swapping in the explanation that survived.

RQ2 (safety). The result does not depend on the environment. What makes it interesting is not the half that confirms the expectation, but the half that bounds it. Within its scope the guarantee is absolute in one specific sense, and the difference from C++ is one of kind and not of degree. A compiler that refuses to build a program rules the defect out of every execution. A sanitiser rules it out only of the executions an instrumented test happens to take. What is absolute is the execution coverage, not the breadth of the catalogue. The scope itself is still the 14 programs written for it. That fits the CVE evidence that Rust memory-safety bugs always involve `unsafe` [42], and the caution that unsafety can still spread through dependencies [43], which is why keeping the core `unsafe`-free matters. The seam double-free is that evidence reproduced under controlled conditions: the one class in this catalogue that safe Rust rejects and unsafe Rust does not.

The one case that reaches past the written programs is the lifecycle defect of Section 6.4.5. It was found in the implementation, not written for the catalogue, and it revises the picture in two directions at once. It strengthens the cost argument. A real defect in code written to be correct survived the entire verification apparatus of this study, meaning tests, parity vectors, and three sanitisers, purely because nothing exercised the path. Runtime detection that depends on test coverage is a weaker proposition in practice than it looks in a table where every row is driven deliberately.

It also narrows what may be credited to Rust, because the contribution there was a bounds check and not a proof. The honest generalisation is that the guarantee worth budgeting for is *checked by default* and not *ruled out at compile time*. The root cause is worth naming too, because it is a risk of the method and not of either language. A specification that states a prohibition without assigning a mechanism leaves each implementation to fail in its own idiom. The two idioms here are a panic and a use-after-free.

Where the guarantee stops matters just as much to a vendor deciding what Rust buys, and the two boundary classes are instructive for different reasons. The lock-order deadlock shows that Rust reasons about *aliasing* and not about *acquisition order*. A `Mutex` guard is an ordinary value whose lifetime the borrow checker tracks perfectly, and tracking it perfectly says nothing about whether two threads take their locks in compatible orders. The tooling built around the language confirms that this is a boundary and not an artefact

of the injected example. Static deadlock detection for Rust is an active research problem in its own right [74], and model checkers for Rust kernel code are motivated explicitly by the observation that ownership and borrowing leave data races, deadlocks, and livelocks reachable [75].

The lost update is sharper still. It escapes ThreadSanitizer for precisely the reason it escapes the borrow checker, as Section 6.4.5 sets out. With every access correctly synchronised there is no unsynchronised access to find. Nor is the class marginal. In a field study of 105 real concurrency bugs, about a third of the non-deadlock cases violated the order the programmers intended, not their locking, and the authors record that such intentions are not readily expressible through locks at all [76]. The distinction it exposes, between a data race and a race condition, is the one the phrase "thread safety" routinely runs together, and Rust removes only the first.

The practical consequence is that adopting Rust retires one category of concurrency defect completely while leaving another untouched. The category left untouched is the one that fails silently, corrupting a counter or a state decision instead of crashing. A migration plan that treats "memory-safe" as "concurrency-safe" would therefore mis-budget its review and testing effort. That is why this boundary is reported as a first-class result and not as a caveat. Figure 7.1 draws the boundary itself, with the two halves of the guarantee on either side of the line and the mechanism that does the enforcing, or the absence of one, named in each panel. It carries the two concurrency cases rather than all seven outside the guarantee, because they are the pair that makes the distinction, and Table 6.11 holds the full catalogue.

RQ3 (overhead). The answer splits by resource, and the splits do not all point the same way. On memory there is a penalty. The Rust build carries a few MiB more peak resident set than the C++ build, and it keeps that sign under both allocators. Two things make it credible beyond the p-value. Peak RSS is a run-level maximum and not a tail statistic, so the scheduling jitter that destroys the p99 does not touch it. And it reproduces on a second machine with a different compiler (Section 6.4.4), which no other timing-adjacent result in this study can claim. The useful way to size it is against the allocator's own effect, and that comparison is the part which does *not* travel. The two are of comparable size on the reported host and on the second bare-metal one, while on the virtualised host the allocator's was an order of magnitude larger, because the allocator effect shrank by more than ten times across the change of machine and the language effect did not move. So the language effect replicates and its ratio to the allocator's does not. A team choosing between these two languages moves a few MiB either way, and whether that is more or less than choosing between these two allocators is a question about the machine.

On CPU the answer depends on the load, and that dependence is the finding rather than a complication. Below saturation Rust draws slightly *less*, consistently under both allocators. Past saturation the sign flips and the magnitude grows (Section 6.4.2). A single number for "the CPU cost of Rust's safety model" would therefore average two regimes that behave in opposite directions, and neither figure is the safety model in any case. Throughput splits the same way and for the same reason. Below saturation there

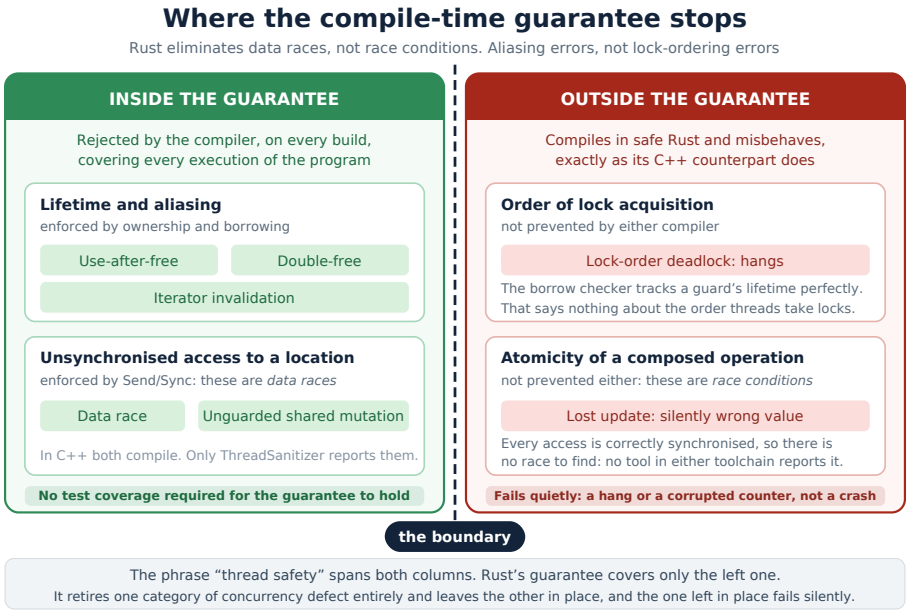


Figure 7.1: Where the compile-time guarantee stops. On the left are the classes safe Rust rejects outright, grouped by the *mechanism* that rejects them rather than by defect family: ownership and borrowing for lifetime and aliasing errors, which is why iterator invalidation sits with them, and the `Send`/`Sync` discipline for unsynchronised access to a location. On the right are the classes the catalogue included to find the edge, which neither compiler prevents. The diagram names one class per mechanism and is not the catalogue, which Table 3.1 lists in full: the use-after-move belongs with the ownership cases on the left and the shared thread-unsafe handle with the `Send`/`Sync` ones, while the leak, the release-build overflow, the worker-failure case, the seam double-free and the forgotten guard sit alongside the two on the right. The phrase "thread safety" spans both sides of the line, and the guarantee covers only the left. The line itself moves once more than the diagram can show, because a program that writes `unsafe` crosses it voluntarily.

is nothing to measure, because achieved throughput tracks offered load by construction. Above it sits the largest effect in the study.

Back to the memory penalty, because three things bound what it licenses and the third is new. It is a fixed cost rather than a per-context one, so its share of a deployment's footprint falls as the deployment grows. It is also located, and that took an instrument no virtualised host in this study offered: the allocator's own accounting puts the difference *outside* the allocator, since the Rust build holds less live data and fewer resident bytes inside `jemalloc` and still peaks +1.74 MiB higher (Section 6.4.4). What a Rust process carries before it allocates is the standard library and the runtime, and that is a one-off charge per process rather than a tax on the work. And it should not be widened into a claim about Rust's safety model, because nothing here isolates bounds checking or ownership-driven copying as the cause. The mechanisms the literature has localised [72, 73] are candidates to test, not conclusions to import. What can be said is narrower and still useful to a vendor sizing a control-plane node. Adopting Rust here costs a few megabytes of fixed footprint per process, and it buys a faster core operation (Section 6.4.1) that this workload's structure never turns into throughput below saturation, and that turns into a deficit above it.

The sign of that result is worth defending, because one of the closest prior works in the domain reports the opposite sign, while two others agree with it. Agreement first, since

it is quickly stated. A second Ericsson-hosted study of RAN node management software found Rust's peak memory usage generally higher than C's [53], and a matched pair of user-space device drivers outside telecom put the Rust version's footprint about 29% above the C version's [77]. Neither separates the allocator from the language, which is the distinction Section 6.4.3 rests on, so they support the direction of the effect and not the way it splits.

The disagreement is the more instructive case. Karlbäck and Orö measure peak resident set on a RAN workload and find Rust the *lowest* of the three languages and C++ the highest, with the gap widening as concurrency rises. They call the pattern surprising and attribute it tentatively to the initial stack size of the green threads [52]. The two findings are compatible once it is clear that they measure different quantities. Their C++ runs on Boost.Fibers, which allocates a stack per fiber, so ten thousand concurrent sequences cost ten thousand stacks. Their Rust runs on Tokio, where a task is a heap-allocated future of a few hundred bytes. What their figures capture is therefore the per-context cost of the concurrency runtime, which is exactly why it grows with the number of contexts.

The design measured here has no green threads on either side. A fixed thread-per-shard pool leaves no per-context allocation to differ, and what remains is the fixed footprint reported above, constant in the number of UEs and not proportional to it. Their disagreement with this result is therefore between two concurrency runtimes and not between two languages, which is as direct a demonstration as the literature offers of why this study holds the concurrency model fixed. It also sharpens the caveat attached to the present number. It licenses nothing about Rust's memory behaviour under a task-per-UE design, which is a different experiment.

RQ4 (migration). The prototype shows that incremental adoption is technically cheap. A C++ host can call an unmodified Rust core across a thin C-ABI seam whose cost on a real operation is bounded at 10.91%, and all `unsafe` stays inside the seam. That bound comes from the matched comparison of Section 6.4.6, not from setting an empty crossing against a full round-trip. The two are not the same experiment, and on this host a single unpaired pass of each was enough to flip the apparent ordering. The dominant cost is therefore human and not machine: designing a C-compatible surface and, as the adoption studies report [78, 79], climbing Rust's learning curve. That makes migration risk an organisational concern and not a performance one, and the two largest documented adoptions into existing C and C++ codebases report the same ordering of costs [80, 81].

One boundary on that result has to be drawn explicitly, because the natural reading overstates it. "All `unsafe` confined to the seam" is a claim about what the compiler verifies, and it is true as measured. The core crate contains no `unsafe`, and every raw-pointer operation sits in 9 reviewable functions. It is not a claim that the core is protected at run time. Host and component share an address space, and a memory error on the C++ side can corrupt Rust-owned data however carefully the Rust side was checked. That is why the remedies proposed in the literature isolate the two heaps with hardware support instead of relying on interface discipline [47, 48], and the defect classes that arise at such boundaries have been catalogued in their own right [49]. So a migration of this shape buys less than "the Rust part is safe". It buys a migrated component that stops

contributing new memory-safety defects, while it stays exposed to those of the host until the host is migrated too. That is still the right direction of travel, and it is a reason to sequence a migration so that the components sharing the most mutable state cross over together instead of one at a time.

The three axes measured beyond the seam sharpen the same conclusion, and they complicate two of its folk versions. Build time is the objection most often raised against Rust in a large codebase, and it survives only in the aggregate. The full build does cost 1.8 times the CPU. But 54% of that is third-party code, so the project's own source compiles for 0.8 times what `g++` spends on its equivalent. An organisation worried about build time should look at its dependency budget before it looks at the compiler. The edit-compile loop is the axis where C++ genuinely leads, and it leads unevenly. Its advantage holds for edits inside a translation unit and reverses for edits to a widely included header.

Binary size behaves worse than an argument would predict. The 6.8× figure that gets quoted compares two linkage policies, and once the policy is held fixed the ordering depends on which one was chosen. The axis that matters most for a regulated codebase has the cleanest answer of the three. The component compared here resolves no third-party code and contains no `unsafe`, so the transitive-dependency argument against Rust adoption does not reach the artefact under discussion.

7.3 Revisiting the hypotheses

Section 1.3 committed to three hypotheses before any measurement was taken. Stating their disposition explicitly, rather than letting the results speak for themselves, is what distinguishes a test from a description.

H1: supported below saturation, refuted above it. The hypothesis had two parts: that the languages are performance-equivalent below saturation, and that any difference is confined to the far tail. The first is supported. The second is refuted, and not in the direction the wording anticipated, for the reasons Section 7.2 gives. The largest difference the study found lies in overload, which H1 did not consider at all. The hypothesis as written would have led the campaign to look in the wrong place, and the result was found only because the sweep was pushed past saturation on methodological grounds rather than in expectation of it.

The clause H1 did name turns out to be half right, and the half it gets right was the harder one to reach. For most of this study the far tail could not be resolved at all, so on the evidence then available H1's second part was untestable rather than refuted. It is resolved now, on the reported campaign, and a difference is present in the far tail below saturation in Rust's favour at loads where the median difference is about a microsecond on an operation of 70 µs. A second bare-metal campaign at another shard count agrees on the direction (Section 6.4.4). So H1 was right that the far tail is where a below-saturation difference would show up, and wrong that the far tail is the only place any difference shows up.

A second disposition is that the hypothesis is not well posed. "Performance-equivalent" was written as though it named one quantity. Section 6.4.1 shows it

names at least three. There is the delivered operation below saturation, where the languages are equivalent. There is the code the compilers emit, where they are not, and where the end-to-end measurement dilutes the difference by more than two orders of magnitude. And there is sustained work under overload, where they are not equivalent either and the difference runs the other way. All three are true, and a hypothesis that cannot tell them apart is one this study should have stated more carefully before measuring.

H2: supported in its precise form, refuted in its loose one. The hypothesis was that safe Rust prevents at compile time "the memory-safety and concurrency defect classes that C++ leaves to best-effort detection". For the seven injected classes of that kind it holds without exception. The fault-injection study still refutes the broader reading the wording invites. The seven boundary classes are exactly the sort C++ leaves to best-effort detection, safe Rust does not prevent them either, and on the leak and the release-build overflow the C++ tooling is the one that reports. H2 is therefore accepted for memory safety and data-race freedom, and rejected for concurrency correctness at large. It also carries an unstated precondition, which the seam case makes explicit: the hypothesis is about *safe* Rust, and the same class it rules out returns as soon as a program writes `unsafe`. The second reading is the one practitioners tend to assume.

H3: supported, and tested the right way on the second attempt. The hypothesis was that interoperability cost is negligible against the operations invoked. Measured directly (Section 6.4.6), no cost is detected and any per-operation tax is bounded at 10.91%. What supports the hypothesis is the bound, not the failure to detect a difference. The first version of this measurement did not deliver that. It set an empty crossing against a full round-trip and inferred the answer, which compares two different experiments and would have given the right conclusion for the wrong reason. The accompanying claim, that the real cost is engineering effort, is supported structurally and not statistically. The seam is small and mechanical, but designing its C-compatible surface was the part of the prototype that took actual design decisions.

7.4 Engineering guidelines for incremental adoption

The results are meant to be usable, and a vendor weighing Rust for a control-plane component needs more than a set of measurements. The following guidelines are the practical form of the evidence above. Each is tied to the finding that supports it, and the limits of that support are stated and not glossed. A guideline quoted without its limit is how the RQ2 result becomes "the migrated component is safe" and the RQ4 result becomes "migration is cheap", and this study establishes neither. They are given in the order a first migration would run them.

Choose the first component for its interface, not its size. The boundary's runtime cost is bounded at 10.91% of an operation (RQ4), so the criterion for a first Rust component is not how small or how hot it is. It is how narrow and how scalar its interface

is. The seam here stayed at 11 entry points because the subsystem's API was already lifecycle-oriented, typed, and free of shared mutable structures crossing the boundary. A component whose interface passes rich C++ objects, templates, or callbacks will need those designed out first, and that redesign is the schedule risk, not the FFI.

Put the ABI in its own crate and keep the core `unsafe`-free. Concentrating every raw-pointer operation in a separate seam crate that depends on an unmodified core (Section 6.4.6) is worth adopting as a rule, not treating as an accident of this prototype. It keeps the dependency-propagation problem Evans et al. describe [43] away from the safety-critical code, and it makes the reviewable surface of the migration a fixed, countable thing. Two qualifications belong with it. It bounds what has to be *reviewed*, not what can be *corrupted*. While the host remains C++ the two share a heap, so a team needing the stronger property has to look at in-process isolation [47, 48] and not at interface discipline alone. And the generated part of a seam is not above suspicion, because binding generators have their own defect history [50]. Review it as code instead of trusting it as tooling output.

Do not re-budget concurrency review effort on the strength of memory safety. This follows directly from the boundary cases. Adopting Rust removes use-after-free, double-free, iterator invalidation, and data races from the defect budget entirely, and those are the classes that dominate the security statistics [8, 9]. It removes nothing from the deadlock and atomicity-violation budget. Lock-ordering discipline, invariants about which locks may be held simultaneously, and review for composed-but-not-atomic operations remain exactly as necessary as they were in C++, and they should stay in the code-review checklist and the design-review agenda after migration.

Prefer a shared-nothing structure to a shared-and-locked one, in either language. The strongest concurrency result in this study belongs to neither language. The thread-per-shard design removes locks from the hot path altogether, which is why the boundary defects had to be built as standalone programs instead of found in the implementations. Where state can be partitioned by a natural key, as UE state can be partitioned by `ue_id`, partitioning it turns a class of concurrency defects into a structural impossibility for both languages, and it does so without either compiler's help.

Budget for the learning curve, and measure it. The dominant cost identified here is human, which matches what adoption studies report [78]. This thesis cannot quantify that cost, because it is a single-author study, so the guideline is correspondingly modest. Treat the first component as a training investment with a deliberately generous schedule, and instrument it, recording review time and defect density, so the second component can be planned on measured effort instead of assumed effort.

Measure the component, not the round-trip, and do not generalise either. The performance findings carry a methodological warning that travels further than the

numbers do. An end-to-end request/response measurement of a design like this one is 0.250% language and the rest scheduler (Section 6.4.1). Below saturation it will therefore return "no difference" between almost any two competent implementations, and reading that as evidence about the languages is a mistake this study made before it measured the components apart. A vendor running its own comparison should budget for both: the end-to-end measurement, which says what a deployment will feel, and the component measurement, which says what the language is doing.

Sweep past saturation, because that is where the languages stopped agreeing. This is the guideline the study would most like to have followed from the start instead of arriving at. Every conclusion it drew about performance while the sweep stopped at nominal load was a null. Each was correct at that operating point and silent about the one where the two implementations differ by 1.7× (Section 6.4.2). The knee is not a detail at the edge of the graph. For a control plane it is the region that decides whether a signalling storm degrades service or ends it, and it is where a bounded, blocking design has no graceful behaviour left. A comparison that stops while the system is still meeting its offered load has measured the easy half of the question. Sweeping past the knee costs one more afternoon of machine time.

Beyond both of these, the reusable artifact is the benchmarking framework and not its current output. The decision it should inform is "measure this on your component, and measure it past the point where your component stops coping", not "Rust is fast enough, proceed".

7.5 Related work

Chapter 2 set out the ground this work stands on in four strands, and restating them would add nothing. Each is reduced here to the position it leaves this thesis in, after which the studies it has to answer to are treated at length.

The performance strand is settled on the average case. Compiled systems languages sit close on time and energy [44, 82], Rust's safety is cheap at run time [26, 83], and that holds where timing is tightest, in embedded hypervisors, production cloud infrastructure, and packet processing within a few percent of a C reference [21, 71, 84, 85]. Where a cost does appear it has been pinned down: to bounds, overflow and division checks, to that same check set against hardware-enforced capabilities, and to recurring anti-patterns [72, 73, 86]. One like-for-like port puts the bounds-check share at 34% of a 46% geometric-mean overhead, and cautions that code credited with cheap checks has sometimes declared its hot functions `unsafe` [87]. That caveat does not reach the `unsafe`-free core compared here. None of this work reports behaviour past a saturation point, which is the regime this thesis targets.

The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect [11, 39, 40], confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it [27, 41, 42], auditing that finds defects at scale even so and unsafety spreading through dependency chains [43, 88, 89, 90], and the reports motivating the shift set against a long

record of memory corruption resisting runtime mitigation [8, 9, 16, 18, 19, 33].

The interoperability strand qualifies the adoption argument, and Section 2.6 states its central limit: confining `unsafe` to a seam does not protect the Rust side, because foreign code in the same address space can corrupt Rust-owned data anyway [47, 48]. What that section does not cover is reach. The seam carries its own defect classes and its own tooling bugs, lighter crossing models exist for embedded use, aliasing patterns expressible in C but not in safe Rust bound translation in principle, interoperation is the central open problem in embedded Rust, `unsafe` concentrates at exactly these boundaries, and the two largest documented adoptions into existing C and C++ report the resulting friction empirically [49, 50, 80, 81, 91, 92, 93, 94]. Automating the rewrite is not a solved problem either, since current translators still emit code whose safety the compiler cannot validate [95]. That is why the migration cost measured here is the cost of a hand-written seam.

The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature [6, 7, 30, 96], thread-per-core structuring moving the tail in its own right [45], O-RAN disaggregation [3], control-plane latency as an active target in the core network too [97, 98], adoption evidence pointing at the learning curve rather than run-time cost [78, 79, 99], and standard guidance on repeat-based evaluation and fault injection [32, 59]. Rust has mobile-network precedent in the datapath [100] and, on the control-plane side, an open-source gNB-CU with a separated CU-CP and CU-UP over E1. That last one bears on the gap rather than on the method. It establishes that the ASN.1 and SCTP protocol surface is no barrier in principle, but it is an existence proof and not a comparison, reporting no latency and carrying no C++ counterpart built to the same specification [101].

Four systems studies come closest in method, though not in domain, because they too build the same component twice and attribute the difference. One compares IPC mechanisms implementing a single network function in both C++ and Rust [102]. RedLeaf is evaluated against C baselines built for the purpose [103]. A manual translation of the STAMP transactional benchmark suite into Rust is the closest published match to this study's design, because its workload is concurrent and both versions implement one specification [104]. The fourth measures a Rust UDP tunnelling driver in the Linux kernel against an equivalent C driver [105]. That last one is worth a second look for its method and not its result. It reports Rust slower by 0.19% at $p = 1.5 \times 10^{-15}$, a difference overwhelming in statistical terms and negligible in operational ones. Keeping those two apart is what the pre-committed interval-versus-p rule of Section 4.4 exists to do. Like the wider literature, none of the four measures past saturation.

Five master's theses, all carried out at Ericsson, are the closest prior work by domain and deserve separate treatment. None of them is peer-reviewed, and none is indexed by any citation database, so they are cited here as the record of what a vendor has actually attempted rather than as established results.

Karlbäck and Orö's control deserves to be quoted and not paraphrased, because it is the clearest statement in this literature of the confound the present study is built to remove. The authors record that they deliberately did not write the same program three times. They "aimed for providing the best solution in each language, instead of trying to make

the code as identical as possible between the languages", and they "stayed away from using the exact same solution in each language" [52]. For the question they ask, which is what a team would get by writing idiomatic code in each language, that is the right choice. It is the wrong control for the question asked here, because the concurrency runtime then varies with the language (Boost.Fibers over libevent, goroutines, and Tokio respectively) and no measured difference can be divided between the two. Section 7.2 returns to this where their resource results and the ones reported here have opposite signs, and the disagreement resolves into a difference between runtimes and not between languages.

Olsson asks a narrower version of the same question two years later, and two of its three findings agree with the ones reported here, including the resource result on which Karlbäck and Orö disagree with this study and which Section 7.2 takes up [53]. What that study does not do is report percentiles or drive the software past its capacity.

Olsson and Stenström are the closest prior work on the concurrency model this thesis adopts, and their result is the sharpest illustration in the literature of why the model has to be held fixed. Read as a language comparison, asynchronous Rust at a fifth to a half of DPDK's throughput is damning [54]. Read as what it is, a comparison between a young asynchronous runtime and a kernel-bypass framework tuned over a decade, it says almost nothing about Rust and a great deal about the runtime: an effect size of two to five times is not the scale at which code generation operates. It is also the reason the design measured here uses one thread per shard and no asynchronous runtime on either side, and Chapter 8 records the comparison that would separate the two properly.

Arvidsson and Kaller are closer still, and their result is the most direct challenge in the literature to the expectation this thesis set out with. Across five asynchronous frameworks on 5G control-plane procedures, Boost.Fibers shows the lowest latency at every load, and while the Rust runtimes track it at the median, their 99th percentile rises more steeply beyond a certain request rate [55]. That is a tail divergence in the right domain, and it would be dishonest to omit it. It is also a comparison in which the concurrency runtime co-varies with the language: the authors attribute the effect to stackless `async/await` permitting a context switch immediately before an I/O operation is initiated, which is a property of Tokio and Monoio rather than of Rust.

That attribution does not have to be taken on trust, because the runtimes have since been examined directly in a real-time setting. Škoudlil et al. dissect the execution model of asynchronous Rust against a thread-based C++ equivalent. They report that the runtimes' scheduling policies are "complex and difficult to understand due to the use of abstraction layers", and that Tokio specifically is "designed to maximize throughput rather than time determinism". Restructuring the same application onto prioritised threads over a different runtime yields response times that match classical response-time analysis, which the idiomatic work-stealing configuration does not [46]. The tooling being built around those runtimes points the same way. Priority scheduling is something an asynchronous Rust runtime has to be given by a library, not something it already provides [106]. The tail behaviour at issue is therefore a property of the runtime, and it changes when the runtime does. The two studies are complementary and not contradictory. Theirs establishes that a tail difference is observable in this domain. This one is built to determine whether the language is what causes it.

Emberg and Wadsten are the closest of the five in method, and the honest way to position this thesis is against them rather than against the framework comparisons. A matched pair, an FFI seam, a statistical comparison and an effort measurement is four of the things this thesis does, in the same industry, two years earlier [56]. Three differences carry the gap. The object is a stateless transformation rather than a stateful, concurrent subsystem, so there is no concurrency model to hold fixed and no shared state to contend for. The measurements are taken at the load the benchmark happens to generate rather than swept to saturation and beyond, so no tail behaviour under overload is reported. And the safety side is argued from developer interviews rather than from a defect catalogue, so nothing establishes which classes the compiler actually rejects.

Their study also supplies something more useful than a comparison, which is a warning. Their two benchmarking sessions reversed the verdict. The first, after a code review of the kind normally performed in practice, found the two implementations comparable. The second, after both sides had been optimised further, found the Rust version worse by 85–91%. The authors reason that the first session is the more representative of software in general, which is a defensible reading. The more important observation is that a single-author matched pair can be made to yield either answer, depending on where the optimisation effort lands. Section 7.6 treats that as the principal threat to this study's own construct validity and describes what is done about it.

Two 2026 theses from outside the vendor set apply parts of this thesis's method to smaller objects and arrive at compatible answers independently. One builds functionally equivalent C and Rust user-space device drivers and finds that safety costs nothing measurable in run time for that class of driver but carries a fixed memory overhead [77]. The other reproduces a set of CWE classes in C++, Rust and Python to establish at which stage each fault is caught, which is the staging question RQ2 asks [107]. Neither involves concurrency, a telecom workload, or a load sweep, and both are bachelor's work, so they are cited as convergent evidence on the smaller questions and not as substitutes for the comparison attempted here.

Table 7.2 sets this study beside the five directly, and two things are worth reading off it. The earlier work is not weaker, only aimed differently: four of the five measure a language together with the framework it was written on, and the fifth measures a matched pair on a stateless object at a fixed load. The verdicts in the last column then contradict one another in every direction, on peak memory, on throughput and at the median, because no two of them hold the same things fixed. That is the signature of a literature in which the independent variable is not controlled, and it is why this thesis builds its object of study instead of finding one.

Section 2.7 recorded that all five come from one vendor, and read against the table that concentration has a sharper consequence. "C++ remains the choice where performance is critical" and "asynchronous Rust is not currently viable" are findings about particular software on particular infrastructure, and repetition inside one organisation does not establish their generality. The question is open rather than settled elsewhere and imported, and the answer given here is one an independent organisation can check, because the specification, both implementations and the harness are published with it.

Not all of this evidence points the same way, and the qualifying findings are taken as

Table 7.2: This study beside the five vendor theses, all carried out at Ericsson and none peer-reviewed. The rows are ordered by year. The last column reports each study's own verdict in its own terms, which is why the entries do not line up: they are answers to different questions, measured on different objects in different regimes. The verdict recorded for this thesis is the one Section 7.2 defends, including the part that runs against its own expectation. Figure 2.6 places the same five studies on the axes of the gap instead, alongside the peer-reviewed literature.

Study	Object measured	Regime reported	Reported verdict on Rust
Karlbäck & Orö, 2021 [52]	RAN applications on x86 and ARM, one idiomatic implementation per language	Runtime and resource use at the loads exercised	C++ preferred where performance is critical. Rust's peak memory the lowest of the three languages
Olsson, 2023 [53]	RAN node management software, Rust against C	Execution time and peak memory	Speed on par with or better than C, peak memory higher, safety materially improved
Olsson & Stenström, 2023 [54]	Packet processing: asynchronous Rust on Glommio against DPDK Eventdev in C	Throughput and latency against core count	20–50% of DPDK's throughput with far worse latency. Asynchronous Rust judged not currently viable
Emberg & Wadsten, 2023 [56]	An ASN.1 decompiler built twice, the Rust side reaching C++ through a C wrapper	Execution time in two benchmarking sessions	Comparable in the first session, 85–91% worse in the second. Preferred in practice on non-performance grounds
Arvidsson & Kaller, 2025 [55]	5G control-plane procedures across five asynchronous frameworks	Median and 99th percentile across a load sweep	Median tracks C++. The 99th percentile rises more steeply past a threshold request rate
This thesis, 2026	A UE Context Management service built twice from one frozen specification, with the concurrency model, allocator and build settings fixed	Median, p99, p99.9, p99.99 and maximum across a sweep to and beyond saturation, on a bare-metal host and reproduced at a second shard count	Equivalent at the median below saturation. The far tail there resolves and favours Rust. Past saturation C++ sustains 1.7× the throughput of Rust

seriously here as the favourable ones. Even the positive networking result records a real cost. The idiomatic Rust driver issues substantially more instructions per packet than its C reference and is measurably slower in absolute terms. Its safety checks are absorbed by out-of-order execution, not literally free [71]. Two further matched comparisons point the same way on different workloads. Idiomatic safe Rust implementations of the STAMP transactional benchmarks run slower than both the C originals and machine-transpiled Rust [104], and a general study of Rust against C reports it significantly slower in some cases [108]. The largest study of Rust inside an existing C codebase reaches a similar conclusion from the other direction. It finds that Rust's safety features introduce runtime overhead and development complexity when they are not carefully managed [80].

The evidence does not run one way, though. Safe low-level code without run-time overhead has been demonstrated for network device drivers, the same class of program as Emmerich's, when the safety obligations are discharged through type invariants rather than checks [109]. The "zero-cost" framing [26] is therefore best read as neither established nor refuted in general, but as a property of a particular program on a particular workload. That is why tail behaviour must be measured on the target workload and not assumed.

The safety story is likewise not absolute. Empirical studies of real-world Rust find that memory- and thread-safety bugs still occur in practice, with blocking bugs and misused synchronisation among them [27, 90]. They also find that a substantial fraction of the ecosystem cannot be fully statically checked, because `unsafe` propagates through dependency chains [43].

These qualifications do not overturn the case for Rust, but they motivate two of the thesis's design choices. The first is measuring predictability and defect prevention directly on the subsystem instead of importing general-purpose, average-case conclusions. The second is keeping the safety-critical core `unsafe`-free, so that the dependency-propagation caveat does not apply to it.

Three axes separate this thesis from all of it: whether the comparison controls for design by building both systems from one specification, whether it reports tail behaviour under overload instead of average-case performance, and whether it measures the cost of adoption instead of describing it. None is unoccupied on its own, as the matched implementations [56, 71, 102, 103, 104, 105, 109], the percentile sweep stopping at a latency requirement [55] and the effort measurements [52, 78] all show. No prior study combines the three for a stateful, concurrent control-plane workload, and that is the narrower gap this work occupies.

7.6 Threats to validity and mitigations

Every conclusion here attributes a difference to the implementation language. The credibility of the study therefore rests on the controls that hold everything else equal, and on an honest account of where those controls stop. Table 7.3 maps the threats first, by the kind of inference each endangers, so the remaining risks can be read as a set instead of gathered one at a time from the prose that follows. Three are load-bearing. The study cannot rule out single-author skill asymmetry. Its end-to-end instrument is the wrong

one for part of what RQ1 and RQ3 ask. And its strongest single finding, the overload divergence, rests on an attribution it did not measure directly.

Internal validity: is the language really the only difference? The central threat is implementation bias between the two versions, mitigated structurally by the controlled-variable strategy of Table 4.2 and the transcript-level parity testing of Section 5.6. What those leave behind is single-author skill asymmetry. One person wrote both implementations and is not equally expert in both, so an advantage to whichever language that person knows better cannot be ruled out.

That the threat is real and not merely conceivable is documented in the same industry, by the two benchmarking sessions of Emberg and Wadsten that reversed their own verdict [56]. One matched pair supported two different conclusions, depending on where the optimisation effort landed. The controls here are chosen against that failure mode. The specification is frozen before either implementation is written, not negotiated between them. The optimisation level and allocator are matched, not tuned per side. And forced divergences are logged, not resolved in whichever direction the author finds easier. None of that reduces the remaining risk to zero. It makes that risk visible, which is why the forced-idiom log is a deliverable and not an appendix.

Construct validity: do the metrics measure what the questions ask? RQ1 is answered with high percentiles and the maximum instead of the mean, which is the standard way of measuring predictability in this literature [6]. Two risks remain. The metric list was narrowed while the instrumentation for inter-operation jitter and p99.99 did not yet exist. Both are now collected and reported, so the narrowing has been undone rather than merely acknowledged, and RQ1 was widened to name the statistic that turned out to decide it. Jitter is the one metric the question still does not name, and it is reported descriptively for that reason rather than tested. And p99.9 on a synthetic workload is not the same quantity as a missed procedure deadline in a live network.

A second and larger construct gap in RQ1 was found rather than anticipated. The end-to-end latency the sweep reports is 0.250% operation and the rest cross-thread handoff (Section 6.4.1), so it measures the delivered cost of the design faithfully and the cost of the language barely at all. Which of those RQ1 asks about depends on the reader. A vendor sizing a node wants the first, a language comparison the second. The study reports both and keeps them apart, but the question as originally worded did not tell them apart. Two remedies are applied, and no amount of statistical care substitutes for either. The operation is measured apart from the handoff, and the sweep is pushed into the regime where the design stops absorbing the difference. What remains is that a microbenchmark of one operation is a narrower object than the sweep it supplements.

The remaining three are stated in Table 7.3 and bounded here. RQ2's catalogue is a sample the author chose, mitigated by boundary classes that force it to contain cases the expected conclusion does not cover, two of which returned the opposite result. Fourteen programs are still fourteen programs. RQ3 captures resource cost, and its origin only in part: hardware counters show less work and worse locality (Section 6.4.4), which accounts

Table 7.3: Threats to validity by the kind of inference each endangers, the control applied, and what that control leaves behind. Only the last column holds live risks. Two of the controls arrived with the host comparison of Section 6.4.4.

Threat	Control applied	What remains
Internal: is the language really the only difference?		
Implementation bias between the two versions	Frozen specification, shared allocator and optimisation level, pinned hash functions, forced-idiom log, transcript-level parity testing	Single-author skill asymmetry. Parity bounds behaviour, not performance
Construct: do the metrics measure what the questions ask?		
The sweep measures the design, not the language	The operation is measured apart from the handoff and both are reported (§6.4.1)	One operation on one data structure is narrower than the sweep it supplements
The overload result is attributed but not localised	Narrowed to two entries in the forced-idiom log (§6.4.2), and the contended-primitive microbenchmark rules those two out alone (§6.4.2)	The full handoff path under load still includes variant dispatch and larger requests on the C++ side
A defect catalogue is a sample the author chose	Seven classes drawn from outside the guarantee, with expectations recorded in advance	Fourteen programs remain fourteen programs
RQ3 captures resource cost, and its origin outside the allocator	The allocator's own accounting, on two bare-metal hosts (§6.4.4)	No call site: the profiling build reverses the sign of the effect (§6.4.4)
RQ4 measures the seam, not the safety of the whole	The distinction is stated wherever the result is quoted	One address space: a compile-time property of the Rust side bounds nothing at run time
Conclusion: are the differences real or noise?		
Run-to-run non-determinism	20 interleaved repeats, bootstrap intervals, exact paired permutation test, Holm correction, floors quoted	Pairing cancels only what a block shared, which is why the far tail needed a quieter machine rather than more repeats
A corrected p-value that does not replicate	The allocator control runs every comparison twice. Replication is followed over the p-value	It has cost a result before and would again. Every comparison now replicates
A contaminated run defeating a mean-based statistic	The reading rule of §4.4, fixed in advance, plus the per-repeat agreement count	Nothing here: the bimodality behind the rule was the previous machine's. The rule is kept anyway
An operating point chosen after seeing the data	Both tested operating points are fixed in §4.4 before the results	The saturation-limit gap (§6.4.1) is therefore descriptive, and the one comparison with no p-value
External: how far do the results travel?		
One subsystem, one workload	RQ2 rests on the compilers, RQ4 on the compilers, the C ABI and the dependency graph	The performance results do not travel, the overload result least of all. RQ4's build seconds describe one machine, its ratio does not
One measurement host	Bare metal, no hypervisor, fixed governor, tickless pinned cores, and the same design re-measured on two further machines (§6.4.4)	No `isolcpus` (§6.1). Each comparison campaign changes something besides the machine, so both corroborate rather than replace

for the direction of both resource results, and the allocator's own accounting puts the resident-set difference outside its mappings (Table 6.10). The call site behind it stays open. The allocation profile taken on the same visit cannot close it, because the build that profiling requires reverses the sign of the very difference the profile was collected to explain, so what the question needs is a profile whose configuration reproduces the effect rather than a further reading of these dumps. The mechanisms most likely responsible have been isolated elsewhere [72, 73]. RQ4 measures the size of the seam, the `unsafe` it concentrates and the per-call cost, which are the right quantities for the question as posed and not a measure of how safe the composed system is. Host and component share an address space, so a compile-time property of the Rust side bounds nothing the C++ side can corrupt at run time [47, 48].

Conclusion validity: are the differences real or noise? The statistical treatment is set out in Section 4.4 and its outcomes in Table 6.3. The repeat count is chosen so that the design is not the binding constraint: an exact test on 20 pairs floors at 0.0000019, and at 0.000027 after correcting 14 comparisons, so a null here reflects the data rather than the arithmetic. An earlier pass at seven repeats did not have that property, and its corrected floor above 0.05 made every null in the table uninterpretable.

Three limitations remain. First, one property of the host is short of the protocol and it is short on purpose. The pinned cores are not isolated from the scheduler with `isolcpus`, because isolating them also removes them from every scheduling domain, and the benchmark pins its threads to a range and relies on the kernel to spread them: an isolated set collapses all of them onto one core, which is not a subtle failure but it is an invisible one, since every individual record still looks plausible. What that costs is a residue of scheduler interference on the pinned cores that a fully isolated set would not carry. It is bounded and not eliminated, by tickless operation on those cores, a fixed governor with boost off, and no hypervisor beneath them (Section 6.1). The reading rule fixed in Section 4.4 is aimed at exactly this residue: peak RSS, CPU and sustained throughput are run-level summaries and come through cleanly, while the tail percentiles are computed from precisely the operations any interference falls on, and their intervals are not read as findings for that reason. On this host the rule changes no verdict, which is a fact about the machine and not a reason to withdraw the rule. It is also why the overload comparison is made on throughput and not on latency.

Second, a corrected p-value below 0.05 is necessary here and visibly not enough, and the converse also holds. On an earlier campaign, before the two `jemalloc` builds were held to one configuration, CPU utilisation cleared the Holm correction under one allocator and reversed sign under the other. No threshold would have caught that. It was visible only because the design runs every comparison twice. In the present campaign the control withholds nothing a threshold would have promoted, and every row clears the correction. The opposite direction had a case too, on the earlier host: peak resident set size under `jemalloc` showed an interval far from zero and most of its repeats agreeing in sign against a p-value near unity, which a handful of runs reporting a resident set 31 MiB low are enough to produce. That row was read from its interval, under the rule fixed in Section 4.4 and not chosen once the disagreement appeared. The bimodality behind it was a defect in

the measurement and not in either allocator, because it affected both languages in equal numbers and appeared at every load level, and Section 6.4.4 shows it does not occur on either bare-metal host. So the rule was needed once, on a machine this study no longer uses, and it is retained and not withdrawn, because that is what a pre-commitment means.

Third, the overload divergence is measured and replicated, and its mechanism is now tested and not only argued. Isolated under the same pattern of 7 producers, the bounded queue and completion primitives do not reproduce the full-service ordering at significance ($p = 0.226$), which rules those two constructs out alone as the whole explanation even though they are where the forced-idiom log predicted contention would matter. The cause is not unknowable, and Section 6.4.2 names the candidates that remain on the C++ handoff path.

External validity: how far do the results travel? The study covers one subsystem, one workload, one concurrency structure, and one machine, and each of those bounds the reach of the conclusions differently. The safety and migration results (RQ2, RQ4) travel most easily, because they are properties of the compilers and of the C ABI and not of this subsystem. Any C++ codebase writing these defect classes gets the same compile-time outcome, and any narrow, scalar-typed interface gets a similarly cheap seam. Two of RQ4's adoption axes carry the same way, because a binary's size and a dependency graph's shape follow from the toolchain. The third does not travel as an absolute, because build seconds belong to the machine that measured them, and what carries there is the ratio between two toolchains timed alternately. The build-time *shape* is specific too, in a way worth naming. A two-file C++ library and a single Rust crate are both small, and the incremental result in particular would move on a codebase with a different header structure.

The performance results travel least. UECM was deliberately chosen to be small and partitionable, and its shared-nothing design removes lock contention from the hot path. A subsystem with genuinely shared mutable state, deep call graphs, or heavy allocation churn could therefore exercise the two languages' abstraction and allocation costs quite differently. The workload is synthetic and spec-justified, not replayed from production traffic, so its operation mix is plausible but unvalidated. Nothing here speaks to PHY or L2 processing, to the data-heavy user plane, or to the operational concerns (upgrade, observability, multi-tenancy) that a production deployment would raise.

The defensible claim is therefore narrow and carries a condition. For stateful, concurrent, partitionable control-plane services of this kind, and while they are meeting their offered load, the language choice is not a performance decision. Once they are not meeting it, the language choice is one. And the overload result is the least transferable finding in the study rather than the most, because it depends on the queue bound, on one request being outstanding per producer, and on two particular standard-library primitives. Extending either claim beyond this class means rerunning the framework, which is why the framework and not the numbers is offered as the reusable contribution.

A final limit on reach is temporal and not structural, and it applies to the comparison itself rather than to any one result. One of the two objects being compared is moving. The C++

side is pinned to C++23 and to the compiler versions recorded in Section 6.1, while the standards committee pursues enforced safety profiles (Section 2.3). That qualifies nothing measured here, because none of that work is standardised or generally implemented and the concurrency profile that would reach the boundary classes of RQ2 comes after an initial set still to be specified [38]. It does mean the RQ2 result describes these languages as they stand and not permanently. An enforced lifetime profile would narrow the memory-safety half of the catalogue and a concurrency profile would bear on the half placed outside Rust's guarantee, which is why the catalogue is designed to be re-run against a compiler that enforces them.

Replication status: which findings stand alone. Section 4.1 records that no prior study measured this comparison. The converse statement belongs here, now that the results exist: which of them the wider literature agrees with. Internal replication is designed into the campaign, in 20 interleaved repeats and in every comparison being run twice under different allocators. External replication is a separate question, and the answer is uneven. Three findings have published company. The median equivalence below saturation is the best-supported result in the study, with independent matched pairs in three settings putting safe Rust within single-digit percentages of its C or C++ counterpart [71, 102, 105]. That a cross-thread handoff rather than the operation dominates the end-to-end cost is an instance of a pattern reported for a mature shared-nothing database [65]. Most of RQ2's boundary classes are hazards the literature already documents. Three findings stand alone. No study located by the protocol of Appendix B reports a load-dependent reversal between the two languages, and the one reporting the same shape is a master's thesis attributing it to a runtime this design excludes [55]. None measures the two toolchains' build time at all, let alone with the dependency tree separated from the project's own source. The per-operation bound on the seam has no published counterpart either. Peak resident set size sits between the two groups, with the sign found here reported by two neighbouring theses and reversed by a third [52, 53, 77]. Unreplicated is not unsupported. What it sets is how much weight each result can carry alone, and the overload divergence, the largest single effect measured here, is the finding with the least external corroboration.

7.7 Ethical and professional reflection

The conduct of the study raised no data-protection questions, because it processes no personal, subscriber, or operator-confidential data. All traffic is synthetic, the contexts carry fabricated identifiers and a fixed dummy security field, and the subsystem is modelled on production code rather than copied from it. No privacy, consent, or data-retention concern arises, so the artifacts can be released openly for reproduction. The *use* of the study is a different matter and deserves stating explicitly, because a language-selection recommendation in this domain is not a neutral technical output.

Two responsibilities follow. The first concerns over-claiming. A vendor could read the RQ2 result as licence to describe a migrated component as "safe", and the boundary cases are why that reading would be wrong. A deadlocked control plane is an outage, a silently lost counter update can misinform a mobility decision, and nothing demonstrated here

prevents either. Reporting the boundary as prominently as the guarantee is a professional obligation, because the audience for this work is making a decision that affects networks people depend on.

The second concerns how far the evidence reaches. Tail-latency numbers from any single machine, this one included, are a narrower basis for a production commitment than they look, and saying so only works if the qualification travels with the numbers, which is why Table 7.1 carries an evidence status per research question and why the earlier virtualised campaign is reported rather than quietly discarded: a study that cannot show what its own instrument changed is asking to be taken on trust. A migration recommendation also carries organisational consequences that fall on people rather than systems: retraining, hiring, and a period of reduced velocity. The cost moved from run time to human effort rather than disappearing.

8 Conclusion and Future Work

This thesis compared modern C++ and Rust on a UE Context Management service modelled on a real Nokia gNB-CU-CP component, along four dimensions: tail-latency predictability, compile-time defect prevention, safety-model overhead, and incremental-adoption cost. Built against a single frozen specification and driven through identical code paths, the two implementations isolate language effects from design and skill differences.

Three conclusions follow from the evidence. First, on safety the difference is one of *kind*, and it has a clear boundary. Safe Rust rejects all seven injected memory-safety and data-race classes at compile time, and one of those seven is a class no C++ sanitiser reports at all. Hazards C++ can only hope to catch under an instrumented test become unwriteable. The edge stops there. All seven boundary classes compile in safe Rust and all seven misbehave, though not always in the same way as in C++, and on two of them the C++ tooling is the one that reports. The guarantee is therefore best described precisely. Rust eliminates data races, not race conditions, and aliasing errors, not lock-ordering errors, leaks, wrapping arithmetic or fault containment, and it eliminates none of them in a region marked `unsafe`. Where the boundary defects are caught, they are caught at test time, in two of the three cases only after being rewritten into a model checker's primitives, which is a real cost and a different one from the sanitiser's.

Second, on migration the cost is dominated by API design and the learning curve, not by any run-time tax. An unmodified Rust core can be driven from C++ across a thin, `unsafe`-isolated seam whose cost on a real operation, measured against the same operation called natively, is bounded at 10.91%. The objections a build engineer raises first do not change that ordering. Most of the Rust build's extra expense is compiling dependencies and not the compiler's own work, the binary-size gap is a linkage policy and the ordering depends on which matched policy is chosen, Rust smaller when both standard libraries are linked statically and C++ smaller when both are resolved dynamically. The compared component resolves no third-party code at all. Where C++ leads is the edit-compile loop, and it leads only for edits that stay inside a translation unit.

Third, on performance the evidence divides twice before it divides by resource: first by *what is being measured*, and then by *whether the service is coping*. An end-to-end operation in this design is 0.250% the work the language performs, and the rest is a cross-thread handoff the operating system performs, so an end-to-end null cannot bound a language effect at all. Measured on the operation alone the languages are not equivalent. Rust is the faster of the two, by 1.6×, for reasons that trace to the standard library's hash-map layout and not to anything about safety.

The second of those divisions is the more consequential. Below saturation the two languages are equivalent at the median for any purpose a deployment would care about, and that is a real result about deployments of this shape. The equivalence is practical rather than statistical, since the paired test does resolve 1.3 μs in Rust's favour under both allocators. Their far tail resolves too, and it resolves in Rust's favour: at p99.99, which is where a deployment sizes its margin, the slowest Rust variant sits 50 μs below the

fastest C++ one. Past saturation they are not equivalent, and the two implementations differ both in where they stop coping and in how they behave once both have. The Rust arms hold their offered rate to 448 kops/s against the C++ arms' 392. Beyond that, the C++ build sustains 1.7× the throughput of the Rust build, in 20/20 interleaved repeats under both allocators, while the Rust build spends 1.8× the CPU on each operation it completes, so what it loses is not idleness but effort that finishes nothing. The isolated primitives do not reproduce that ordering at significance (Section 6.4.2). On memory the Rust build carries a few MiB more fixed footprint under both allocators, in the same direction and the same order of magnitude on every machine measured, while the cost of the allocator choice moved by more than tenfold between them.

Taken together these support a narrow claim, narrower than the one this thesis expected to make. For stateful, concurrent, partitionable control-plane services of this kind, and *at nominal load*, the choice between modern C++ and Rust is not a performance decision. It is a decision about which classes of defect an organisation wants to make unwriteable, and about whether it will pay the engineering and learning cost of getting them. Around and past saturation it becomes a performance decision again, and which way it goes depends on where the service is expected to sit: the Rust build meets its offered rate a rung further up, and deeper into overload the C++ build sustains more of what it is given. The reasons lie in the concurrency plumbing and not in either language's guarantees, which a service required to degrade gracefully cannot treat as incidental. What this thesis adds is therefore an instrument and not a verdict: the frozen specification, the parity-tested pair, a measurement framework that reaches past the saturation knee, a decomposition that says how much of what it measures the language is answerable for, and a defect catalogue that maps where the guarantee stops.

Several directions extend the work, each the direct remedy to a limitation named above. The immediate priority needs no new machine, which is why it is first: vary the transparent-hugepage policy on one host with everything else held fixed. Three hosts measured the peak-RSS comparison and all three agree that the Rust build carries the larger footprint, while they disagree about how that compares against the cost of the allocator, which the virtualised host put an order of magnitude higher and neither bare-metal host does (Section 6.4.4). The hugepage policy is the candidate this study can name but not test, because it differed between those hosts and only the system-allocator arm inherits it. A single machine, two settings, everything else pinned, and the question of whether a footprint claim of this kind belongs to an allocator or to a kernel setting is settled either way. Beyond that:

- Replay captured, anonymised production traces in place of the synthetic workload, so that the operation mix is validated rather than merely spec-justified.
- Scale the comparison to further control-plane subsystems, or to the data-heavy L2/PHY path, where shared mutable state and allocation churn would exercise the two languages differently.
- Extend the fault catalogue along the boundary mapped here, through misuse of `unsafe` and through the boundary classes under a whole-program model checker

rather than a hand-shrunk one. A checker that also covers a program's foreign dependencies [51] would reach the seam of Section 6.4.6, whose interleavings this study does not explore.

- Evaluate Rust async runtimes against the thread-per-shard model used here, under the same controls. That would separate the tail divergence Arvidsson and Kaller attribute to stackless `async`/`await` [55] from the language itself, and settle how much of the DPDK gap Olsson and Stenström measured [54] belongs to the runtime.
- Measure what in-process isolation of the two heaps costs at the seam, since an `unsafe`-free core needs protecting as well as verifying [47, 48].
- Track defect density and review effort as Rust is adopted incrementally in a production RAN codebase.

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5, used through the Cursor development environment, is the only generative AI system used in preparing this thesis, and it was used throughout, from June 2026 until submission. It is named neither as an author, which University of Helsinki policy forbids, nor as a source, which is the author's own choice.

What the assistance covered. It was extensive. The model drafted and revised the running prose and proposed the order in which the arguments are made. It wrote and refactored both implementations of the UE Context Management service, the load generator, the fault-injection catalogue, the interoperability prototype, the analysis code behind the bootstrap intervals and the paired permutation tests, and the figure-generating, build and LaTeX sources. It also suggested candidate literature. The abstract is the one part it must not touch: University of Helsinki policy never permits a language model in a maturity test, and in this programme the maturity test is the abstract. In this draft it has not yet been rewritten unaided, and this statement will say so until it has been.

What the model did not settle. The research problem, the four research questions, the three hypotheses, and the control on which the study's value rests are the author's. That control is one frozen language-neutral specification and one concurrency model, so language remains the only systematic difference between the two implementations. The author also chose to carry defect classes from outside Rust's compile-time guarantee, which is what keeps RQ2 falsifiable. Every measurement was run by the author, on the machines each result names, and the quantities the thesis quotes reach the page as macros regenerated from the raw logs rather than as figures typed into a paragraph. The readings are the author's too, including the restraints the evidence imposes: a sweep dominated by cross-thread handoff cannot bound an effect of the language, an unresolved comparison is not a demonstration of equivalence, and an operating point noticed after the data was in hand is described rather than tested.

How the output was checked. Model output was treated as a draft to be falsified. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body. Most were then graded by how far the claim made of it could be traced: to the page or clause itself, to a publisher-deposited record carrying it, or no further than confirmed metadata behind a paywall. The audit records that grade per entry, so the entries whose supporting passage was never opened are listed rather than absorbed, and it records which entries were added after the grading sweep and carry verified metadata only. It found four passages that overstated or misplaced their source, one a quotation absent from the work it was attributed to. All four were corrected, and the audit is kept beside the artefacts. Code was checked by running it, against a conformance suite whose transcripts are diffed line by line and a fault catalogue that fixes each expected outcome before the defect is injected.

What the author took from it. The model is dependable at mechanical work, at transposing an idiom between two languages or keeping a build honest. It is undependable in one predictable way: it will supply a plausible citation for a claim and a confident

sentence where the data supports a hedge. Its fluency is not evidence. The effort saved in drafting is spent again in checking, and the defensible parts of this thesis are the parts checked against a source, a specification, or a measurement.

Responsibility. The author is responsible for the whole of this thesis. Where it is wrong, the error is the author's.

Bibliography

[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation*. BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond*. Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[13] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[14] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[15] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[16] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[17] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[18] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[19] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[20] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[21] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. "Firecracker: Lightweight Virtualization for Serverless Applications". In: *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2020, pp. 419–434. URL: https://www.usenix.org/conference/nsdi20/presentation/agache.

[22] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[23] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[24] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[25] Cybersecurity and Infrastructure Security Agency. *Product Security Bad Practices*. Joint Guidance. Version 2.0. Bad practice 1 covers the development of new product lines in memory-unsafe languages and the memory-safety roadmap expectation, from which only products with an announced end-of-support date before 1 January 2030 are exempt. CISA and FBI, Jan. 2025. URL: https://www.cisa.gov/resources-tools/resources/product-security-bad-practices.

[26] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[27] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[28] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[29] 3GPP. *NG-RAN; E1 Application Protocol (E1AP)*. Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip.

[30] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble. "Tales of the Tail: Hardware, OS, and Application-level Sources of Tail Latency". In: *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. 2014, 9:1–9:14. DOI: 10.1145/2670979.2670988. URL: https://doi.org/10.1145/2670979.2670988.

[31] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[32] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[33] L. Szekeres, M. Payer, T. Wei, and D. Song. "SoK: Eternal War in Memory". In: *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*. 2013, pp. 48–62. DOI: 10.1109/SP.2013.13. URL: https://doi.org/10.1109/SP.2013.13.

[34] S. Bensalem and K. Havelund. "Dynamic Deadlock Analysis of Multi-threaded Programs". In: *Hardware and Software, Verification and Testing (Haifa Verification Conference)*. Vol. 3875. Lecture Notes in Computer Science. Springer, 2005, pp. 208–223. DOI: 10.1007/11678779_15. URL: https://doi.org/10.1007/11678779_15.

[35] The Sanitizers Project. *ThreadSanitizer deadlock detector*. Project documentation. Accessed August 2026. 2026. URL: https://github.com/google/sanitizers/wiki/ThreadSanitizerDeadlockDetector.

[36] H. Sutter. *Core Safety Profiles for C++26*. ISO/IEC JTC1/SC22/WG21 document P3081R2. Feb. 2025. URL: https://isocpp.org/files/papers/P3081R2.pdf.

[37] S. Baxter and C. Mazakas. *Safe C++*. ISO/IEC JTC1/SC22/WG21 document P3390R0. Sept. 2024. URL: https://safecpp.org/P3390R0.html.

[38] D. Vandevoorde, J. Garland, P. E. McKenney, R. Orr, B. Stroustrup, and M. Wong. *Profiles and Safety: A Call to Action*. ISO/IEC JTC1/SC22/WG21 document P3970R0. Jan. 2026. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3970r0.pdf.

[39] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "Safe Systems Programming in Rust". In: *Communications of the ACM* 64.4 (2021), pp. 144–152. DOI: 10.1145/3418295. URL: https://doi.org/10.1145/3418295.

[40] R. Jung, H.-H. Dang, J. Kang, and D. Dreyer. "Stacked Borrows: An Aliasing Model for Rust". In: *Proceedings of the ACM on Programming Languages* 4.POPL (2020), 41:1–41:32. DOI: 10.1145/3371109. URL: https://doi.org/10.1145/3371109.

[41] V. Astrauskas, C. Matheja, F. Poli, P. Müller, and A. J. Summers. "How Do Programmers Use Unsafe Rust?" In: *Proceedings of the ACM on Programming Languages* 4.OOPSLA (2020), 136:1–136:27. DOI: 10.1145/3428204. URL: https://doi.org/10.1145/3428204.

[42] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642.

[43] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.

[44] W. Bugden and A. Alahmar. "Rust: The Programming Language for Safety and Performance". In: *arXiv preprint arXiv:2206.05503* (2022). DOI: 10.48550/arXiv.2206.05503. URL: https://doi.org/10.48550/arXiv.2206.05503.

[45] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[46] M. Škoudlil, M. Sojka, and Z. Hanzálek. "A First Look at ROS 2 Applications Written in Asynchronous Rust". In: *37th Euromicro Conference on Real-Time Systems (ECRTS 2025)*. Vol. 335. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 1:1–1:21. DOI: 10.4230/LIPIcs.ECRTS.2025.1. URL: https://doi.org/10.4230/LIPIcs.ECRTS.2025.1.

[47] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[48] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.

[49] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[50] X. Cai, J. Liu, C. Liu, L. Bao, Y. Yu, and L. Jiang. "Fortifying the Seams Between C/C++ and Rust: Characterizing Bugs in Interop Tools". In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3795532. URL: https://doi.org/10.1145/3795532.

[51] O. Pearce, J. Lange, and D. O'Keeffe. "RustMC: Automated Verification of Real-World Concurrent Rust". In: *Formal Techniques for Distributed Objects, Components, and Systems (FORTE)*. Lecture Notes in Computer Science. Springer, 2026, pp. 235–255. DOI: 10.1007/978-3-032-28187-6_14. URL: https://doi.org/10.1007/978-3-032-28187-6_14.

[52] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[53] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[54] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[55] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[56] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[57] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[58] S. Michaelides, J. Lapawa, D. Eguiguren Chavez, and M. Henze. "Evaluation of Security-Induced Latency on 5G RAN Interfaces and User Plane Communication". In: *Proceedings of the 19th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec '26)*. 2026, pp. 176–181. DOI: 10.1145/3765613.3797453. URL: https://doi.org/10.1145/3765613.3797453.

[59] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[60] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7.

[61] C. Demichelis and P. Chimento. *IP Packet Delay Variation Metric for IP Performance Metrics (IPPM)*. Request for Comments (RFC) 3393. Standards Track. Defines packet delay variation as the difference between consecutive one-way delays. Internet Engineering Task Force (IETF), Nov. 2002. DOI: 10.17487/RFC3393. URL: https://www.rfc-editor.org/rfc/rfc3393.

[62] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[63] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[64] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[65] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[66] The LLVM Project. *clang-tidy check `bugprone-use-after-move`*. Extra Clang Tools documentation. Accessed August 2026. 2026. URL: https://clang.llvm.org/extra/clang-tidy/checks/bugprone/use-after-move.html.

[67] J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179.

[68] M. Cui, H. Xu, H. Tian, and Y. Zhou. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624.

[69] W. Dietz, P. Li, J. Regehr, and V. Adve. "Understanding Integer Overflow in C/C++". In: *ACM Transactions on Software Engineering and Methodology* 25.1 (2015), 2:1–2:29. DOI: 10.1145/2743019. URL: https://doi.org/10.1145/2743019.

[70] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[71] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[72] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494.

[73] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39.

[74] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[75] R. Tang, M. Wang, X. Sun, L. Huang, Y. Huang, and X. Ma. "Converos: Practical Model Checking for Verifying Rust OS Kernel Concurrency". In: *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association, 2025, pp. 1143–1159. URL: https://www.usenix.org/conference/atc25/presentation/tang.

[76] S. Lu, S. Park, E. Seo, and Y. Zhou. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323.

[77] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[78] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[79] W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[80] H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu. "Rust Meets Linux: Lessons from an Evolving Experiment". In: *ACM Transactions on Computer Systems* 44.2 (2026). Extended version of the authors' USENIX ATC '24 study of Rust-for-Linux, 9:1–9:33. DOI: 10.1145/3789260. URL: https://doi.org/10.1145/3789260.

[81] Y. Fan, C. Wang, Z. Dong, T. Liu, and H. Wang. "An Empirical Analysis of Rust Integration in Android Open Source Project". In: *Companion Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE Companion)*. 2026, pp. 545–556. DOI: 10.1145/3803437.3805227. URL: https://doi.org/10.1145/3803437.3805227.

[82] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[83] A. Levy, B. Campbell, B. Ghena, P. Pannuto, P. Dutta, and P. Levis. "The Case for Writing a Kernel in Rust". In: *Proceedings of the 8th Asia-Pacific Workshop on Systems (APSys)*. 2017, pp. 1–7. DOI: 10.1145/3124680.3124717. URL: https://doi.org/10.1145/3124680.3124717.

[84] C. Mo, L. Wang, S. Li, K. Hu, and B. Jiang. "Rust-Shyper: A Reliable Embedded Hypervisor Supporting VM Migration and Hypervisor Live-Update". In: *Journal of Systems Architecture* 142 (2023), p. 102948. DOI: 10.1016/j.sysarc.2023.102948. URL: https://doi.org/10.1016/j.sysarc.2023.102948.

[85] Y. Xiao, Y. Chen, X. Zhao, J. Duan, X. Zhang, W. Li, and X. Tao. "rpkt: A Generic, Safe, and Efficient Userspace Packet Processing Library in Rust". In: *Network and Parallel Computing (NPC 2024)*. Lecture Notes in Computer

Science. 2025, pp. 454–465. DOI: 10.1007/978-981-96-2864-3_36. URL: https://doi.org/10.1007/978-981-96-2864-3_36.

[86] C. Cui and H. Xu. "Unleashing the Efficiency of Rust: An Empirical Study of Performance Bugs in Rust Projects". In: *Proceedings of the 2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*. 2025, pp. 371–381. DOI: 10.1109/ISSRE66568.2025.00045. URL: https://doi.org/10.1109/ISSRE66568.2025.00045.

[87] M. Naylor, A. Joannou, A. T. Markettos, P. Metzger, S. W. Moore, and T. M. Jones. "CHERI-SIMT: Implementing Capability Memory Protection in GPUs". In: *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2026, pp. 50–66. DOI: 10.1145/3760250.3762234. URL: https://doi.org/10.1145/3760250.3762234.

[88] Y. Bae, Y. Kim, A. Askar, J. Lim, and T. Kim. "Rudra: Finding Memory Safety Bugs in Rust at the Ecosystem Scale". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 84–99. DOI: 10.1145/3477132.3483570. URL: https://doi.org/10.1145/3477132.3483570.

[89] X. Zheng, Z. Wan, Y. Zhang, R. Chang, and D. Lo. "A Closer Look at the Security Risks in the Rust Ecosystem". In: *ACM Transactions on Software Engineering and Methodology* 33.2 (2024), 34:1–34:30. DOI: 10.1145/3624738. URL: https://doi.org/10.1145/3624738.

[90] B. Qin, Y. Chen, H. Liu, H. Zhang, Q. Wen, L. Song, and Y. Zhang. "Understanding and Detecting Real-World Safety Issues in Rust". In: *IEEE Transactions on Software Engineering* 50.6 (2024), pp. 1306–1324. DOI: 10.1109/TSE.2024.3380393. URL: https://doi.org/10.1109/TSE.2024.3380393.

[91] L. Schuermann, A. Thomas, and A. Levy. "Encapsulated Functions: Fortifying Rust's FFI in Embedded Systems". In: *Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification (KISV)*. 2023, pp. 41–48. DOI: 10.1145/3625275.3625397. URL: https://doi.org/10.1145/3625275.3625397.

[92] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf. "Aliasing Limits on Translating C to Safe Rust". In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1 (2023), pp. 551–579. DOI: 10.1145/3586046. URL: https://doi.org/10.1145/3586046.

[93] A. Sharma, S. Sharma, S. R. Tanksalkar, S. Torres-Arias, and A. Machiry. "Rust for Embedded Systems: Current State and Open Problems". In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024, pp. 2296–2310. DOI: 10.1145/3658644.3690275. URL: https://doi.org/10.1145/3658644.3690275.

[94] I. McCormack, T. Dougan, S. Estep, H. Hibshi, J. Aldrich, and J. Sunshine. "A Mixed Methods Study on the Implications of Unsafe Rust for Interoperation, Encapsulation, and Tooling". In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3798277. URL: https://doi.org/10.1145/3798277.

[95] J. Hong and S. Ryu. "Automatically Translating C to Rust". In: *Communications of the ACM* 68.11 (2025), pp. 58–65. DOI: 10.1145/3737696. URL: https://doi.org/10.1145/3737696.

[96] J. Huang, B. Mozafari, G. Schoenebeck, and T. F. Wenisch. "A Top-Down Approach to Achieving Performance Predictability in Database Systems". In: *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*. 2017, pp. 745–758. DOI: 10.1145/3035918.3064016. URL: https://doi.org/10.1145/3035918.3064016.

[97] Y.-S. Liu, S. Qi, P.-Y. Lin, H.-S. Tsai, K. K. Ramakrishnan, and J.-C. Chen. "L^{2}5GC+: An Improved, 3GPP-Compliant 5G Core for Low-Latency Control Plane Operations". In: *Proceedings of the 2023 IEEE 12th International Conference on Cloud Networking (CloudNet)*. 2023, pp. 203–211. DOI: 10.1109/CloudNet59005.2023.10490024. URL: https://doi.org/10.1109/CloudNet59005.2023.10490024.

[98] C. Park and J. Nam. "CtrlBench: Towards Accurate API-Level Benchmarking in 5G Core Control Plane". In: *Proceedings of the 2025 16th International Conference on Information and Communication Technology Convergence (ICTC)*. 2025, pp. 1832–1837. DOI: 10.1109/ICTC66702.2025.11388531. URL: https://doi.org/10.1109/ICTC66702.2025.11388531.

[99] L. Ardito, L. Barbato, R. Coppola, and M. Valsesia. "Evaluation of Rust Code Verbosity, Understandability and Complexity". In: *PeerJ Computer Science* 7 (2021), e406. DOI: 10.7717/peerj-cs.406. URL: https://doi.org/10.7717/peerj-cs.406.

[100] Z. A. Qazi, M. Walls, A. Panda, V. Sekar, S. Ratnasamy, and S. Shenker. "A High Performance Packet Core for Next Generation Cellular Networks". In: *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. 2017, pp. 348–361. DOI: 10.1145/3098822.3098848. URL: https://doi.org/10.1145/3098822.3098848.

[101] N. Larkin. *Alsoran: A 5G gNodeB Centralized Unit Written in Rust*. Open-source software repository. Proof of concept, development discontinued. Accessed August 2026. URL: https://github.com/nplrkn/alsoran.

[102] Z. Li, T. Huang, V. Narayanan, and A. Burtsev. "Understanding the Overheads of Hardware and Language-Based IPC Mechanisms". In: *Proceedings of the 11th Workshop on Programming Languages and Operating Systems (PLOS)*. 2021, pp. 53–61. DOI: 10.1145/3477113.3487275. URL: https://doi.org/10.1145/3477113.3487275.

[103] V. Narayanan, T. Huang, D. Detweiler, D. Appel, Z. Li, G. Zellweger, and A. Burtsev. "RedLeaf: Isolation and Communication in a Safe Operating System". In: *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2020, pp. 21–39. URL: https://www.usenix.org/conference/osdi20/presentation/narayanan-vikram.

[104] F. Suchert and J. Castrillon. "STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks". In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10.

[105] A. Gonzalez, D. Mvondo, and Y.-D. Bromberg. "Takeaways of Implementing a Native Rust UDP Tunneling Network Driver in the Linux Kernel". In: *Proceedings of the 12th Workshop on Programming Languages and Operating Systems (PLOS)*. 2023, pp. 18–25. DOI: 10.1145/3623759.3624547. URL: https://doi.org/10.1145/3623759.3624547.

[106] K. Zhuang. "FlowPrioritize: A Model-Driven Priority Scheduling Library for Asynchronous Rust Runtimes". In: *SoftwareX* 35 (2026), p. 102771. DOI: 10.1016/j.softx.2026.102771. URL: https://doi.org/10.1016/j.softx.2026.102771.

[107] O. Andersson. "Programming Language Standards and Their Impact on Memory Safety: C++, Rust and Python". Report LIU-IDA/LITH-EX-G–26/057–SE. Bachelor's thesis. Linköping University, Department of Computer and Information Science, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-226479.

[108] S. Okawa and S. Yamaguchi. "A Performance Study on Rust and C Programs". In: *Proceedings of the 12th International Symposium on Computing and Networking Workshops (CANDARW)*. 2024, pp. 355–359. DOI: 10.1109/CANDARW64572.2024.00066. URL: https://doi.org/10.1109/CANDARW64572.2024.00066.

[109] S. Pirelli and G. Candea. "Safe Low-Level Code Without Overhead is Practical". In: *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2023, pp. 2173–2184. DOI: 10.1109/ICSE48619.2023.00183. URL: https://doi.org/10.1109/ICSE48619.2023.00183.

Appendix A Abbreviations and key terms

The abbreviations and terms used throughout the thesis, for readers outside the telecom domain.

Table A.1: Abbreviations and key terms used throughout the thesis.

Term	Meaning
RAN	Radio Access Network: the part of the mobile network connecting devices to the core
NG-RAN	Next-Generation RAN, the 5G radio-access-network architecture
IMT-2030	The ITU framework for the generation after 5G, commonly called 6G
AI-RAN	An architecture in which AI and RAN workloads share one accelerated infrastructure
gNB	5G base station (“next-generation NodeB”)
gNB-CU / DU	Central Unit / Distributed Unit: the disaggregated halves of a gNB
gNB-CU-CP	Control-plane part of the central unit: the system class studied here
UE	User Equipment: an end device (phone, modem) attached to the network
UECM	UE Context Management: the per-UE state-management subsystem reimplemented here
RRC	Radio Resource Control: the control-plane signalling protocol between UE and gNB
DRB	Data Radio Bearer: a per-UE user-plane data channel set up during a session
Control plane	The signalling and management path, as opposed to the user-plane data path
Tail latency	High-percentile latency (p99, p99.9, p99.99), the worst-case-oriented timing metric
Jitter	How much latency varies from one operation to the next, as opposed to its typical value
Saturation	The offered load beyond which the service can no longer keep up and requests start queueing
Shard	One of several independent slices of the UE state, each owned by exactly one worker thread
FFI	Foreign Function Interface: calling Rust from C++ across a C ABI
HDR histogram	High-Dynamic-Range histogram, used for accurate latency capture

Appendix B Literature search protocol

Section 4.1 reports the two findings this search produced. The protocol is recorded here so that the gap the study claims can be checked rather than taken on trust.

Candidate literature was gathered from the ACM Digital Library, IEEE Xplore, USENIX, and Google Scholar, using combinations of the terms *Rust*, *C++*, *memory safety*, *tail latency*, *systems programming*, *performance comparison*, and *RAN / 5G control plane*. The search was restricted to peer-reviewed venues, except where an industry or government report is itself the object of study, as the memory-safety mandates and vendor vulnerability statistics are. The result set was then extended by backward and forward citation chaining from the core Rust-safety papers, and filtered to work that either measures a systems language empirically, formalises Rust's guarantees, or reports adoption experience. The domain literature (3GPP specifications and the O-RAN survey) was added separately to fix the architectural context.

Those sources alone would have missed the closest prior work, which is why two more were added. The vendor-hosted theses that come nearest to this study (Section 7.5) appear in none of the citation databases: not in Crossref, OpenAlex, DBLP, or Semantic Scholar. Neither forward nor backward chaining can reach them, because nothing indexed cites them and their own reference lists are not deposited. They were found instead by searching the institutional repositories directly, DiVA for the Nordic universities and Finna for the Finnish ones. The same two searches establish the absence on the other side. Finna, which federates the Finnish university repositories, returns no thesis on Rust in a radio-access or telecom context. An author-affiliation search of Nokia's indexed publications in OpenAlex returns none either. Its entities hold on the order of thirty thousand indexed works, several hundred of which concern the radio access network, and one mentions Rust, in an unrelated preprint on trusted-execution monitoring.

One practical note belongs with the protocol, since it distorts a naive search. Free-text queries for *Rust* in the general indexes rank plant pathology and corrosion chemistry above the programming language, so field-restricted or discipline-restricted queries are necessary.

Appendix C Statistical design: family membership and resolving power

Section 4.4 fixes which comparisons enter the corrected family and how far into a difference the design can see. Three points behind those commitments are recorded here rather than in the running text.

Why an unresolvable comparison is not free. Testing achieved throughput below saturation would re-measure the operating point and not the code. It would also cost the comparisons that can resolve something, because Holm divides its budget across the whole family and a test that separates nothing still takes a share of it. Above saturation the same reasoning runs the other way. What each variant still sustains is pinned to nothing there, so excluding it would discard the only observable overload offers.

Why no percentile is tested in overload. Under a fixed intended send schedule, a producer that cannot keep up falls behind cumulatively, so a percentile measured there reports accumulated backlog against a window that cuts it off. Attaching a p-value to a lower bound would test the harness and not the software.

The floors the design can reach. An exact test on n pairs cannot return a two-sided p below $2/2^n$, and after correcting a family of m comparisons the smallest attainable adjusted value is m times that. That is where the 0.0000019 and 0.000027 of Section 4.4 come from. A campaign whose corrected floor lies above the threshold being applied cannot demonstrate significance however large the effect, so reporting "not significant" from such a campaign would say more about its repeat count than about the languages.

Appendix D Host and campaign configuration

Table D.1 itemises the three machines the campaigns ran on: the bare-metal host of Section 6.1 that every performance and resource table of the main sweep is drawn from, the virtualised host it replaced, and the second bare-metal host at a different design that Section 6.4.4 reads against it. It is the host every performance and resource table of the main sweep is drawn from, and no other table depends on it. The resident-set decomposition belongs to the second bare-metal host. The fault catalogue was compiled and run on the development machine, whose toolchain its own caption records, and its outcomes are categorical and do not move with load. Of the adoption-cost axes only build time is host-sensitive, which is why that one is reported as a paired ratio while the footprint and dependency figures are deterministic. And the three small paired experiments predate the move to bare metal and carry no environment record, so each supports a ratio between two arms measured alternately in one session and is quoted that way. Every column reports what was measured, and the protocol they are read against is published as `bench/TESTBED.md` rather than restated here. Table D.2 gathers the configuration of the service, the workload and the campaign, whose derived values Section 6.2 quotes.

Table D.1: The three machines the campaigns ran on, each reported as measured. Every hardware, kernel and toolchain cell is generated from that campaign's own environment capture rather than restated from the protocol of `bench/TESTBED.md`. The last two rows are prose, because what an instrument set is adequate for is a judgement and not a captured field. The columns are arranged so that each adjacent pair isolates one change. Reported sweep against virtualised host: the same design on a different machine, which is what Section 6.4.4 reads a hardware effect from. Reported sweep against second bare-metal host: a different shard count, load grid, repeat count and compiler on comparable hardware, which is what tests whether the findings depend on those choices. Rows where all three agree describe the source or the method rather than any machine. The *Short of protocol* marks are deliberate and are argued where they appear.

Aspect	Reported sweep	Virtualised host, same design	Second bare-metal host, different design
Host	AMD EPYC 7443, 24 cores, 1 socket	Intel(R) Xeon(R) CPU E5-2680 v4 guest, 16 vCPUs	AMD EPYC 7443P, 24 cores, 1 socket
Hypervisor	none. Governor performance, boost off, verified on the machine	kvm. Frequency and turbo host-controlled, unobservable from inside	none. Governor performance, verified on the machine
CPU steal	None: no hypervisor to lose a timeslice to	Not characterised. The capture's single one-second sample read 0, and no steal probe was run on this host	0 of 60 intervals on 16 pinned cores
Isolation	`nohz_full` 2-17. `isolcpus` none, omitted on purpose (§7.6)	`isolcpus` none, `nohz_full` none	`isolcpus` none, `nohz_full` none. *Short of protocol*
Hugepages	madvise	always, and only the system-allocator arm inherits it	madvise
Toolchain	`g++ 15.2.1`, `rustc 1.97.1`, pinned in a container image	`g++ 15.1.1`, `rustc 1.97.1`	`g++ 13.3.0`, `rustc 1.98`, so a compiler change is tested too
Pinning	Cores 2–17, 7 shards, 7 producers	Cores 2–15, 7 shards, 7 producers	Cores 2–17, 8 shards, 8 producers
SMT	off, verified	Absent from the topology the guest is shown, and decided on the host	off, verified
Repeats	20 over 14 loads, 1120 runs	20 over 14 loads, 1120 runs	15 over 8 loads, 480 runs. *Short of protocol*
Instruments	Counters, pooled HDR sidecars, jitter, allocator accounting	None available in the guest	Counters at two scopes, HDR sidecars, allocator accounting
Adequate for	Every result reported here	Everything except the far tail below saturation	Confirming each finding survives a change of design

Table D.2: Configuration of the service under test, the workload driving it, and the measurement campaign. Service parameters are fixed by the frozen specification, with one exception: the shard count is set to match the cores the host offers (Section 6.1). All of them, that one included, are identical in both languages. Campaign figures are generated from the measurement logs together with the tables that report them. The representative load is fixed rather than derived, for the reason Section 6.2 gives, and the throughput plateau is the lowest of the four variants' own plateaus, which differ.

Parameter	Value	Note
Service under test, identical in both languages		
Shards, and worker threads	7	one thread owns one shard. Identical in both languages, which is what the fairness control requires (§4.3)
Queue bound per shard	1 024 requests	bounded and blocking, so overload is observable
Initial map capacity	4 096 entries	reserved so steady state does not rehash
Bearers per context	at most 8	

Appendix E Fault-injection example

As an illustration of the fault-injection study (Chapter 6, RQ2), the use-after-free defect is shown in both languages. In C++ the alias into the context map survives the `erase` that frees it, so the following program compiles cleanly and is flagged only at runtime, by AddressSanitizer and ThreadSanitizer:

```
std::unordered_map<unsigned long, UeContext> ctxs;
ctxs[1001] = UeContext{1001, 0};
UeContext* p = &ctxs.at(1001);  // alias into the container
ctxs.erase(1001);               // frees the node that p refers to
p->messages_handled += 1;       // use-after-free
```

The analogous safe-Rust program does not compile: the borrow checker forbids mutating the map (`remove`) while the reference `p` into it is still live, so the defect cannot be written in the first place (error `E0499`):

```
let mut ctxs: HashMap<u64, UeContext> = HashMap::new();
ctxs.insert(1001, UeContext { messages_handled: 0 });
let p = ctxs.get_mut(&1001).unwrap(); // mutable borrow starts
ctxs.remove(&1001);                   // borrow while p live -> rejected
p.messages_handled += 1;              // p used after the remove
```